

Informatik 2 – Rekursionen

+

K

A



©image: various



Heute

- Fraktale
- Iteration
- Rekursion
 - Türme von Hanoi
 - Baumausgabe

Fraktale



Fraktale



Romanesco-Broccoli

Die wichtigsten Merkmale fraktaler Strukturen sind: [1]



1. Selbstähnlichkeit

Mandelbrots Definition:

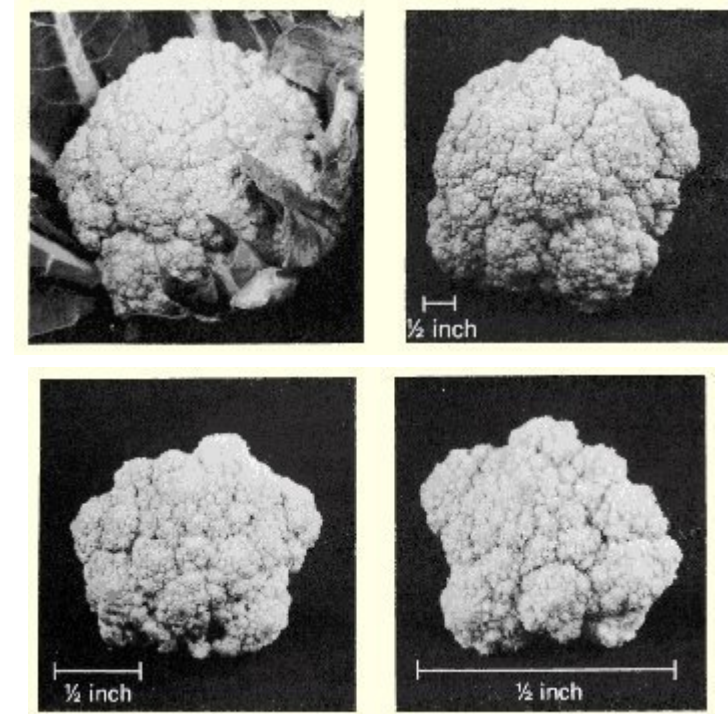
"When each piece of a shape is geometrically similar to the whole, both the shape and the cascade that generate it are called self-similar."

2. Skaleninvarianz

Absolutwerte sind irrelevant; ein Fraktal kann theoretisch unendlich oft vergrößert werden, ohne dass eine Veränderung der Form bemerkbar wird.

3. eine gebrochene Dimension:

Da das Fraktal nicht einen Teil der Ebene bzw. des Raumes voll ausfüllt* spricht man von einer gebrochenen oder fraktalen Dimension, die nicht ganzzahlig ist.



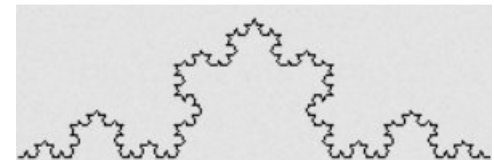
[1] <http://michael.szell.net/fba/kapitel1.html>
<https://de.wikipedia.org/wiki/Monsterkurve>
<https://de.wikipedia.org/wiki/Koch-Kurve>

- Ein Großteil der Natur aus Fraktalen aufgebaut.
- Beispiele aus der Natur sind z.B. Küstenlinien, Wolken oder Organe wie die Lunge.
- Das Wort „Fraktal“ wurde in den 1960er vom französischen Mathematiker Benoit B. Mandelbrot eingeführt.
- Das Wort leitet es vom lateinischen Verb "frangere" und vom zugehörigen Adjektiv "fractal" ab. Das Verb bedeutet brechen, Fragmente bilden.
- Sie stehen in enger Verbindung mit der sogenannten Chaostheorie.

Kochkurve



- Schwedischen Mathematiker Helge von Koch 1904
- Eines der Ersten formal beschriebenen fraktalen Objekte.
- Eines der am häufigsten zitierten Beispiele für ein Fraktal.
- Wurde bei der Entdeckung als Monsterkurve bezeichnet (überall stetige, aber nirgends differenzierbare Kurve).
- Die Koch-Kurve ist auch in Form der kochschen Schneeflocke bekannt, die durch geeignete Kombination dreier Koch-Kurven entsteht.



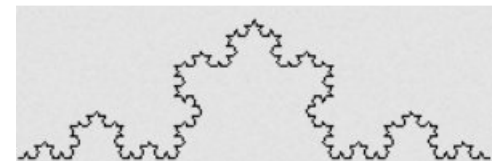
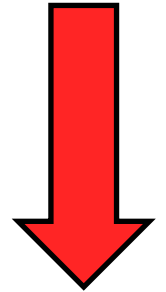
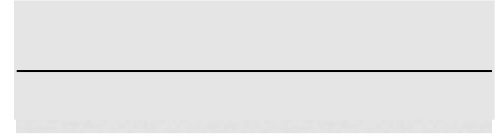
<http://michael.szell.net/fba/kapitel1.html>
<https://de.wikipedia.org/wiki/Monsterkurve>
<https://de.wikipedia.org/wiki/Koch-Kurve>



Kochkurve

- Schwedischen Mathematiker Helge von Koch 1904
- Eines der Ersten formal beschriebenen fraktalen Objekte.
- Eines der am häufigsten zitierten Beispiele für ein Fraktal.
- Wurde bei der Entdeckung als Monsterkurve bezeichnet (überall stetige, aber nirgends differenzierbare Kurve).
- Die Koch-Kurve ist auch in Form der kochschen Schneeflocke bekannt, die durch geeignete Kombination dreier Koch-Kurven entsteht.

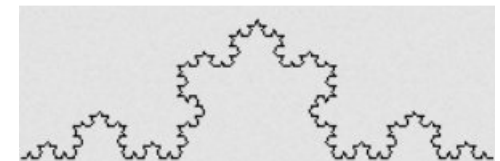
<http://michael.szell.net/fba/kapitel1.html>
<https://de.wikipedia.org/wiki/Monsterkurve>
<https://de.wikipedia.org/wiki/Koch-Kurve>



Kochkurve



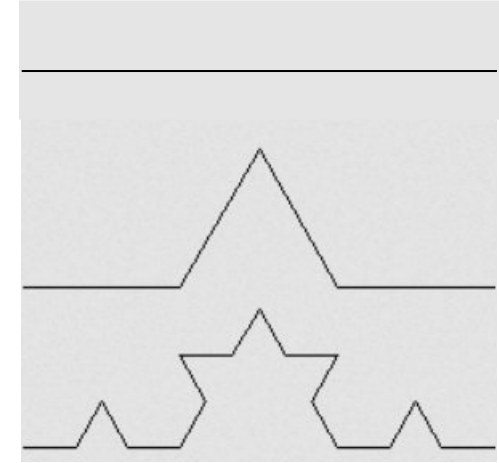
- Schwedischen Mathematiker Helge von Koch 1904
- Eines der Ersten formal beschriebenen fraktalen Objekte.
- Eines der am häufigsten zitierten Beispiele für ein Fraktal.
- Wurde bei der Entdeckung als Monsterkurve bezeichnet (überall stetige, aber nirgends differenzierbare Kurve).
- Die Koch-Kurve ist auch in Form der kochschen Schneeflocke bekannt, die durch geeignete Kombination dreier Koch-Kurven entsteht.



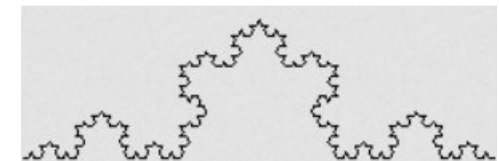
Kochkurve



- Schwedischen Mathematiker Helge von Koch 1904
- Eines der Ersten formal beschriebenen fraktalen Objekte.
- Eines der am häufigsten zitierten Beispiele für ein Fraktal.
- Wurde bei der Entdeckung als Monsterkurve bezeichnet (überall stetige, aber nirgends differenzierbare Kurve).
- Die Koch-Kurve ist auch in Form der kochschen Schneeflocke bekannt, die durch geeignete Kombination dreier Koch-Kurven entsteht.



usw..



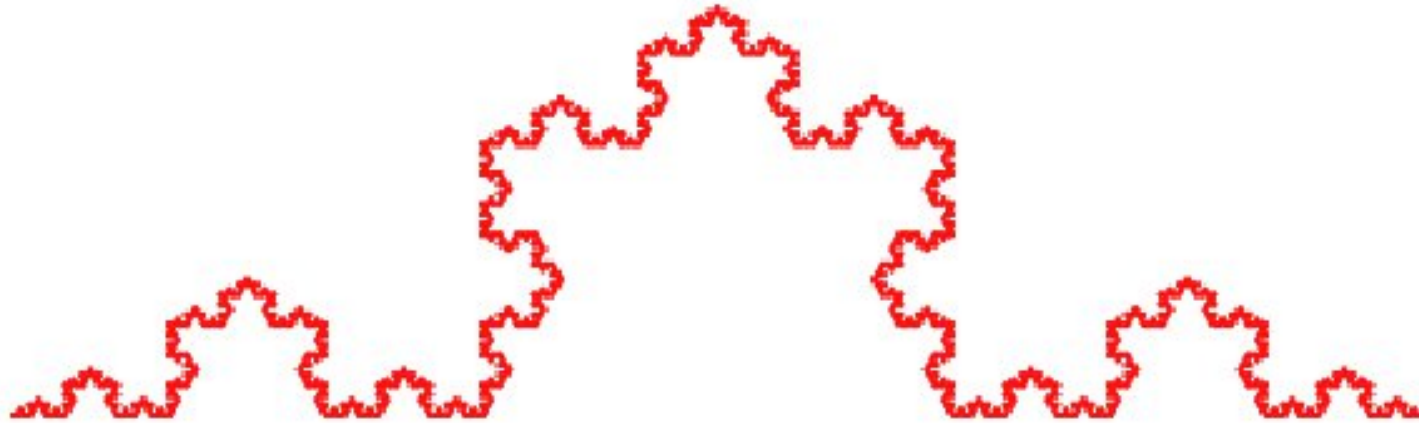
<http://michael.szell.net/fba/kapitel1.html>
<https://de.wikipedia.org/wiki/Monsterkurve>
<https://de.wikipedia.org/wiki/Koch-Kurve>



Kochkurve



max depth 5



Schneeflocke vs. kochschen Schneeflocke

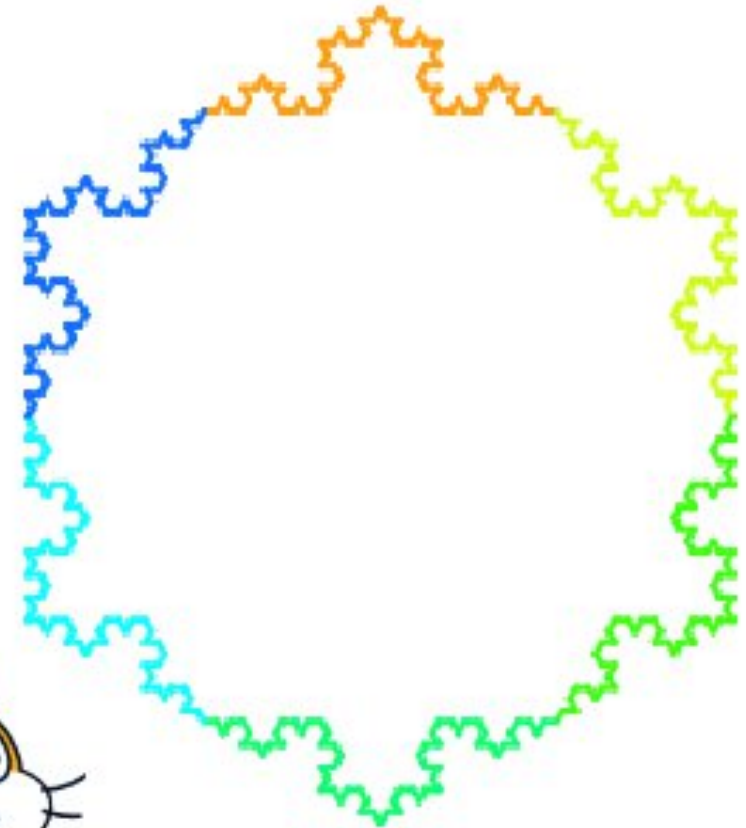


Nathan Myhrvold / Modernist Cuisine Gallery LLC

<https://www.spiegel.de/wissenschaft/technik/fotografie-auf-der-jagd-nach-der-perfekten-schneeflocke-a-66d6b4c6-50b5-4952-8c18-94f75e43fb36>

max depth

3



K
A

Schneeflocke vs. kochschen Schneeflocke



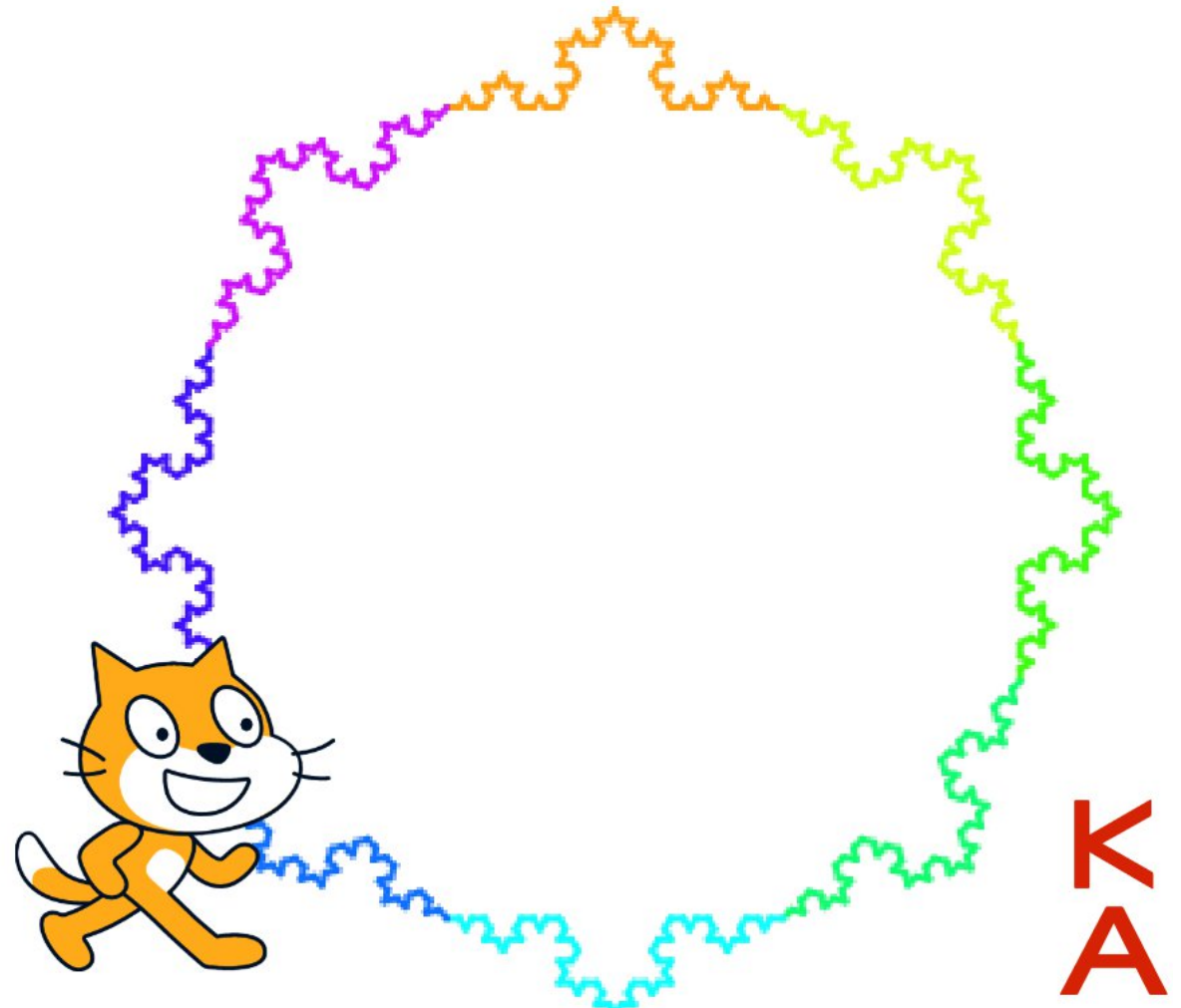
max depth

3



Nathan Myhrvold / Modernist Cuisine Gallery LLC

<https://www.spiegel.de/wissenschaft/technik/fotografie-auf-der-jagd-nach-der-perfekten-schneeflocke-a-66d6b4c6-50b5-4952-8c18-94f75e43fb36>



K
A

Dreidimensionale buschähnliche Pflanze



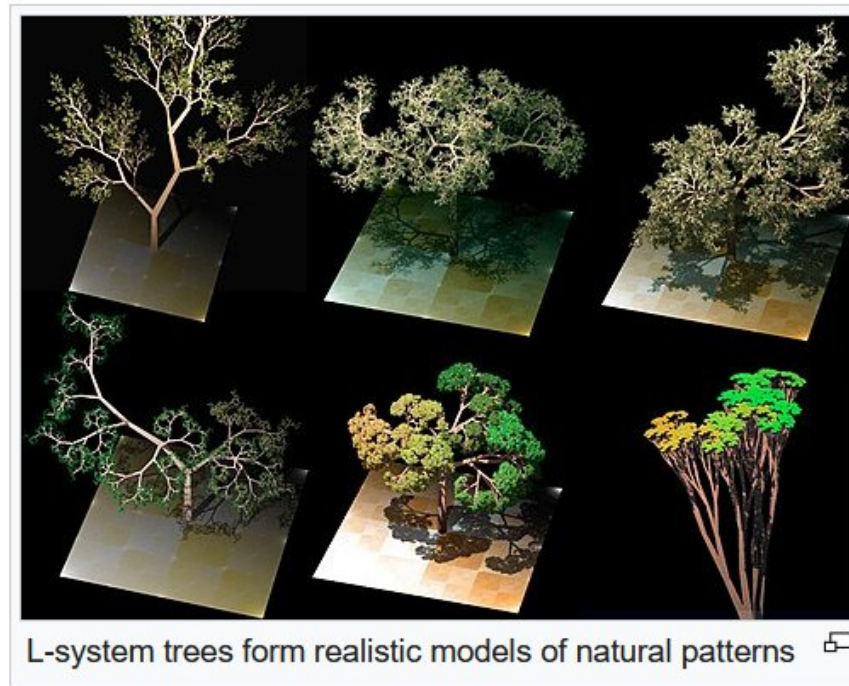
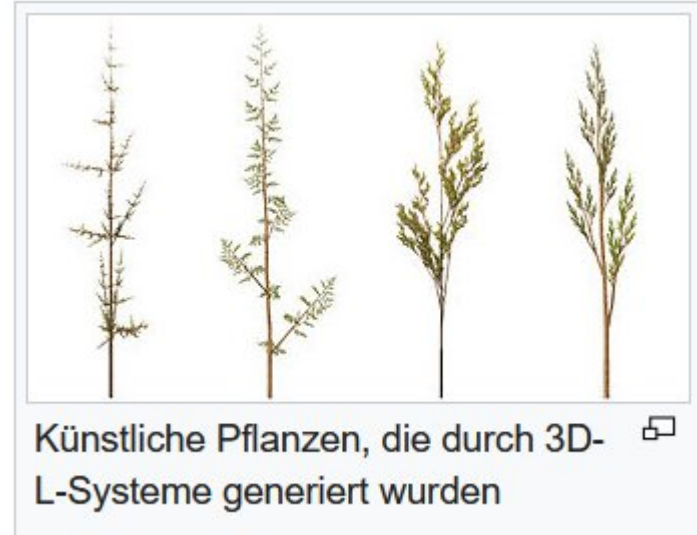
Pflanzen sind sogenannte Multifraktale, d.h. sie besitzen auf unterschiedlichen Größenebenen bestimmte verschiedene fraktale Dimensionen [1].



L-System / Lindenmayer-System



- mathematischen Formalismus
- axiomatischen Theorie biologischer Entwicklung
- „In jüngerer Zeit“ fanden L-Systeme Anwendung in der Computergrafik bei der Erzeugung von Fraktalen und in der realitätsnahen Modellierung von Pflanzen.



Pflanzen sind sogenannte Multifraktale, d.h. sie besitzen auf unterschiedlichen Größenebenen bestimmte verschiedene fraktale Dimensionen [1].



[1] <https://de.wikipedia.org/wiki/Lindenmayer-System>

[2] <https://en.wikipedia.org/wiki/L-system>

L-System / Lindenmayer-System

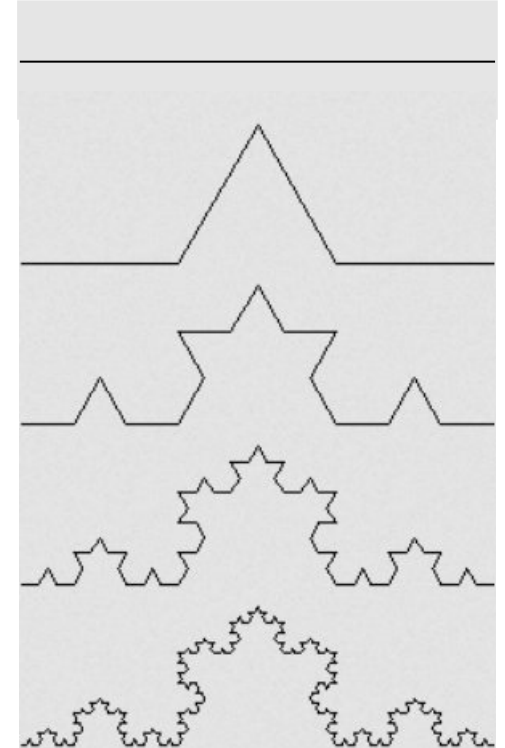


- mathematischen Formalismus
- axiomatischen Theorie biologischer Entwicklung
- „In jüngerer Zeit“ fanden L-Systeme Anwendung in der Computergrafik bei der Erzeugung von Fraktalen und in der realitätsnahen Modellierung von Pflanzen.

Beispiel: **Koch Kurve**
Variable: F
Konstante: + -
Start: F
Regel: F → F+F--F+F

Hierbei bedeutet F „vorwärts ziehen“, + drehen um 60° nach links, und - drehen um 60° nach rechts.

F
F+F--F+F
F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F
usw..



[1] <https://de.wikipedia.org/wiki/Lindenmayer-System>

[2] <https://en.wikipedia.org/wiki/L-system>

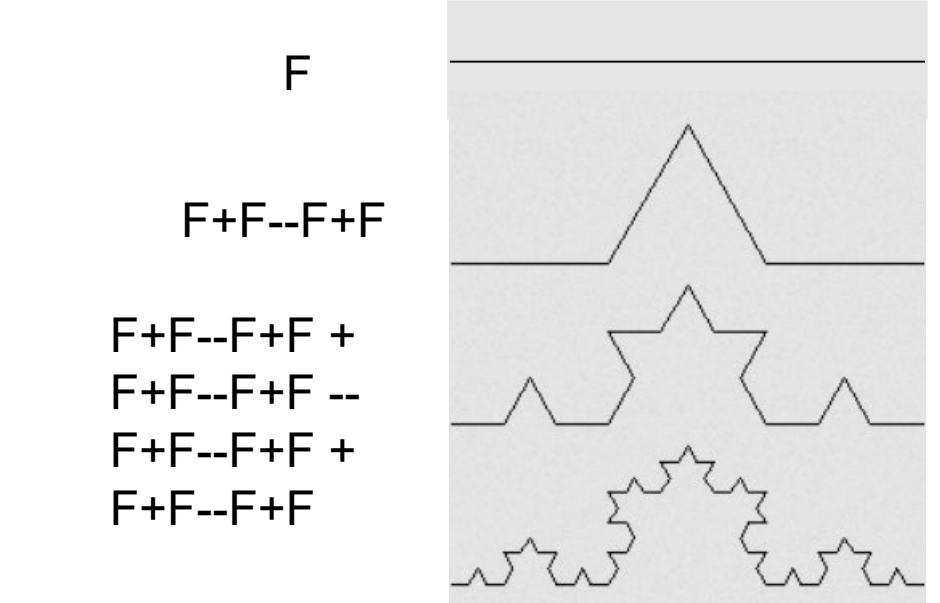
L-System / Lindenmayer-System



- mathematischen Formalismus
- axiomatischen Theorie biologischer Entwicklung
- „In jüngerer Zeit“ fanden L-Systeme Anwendung in der Computergrafik bei der Erzeugung von Fraktalen und in der realitätsnahen Modellierung von Pflanzen.

Beispiel: **Koch Kurve**
Variable: F
Konstante: + -
Start: F
Regel: F → F+F--F+F

Hierbei bedeutet F „vorwärts ziehen“, + drehen um 60° nach links, und - drehen um 60° nach rechts.



[1] <https://de.wikipedia.org/wiki/Lindenmayer-System>

[2] <https://en.wikipedia.org/wiki/L-system>

L-System / Lindenmayer-System



- mathematischen Formalismus
- axiomatischen Theorie biologischer Entwicklung
- „In jüngerer Zeit“ fanden L-Systeme Anwendung in der Computergrafik bei der Erzeugung von Fraktalen und in der realitätsnahen Modellierung von Pflanzen.

Beispiel: Koch Kurve

Variable: F

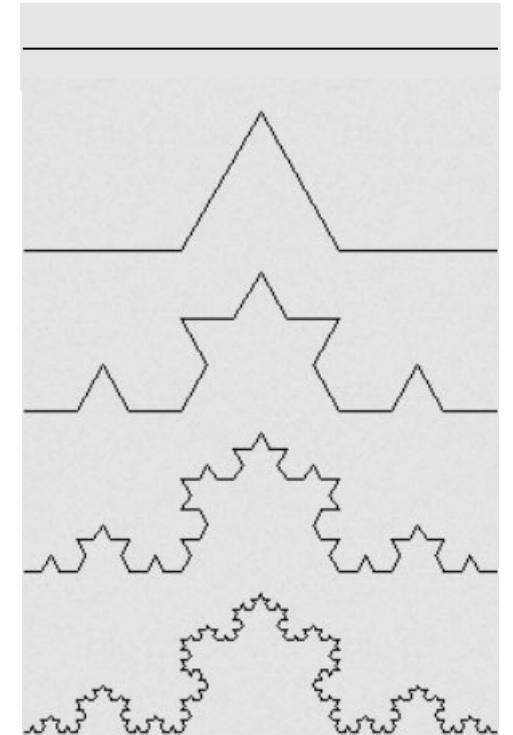
Konstante: + -

Start: F

Regel: $F \rightarrow F+F--F+F$

Rekursive Definition:
F ist wieder durch F definiert

usw..



Hierbei bedeutet F „vorwärts ziehen“, + drehen um 60° nach links, und - drehen um 60° nach rechts.

[1] <https://de.wikipedia.org/wiki/Lindenmayer-System>

[2] <https://en.wikipedia.org/wiki/L-system>

Von Fraktalen zur Rekursion



- Als Rekursion (lateinisch recurrere ‚zurücklaufen‘) wird ein prinzipiell unendlicher Vorgang, der sich selbst als Teil enthält oder mithilfe von sich selbst definierbar ist, bezeichnet.



Unendlichfache Spiegelung als Beispiel für **Rekursion**

Von Fraktalen zur Rekursion



- Als Rekursion (lateinisch recurrere ‚zurücklaufen‘) wird ein prinzipiell unendlicher Vorgang, der sich selbst als Teil enthält oder mithilfe von sich selbst definierbar ist, bezeichnet.

Beispiele für rekursive Definitionen:

Nachkommen:

„Nachkommen eines Menschen sind seine Kinder und deren Nachkommen.“



Unendlichfache Spiegelung als Beispiel für **Rekursion**

Von Fraktalen zur Rekursion



- Als Rekursion (lateinisch recurrere ‚zurücklaufen‘) wird ein prinzipiell unendlicher Vorgang, der sich selbst als Teil enthält oder mithilfe von sich selbst definierbar ist, bezeichnet.

Beispiele für rekursive Definitionen:

Nachkommen:

„Nachkommen eines Menschen sind seine Kinder und deren Nachkommen.“

Marsch zum Nordpol:

Schon am Nordpol? Dann fertig.

Sonst: Schritt nach Norden und „Marsch zum Nordpol“.

Ist ein Wort ein Palindrom?

Wenn das Wort weniger als zwei Buchstaben hat: Ja!

Sonst: Wenn der erste und letzte Buchstabe gleich sind und der Rest ein Palindrom ist.



Unendlichfache Spiegelung als Beispiel für **Rekursion**

Von Fraktalen zur Rekursion



- Als Rekursion (lateinisch recurrere ‚zurücklaufen‘) wird ein prinzipiell unendlicher Vorgang, der sich selbst als Teil enthält oder mithilfe von sich selbst definierbar ist, bezeichnet.

Beispiele für rekursive Definitionen:

Nachkommen:

„Nachkommen eines Menschen sind seine Kinder und deren Nachkommen.“

Marsch zum Nordpol:

Schon am Nordpol? Dann fertig.

Sonst: Schritt nach Norden und „Marsch zum Nordpol“.

Ist ein Wort ein Palindrom?

Wenn das Wort weniger als zwei Buchstaben hat: Ja!

Sonst: Wenn der erste und letzte Buchstabe gleich sind und der Rest ein Palindrom ist.

Zinseszins:

$K_n = K_{n-1} \left(1 + \frac{\text{Zinssatz}}{100}\right) + \text{Jahresrate} \frac{\text{Zinssatz}}{100}$ und K_0 ist der Geldbetrag zu

Beginn.



Unendlichfache Spiegelung als Beispiel für **Rekursion**

<https://de.wikipedia.org/wiki/Rekursion>
https://de.wikipedia.org/wiki/Rekursive_Programmierung
<https://de.wikipedia.org/wiki/Fraktal>

Beispiele: Prof. Ivo Rogina



Von Fraktalen zur Rekursion



- Als Rekursion (lateinisch recurrere ‚zurücklaufen‘) wird ein prinzipiell unendlicher Vorgang, der sich selbst als Teil enthält oder mithilfe von sich selbst definierbar ist, bezeichnet.
- „Der Begriff [Rekursion] ist sehr umfassend“. In der Natur handelt es sich um einen häufig beobachtbaren Vorgang (z. B. beim Pflanzenwachstum)
- Fraktale Muster werden oft durch rekursive Operationen erzeugt. Auch einfache Erzeugungsregeln ergeben nach wenigen Rekursionsschritten schon komplexe Muster.

Bei der rekursiven Programmierung ruft sich eine Funktion in einem Computerprogramm selbst wieder auf (d. h. enthält eine Rekursion). Auch der gegenseitige Aufruf stellt eine Rekursion dar.



Unendlichfache Spiegelung als Beispiel für **Rekursion**

Von Fraktalen zur Rekursion



- Als Rekursion (lateinisch recurrere ‚zurücklaufen‘) wird ein prinzipiell unendlicher Vorgang, der sich selbst als Teil enthält oder mithilfe von sich selbst definierbar ist, bezeichnet.
- „Der Begriff [Rekursion] ist sehr umfassend“. In der Natur handelt es sich um einen häufig beobachtbaren Vorgang (z. B. beim Pflanzenwachstum)
- Fraktale Muster werden oft durch rekursive Operationen erzeugt. Auch einfache Erzeugungsregeln ergeben nach wenigen Rekursionsschritten schon komplexe Muster.

Bei der rekursiven Programmierung **ruft** sich eine **Funktion** in einem Computerprogramm **selbst wieder auf** (d. h. enthält eine Rekursion). Auch der gegenseitige Aufruf stellt eine Rekursion dar.



Unendlichfache Spiegelung als Beispiel für **Rekursion**

Kurze Wiederholung Funktionen



Funktionen

■ Aufbau:

- Funktionsaufruf
- Funktionskopf (Prototyp, mit Argumenten)
- Funktionsrumpf
- Rückgabewert

```
int KleinerGauss(int n)
{
    // Ausrechnen
    int summe = 0;
    for (int i = n; i > 0; i--)
    {
        summe = summe + i;
    }

    // Summe als Funktionsresultat zurück
    return summe;
}
```

Beispiele

■ Autowerkstatt

- Auto mit Auftrag abgeben (Aufruf, Parameter)
- Arbeiten in der Werkstatt
- Rückgabe Auto und Rechnung

■ Pizza liefern

- Bestellung
- Kochen
- Liefern



- **Iteration:** Darunter versteht man das mehrmalige Ausführen einer Aktion. Hierzu nutzt man Kontrollstrukturen wie Schleifen (for, while, ...). Eine iterative Funktion hat also eine derartige Kontrollstruktur im Funktionskörper.

- Kennen wir bereits!

- **Iteration:** Darunter versteht man das mehrmalige Ausführen einer Aktion. Hierzu nutzt man Kontrollstrukturen wie Schleifen (for, while, ...). Eine iterative Funktion hat also eine derartige Kontrollstruktur im Funktionskörper.
 - Kennen wir bereits!
- **Rekursion:** Darunter versteht man ebenfalls das mehrmalige Ausführen einer Aktion. Doch diesmal nutzen wir keine Schleife, **sondern die Funktion ruft sich selbst auf**. Rekursive Funktionen sind dadurch charakterisiert, dass diese
 - Eine Abbruchbedingung brauchen
 - Sich selbst aufrufen

- **Iteration:** Darunter versteht man das mehrmalige Ausführen einer Aktion. Hierzu nutzt man Kontrollstrukturen wie Schleifen (for, while, ...). Eine iterative Funktion hat also eine derartige Kontrollstruktur im Funktionskörper.

- Kennen wir bereits!

- **Rekursion:** Darunter versteht man ebenfalls das mehrmalige Ausführen einer Aktion. Doch diesmal nutzen wir keine Schleife, **sondern die Funktion ruft sich selbst auf**. Rekursive Funktionen sind dadurch charakterisiert, dass diese

- Eine Abbruchbedingung brauchen
- Sich selbst aufrufen

- **Algorithmen können sowohl iterativ als auch rekursiv formuliert werden**

- Rekursionen lassen sich oftmals einfacher darstellen
- Iterationen sind oftmals effizienter, da das Zwischenergebnis der einzelnen Funktionsaufrufe nicht gespeichert werden muss. Bei einer Rekursion müssen im Stack des Arbeitsspeichers die Zwischenergebnisse abgelegt werden.

Iterative und rekursive Funktionen



- **Iteration:** Darunter versteht man das mehrmalige Ausführen einer Aktion. Hierzu nutzt man Kontrollstrukturen wie Schleifen (for, while, ...). Eine iterative Funktion hat also eine derartige Kontrollstruktur im Funktionskörper.

- Kennen wir bereits!

- **Rekursion:** Darunter versteht man ebenfalls das mehrmalige Ausführen einer Aktion. Doch diesmal nutzen wir keine Schleife, **sondern die Funktion ruft sich selbst auf**. Rekursive Funktionen sind dadurch charakterisiert, dass diese

- Eine Abbruchbedingung brauchen
- Sich selbst aufrufen

- **Algorithmen können sowohl iterativ als auch rekursiv formuliert werden**

- Rekursionen lassen sich oftmals einfacher darstellen
- Iterationen sind oftmals effizienter, da das Zwischenergebnis der einzelnen Funktionsaufrufe nicht gespeichert werden muss. Bei einer Rekursion müssen im Stack des Arbeitsspeichers die Zwischenergebnisse abgelegt werden.

- Damit wir das besser verstehen können, schauen wir uns Beispiele an ..



Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```

Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

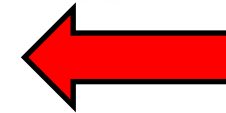
■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```



Abbruchbedingung

Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

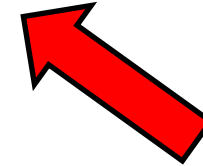
■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```



Sich selbst aufrufend

Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```

■ Beispiel $a = 3, n = 2 \rightarrow 3^2$

1. Aufruf	Berechnung	Ergebnis
$a=3, n=2$	$3 * \text{powRe}(3, 1)$	

Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```

Sich selbst aufrufend

■ Beispiel $a = 3, n = 2 \rightarrow 3^2$

1. Aufruf	Berechnung	Ergebnis
$a=3, n=2$	$3 * \text{powRe}(3, 1)$	

Sich selbst aufrufend

Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```

Sich selbst aufrufend

■ Beispiel $a = 3, n = 3 \rightarrow 3^2$

1. Aufruf	2. Aufruf		Berechnung	Ergebnis
a=3, n=2			3 * powRe(3, 1)	
	a=3, n=1			

Sich selbst aufrufend

Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```

■ Beispiel $a = 3, n = 3 \rightarrow 3^2$

1. Aufruf	2. Aufruf	3. Aufruf	Berechnung	Ergebnis
a=3, n=2			3 * powRe(3, 1)	
	a=3, n=1		3 * powRe(3, 0)	

Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

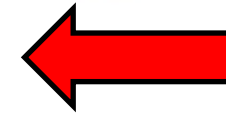
■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```



Abbruchbedingung

■ Beispiel $a = 3, n = 3 \rightarrow 3^2$

1. Aufruf	2. Aufruf	3. Aufruf	Berechnung	Ergebnis
a=3, n=2			3 * powRe(3, 1)	
	a=3, n=1		3 * powRe(3, 0)	
		a=3, n=0	1	1

Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```



■ Beispiel $a = 3, n = 3 \rightarrow 3^2$

1. Aufruf	2. Aufruf	3. Aufruf	Berechnung	Ergebnis
a=3, n=2			3 * powRe(3, 1)	
	a=3, n=1		3 * powRe(3, 0)	
		a=3, n=0	1	1



Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```

■ Beispiel $a = 3, n = 3 \rightarrow 3^2$

1. Aufruf	2. Aufruf	3. Aufruf	Berechnung	Ergebnis
a=3, n=2			3 * powRe(3, 1)	
	a=3, n=1		3 * powRe(3, 0)	
		a=3, n=0	1	1



Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```

■ Beispiel $a = 3, n = 3 \rightarrow 3^2$

1. Aufruf	2. Aufruf	3. Aufruf	Berechnung	Ergebnis
a=3, n=2			3 * powRe(3, 1)	
	a=3, n=1		3 * powRe(3, 0)	3 * 1
		a=3, n=0	1	1



Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```

■ Beispiel $a = 3, n = 3 \rightarrow 3^2$

1. Aufruf	2. Aufruf	3. Aufruf	Berechnung	Ergebnis
a=3, n=2			3 * powRe(3, 1)	
	a=3, n=1		3 * powRe(3, 0)	3 * 1
		a=3, n=0	1	1



Iterative und rekursive Funktionen (Beispiel)



■ Beispiel Potenzfunktion mit $p = a^n$

■ Iterative Formulierung powIt()

```
float powIt(float a, int n)
{
    float p = 1;
    while (n > 0)
    {
        p = p * a;
        n = n - 1;
    }
    return p;
}
```

■ Anstatt $p = a^n$ können wir auch schreiben

$$a^n = \begin{cases} 1, & \text{für } n = 0 \\ a \cdot (a^{n-1}), & \text{sonst} \end{cases}$$

➤ Somit haben wir einen rekursiven Ansatz

■ Rekursive Formulierung powRe()

```
float powRe(float a, int n)
{
    if (n <= 0)
    {
        return 1.0;
    }
    else
    {
        return a * powRe(a, n - 1);
    }
}
```

■ Beispiel $a = 3, n = 3 \rightarrow 3^2$

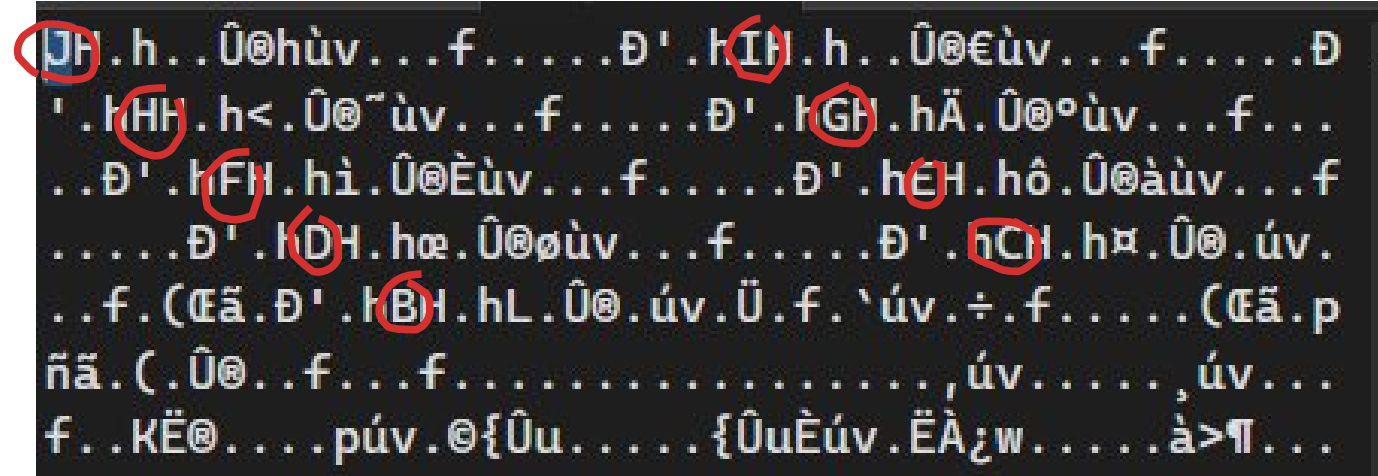
1. Aufruf	2. Aufruf	3. Aufruf	Berechnung	Ergebnis
a=3, n=2			3 * powRe(3, 1)	3 * 3 * 1
	a=3, n=1		3 * powRe(3, 0)	3 * 1
		a=3, n=0	1	1



Rekursive Funktion im Stack...



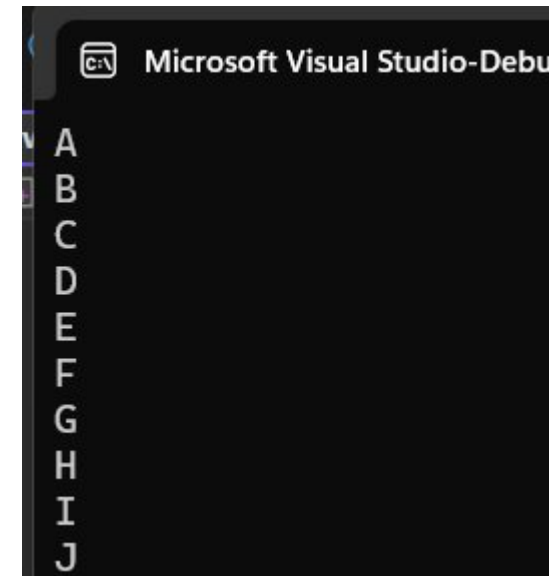
Stack:



```
void Re(char Zeichen, int tiefe) {  
    cout << Zeichen << endl;  
    if (tiefe == 1) return;  
    Re(Zeichen + 1, tiefe - 1);  
}
```

```
int main() {  
    Re('A', 10);  
}
```

Ausgabe:



Iterative und rekursive Funktionen (Beispiel)



■ Hauptprogramm Potenzfunktion

```
// HSKA - Technische Informatik 1
// Iteration und Rekursion
// Potenzfunktionen

#include <iostream>
using namespace std;

float powIt(float a, int n) { ... }

float powRe(float a, int n) { ... }

int main()
{
    // Iterativer Aufruf
    cout << "Iterative 10 to the power of 5 gives: " << powIt(10, 5) << endl;

    // Rekursiver Aufruf
    cout << "Recursive 10 to the power of 5 gives: " << powRe(10, 5) << endl;
}
```

■ Ausgabe

```
Microsoft Visual Studio-Debugging-Konsole
Iterative 10 to the power of 5 gives: 100000
Recursive 10 to the power of 5 gives: 100000
```

Iterative und rekursive Funktionen (Beispiel 2)



■ Beispiel Fakultät `faculty(k)` mit $k! = \prod_{n=1}^k n$

■ Iterative Formulierung `facultyIt()`

```
int facultyIt(int k)
{
    int f = 1;
    for (int i = 1; i <= k; i++)
    {
        f = f * i;
    }
    return f;
}
```

Iterative und rekursive Funktionen (Beispiel 2)



■ Beispiel Fakultät `faculty(k)` mit $k! = \prod_{n=1}^k n$

■ Iterative Formulierung `facultyIt()`

```
int facultyIt(int k)
{
    int f = 1;
    for (int i = 1; i <= k; i++)
    {
        f = f * i;
    }
    return f;
}
```

■ Anstatt $k! = \prod_{n=1}^k n$ können wir auch schreiben

$$k! = \begin{cases} 1, & \text{falls } k = 1 \\ k \cdot (k-1)!, & \forall k > 1 \end{cases}$$

➤ Somit haben wir eine rekursive Vorschrift

Iterative und rekursive Funktionen (Beispiel 2)



■ Beispiel Fakultät `faculty(k)` mit $k! = \prod_{n=1}^k n$

■ Iterative Formulierung `facultyIt()`

```
int facultyIt(int k)
{
    int f = 1;
    for (int i = 1; i <= k; i++)
    {
        f = f * i;
    }
    return f;
}
```

■ Anstatt $k! = \prod_{n=1}^k n$ können wir auch schreiben

$$k! = \begin{cases} 1, & \text{falls } k = 1 \\ k \cdot (k-1)!, & \forall k > 1 \end{cases}$$

➤ Somit haben wir eine rekursive Vorschrift

Iterative und rekursive Funktionen (Beispiel 2)



■ Beispiel Fakultät `faculty(k)` mit $k! = \prod_{n=1}^k n$

■ Iterative Formulierung `facultyIt()`

```
int facultyIt(int k)
{
    int f = 1;
    for (int i = 1; i <= k; i++)
    {
        f = f * i;
    }
    return f;
}
```

■ Anstatt $k! = \prod_{n=1}^k n$ können wir auch schreiben

$$k! = \begin{cases} 1, & \text{falls } k = 1 \\ k \cdot (k-1)!, & \forall k > 1 \end{cases}$$

➤ Somit haben wir eine rekursive Vorschrift

■ Rekursive Formulierung `facultyRe()`

```
int facultyRe(int k)
{
```

Übungsaufgabe...

Checkliste: was muss bei der Rekursion gegeben sein?



- (1) **Klarheit:** mit welchen Argumenten wird die Funktion aufgerufen?

- (2) **Abbruchbedingung:** eine Rekursion muss sicher zum Halten kommen. Dies wird meist über eine Bedingung der Funktionsargumente gelöst.

Checkliste: was muss bei der Rekursion gegeben sein?



- (1) **Klarheit:** mit welchen Argumenten wird die Funktion aufgerufen?
- (2) **Abbruchbedingung:** eine Rekursion muss sicher zum Halten kommen. Dies wird meist über eine Bedingung der Funktionsargumente gelöst.

Checkliste: was muss bei der Rekursion gegeben sein?



(1) **Klarheit:** mit welchen Argumenten wird die Funktion aufgerufen?

(2) **Abbruchbedingung:** eine Rekursion muss sicher zum Halten kommen. Dies wird meist über eine Bedingung der Funktionsargumente gelöst.

(3) **Arbeitslast** (engl. Workload): Was macht oder berechnet die Funktion? Werden diese Resultate auf dem Bildschirm ausgegeben oder als Funktionsresultat zurückgegeben?

■ Beispiel **Bildschirm-Ausgabe**

```
■ cout << "Die Zahl ist: " << i << endl;
```

```
■ printf("Zahl: %03d\n", i);
```

■ Beispiel **Zurückgabe als Funktionsresultat:**

```
■ summe = i + Funktion(i -1);
```

```
■ return summe;
```

```
■ return Funktion (i / 2);
```

Checkliste: was muss bei der Rekursion gegeben sein?



(1) **Klarheit:** mit welchen Argumenten wird die Funktion aufgerufen?

(2) **Abbruchbedingung:** eine Rekursion muss sicher zum Halten kommen. Dies wird meist über eine Bedingung der Funktionsargumente gelöst.

(3) **Arbeitslast** (engl. Workload): Was macht oder berechnet die Funktion? Werden diese Resultate auf dem Bildschirm ausgegeben oder als Funktionsresultat zurückgegeben?

(4) **Rekursion selbst:** Ein Funktionsaufruf mit mindestens einem geänderten Argument muss gegeben sein!

■ Beispiel **Bildschirm-Ausgabe**

```
■ cout << "Die Zahl ist: " << i << endl;
```

```
■ printf("Zahl: %03d\n", i);
```

■ Beispiel **Zurückgabe als Funktionsresultat:**

```
■ summe = i + Funktion(i -1);
```

```
■ return summe;
```

```
■ return Funktion (i / 2);
```

Mehr Übungen: Rekursion (1)



- Welche Ausgabe produziert diese Funktion für Eingabewerte > 0 ? [1]

```
void Beispiel01(int i)
{
    if (i != 0)
    {
        printf("%3d\n", i);
        Beispiel01(-i / 2);
    }
}
```

- Die berühmte Mathematikerin Dr. G. Auss hat folgende Funktion zur Berechnung einer Summe vorgeschlagen. Was hat Dr. Auss übersehen?

```
int SummeRek(int n)
{
    return n + SummeRek(n - 1);
}
```

- Schreiben Sie einen Countdown, der von 10 abwärts zählt und die Zahlen jeweils ausgibt. An Stelle von 0 soll allerdings "Go" ausgegeben werden.

Mehr Übungen: Rekursion (1)



- Welche Ausgabe produziert diese Funktion für Eingabewerte > 0 ? [1]

```
void Beispiel01(int i)
{
    if (i != 0)
    {
        printf("%3d\n", i);
        Beispiel01(-i / 2);
    }
}
```

Mehr Übungen: Rekursion (1)



- Welche Ausgabe produziert diese Funktion für Eingabewerte > 0 ? [1]

```
void Beispiel01(int i)
{
    if (i != 0)
    {
        printf("%3d\n", i);
        Beispiel01(-i / 2);
    }
}
```

Lösung:

- Die Funktion spring immer zwischen positiven und negativen Zahlen hin- und her und halbiert diese im Betrag dabei jeweils, so dass sie sicher bei der Division

$$\frac{|1 \text{ oder } -1|}{2} = 0$$

zum Halten kommt.

Mehr Übungen: Rekursion (1)



- Die berühmte Mathematikerin Dr. G. Auss hat folgende Funktion zur Berechnung einer Summe vorgeschlagen. Was hat Dr. Auss übersehen?

```
int SummeRek(int n)
{
    return n + SummeRek(n - 1);
}
```

Mehr Übungen: Rekursion (1)



- Die berühmte Mathematikerin Dr. G. Auss hat folgende Funktion zur Berechnung einer Summe vorgeschlagen. Was hat Dr. Auss übersehen?

```
int SummeRek(int n)
{
    return n + SummeRek(n - 1);
}
```

Lösung

- Die Funktion hat keine Abbruchbedingung und kommt daher nicht zum Halten!



- Schreiben Sie einen Countdown, der von 10 abwärts zählt und die Zahlen jeweils ausgibt. An Stelle von 0 soll allerdings "Go" ausgegeben werden.

```
void Countdown(int i)
{
    if (i == 0)
        printf("Go!\n");
    else
    {
        printf("%3d\n", i);
        Countdown(i - 1);
    }
}
```

Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

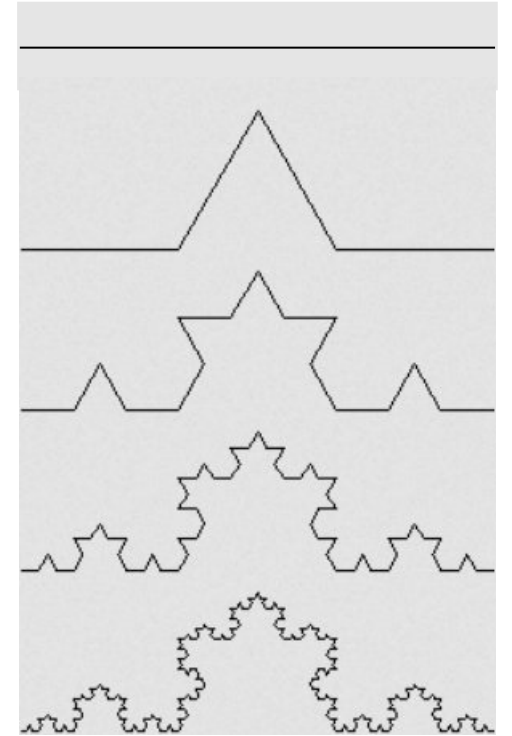
F -> F+F--F+F

**Rekursive Definition:
F ist wieder durch F definiert**

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F -> F+F--F+F

Rekursive Definition:

F ist wieder durch F definiert

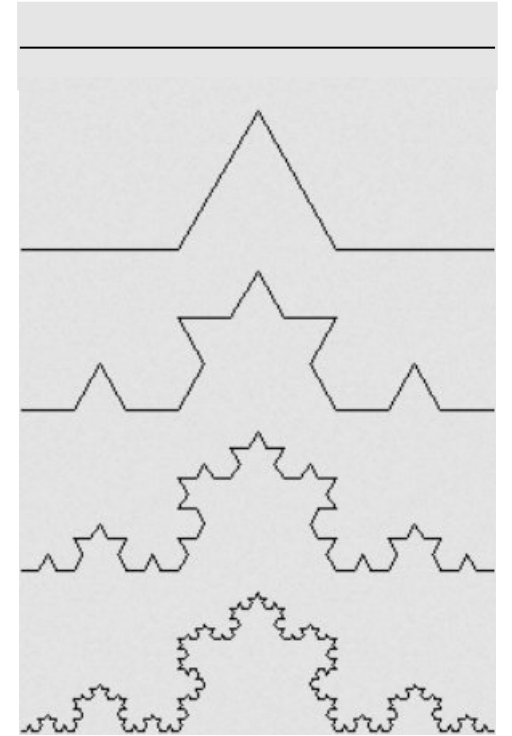
```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

```
int main() {  
    F(3);  
}
```

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

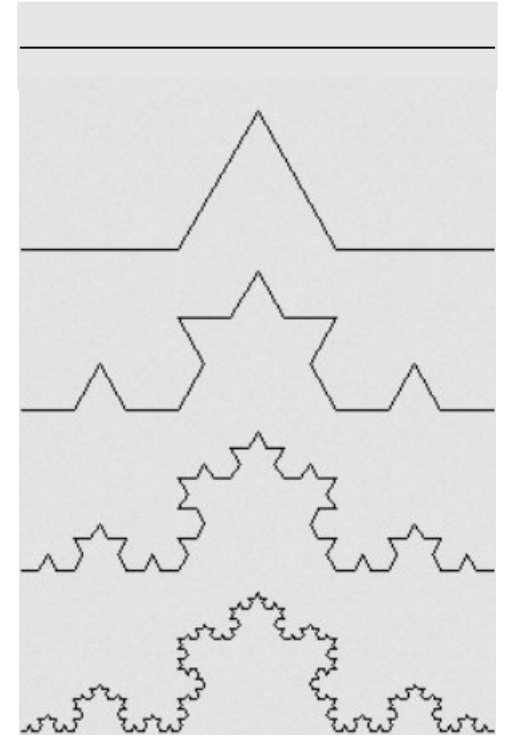
```
int main() {  
    F(1);  
}
```

1. Aufruf: tiefe=1

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

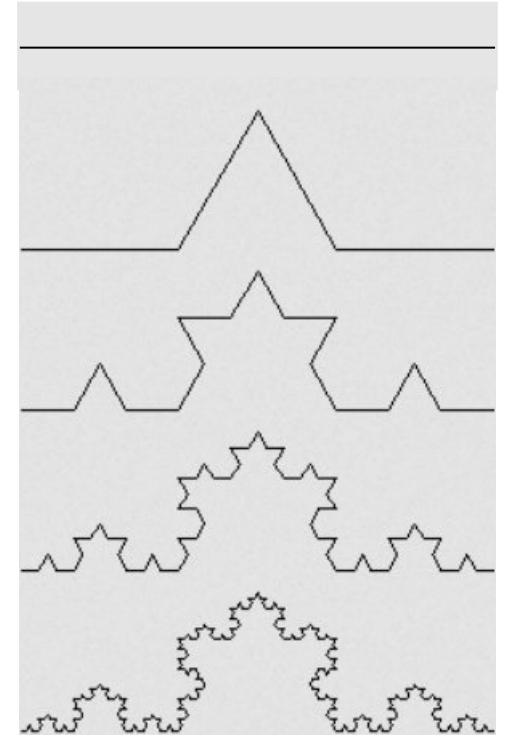
```
int main() {  
    F(1);  
}
```

F

F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

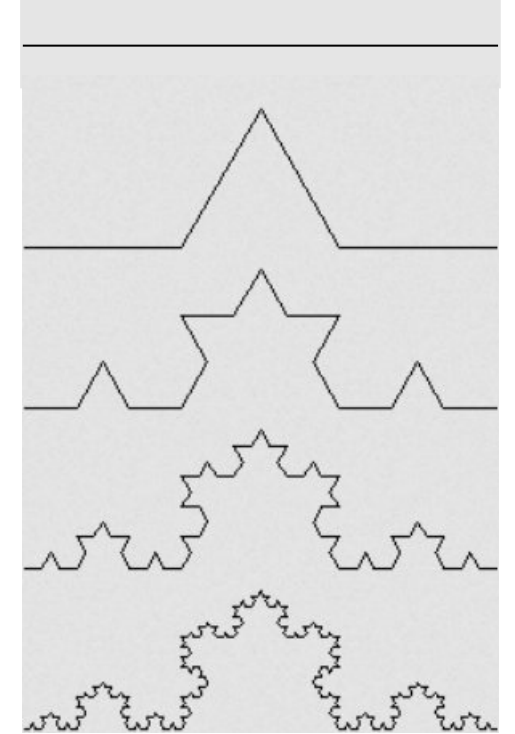
```
int main() {  
    F(1);  
}
```

F

F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

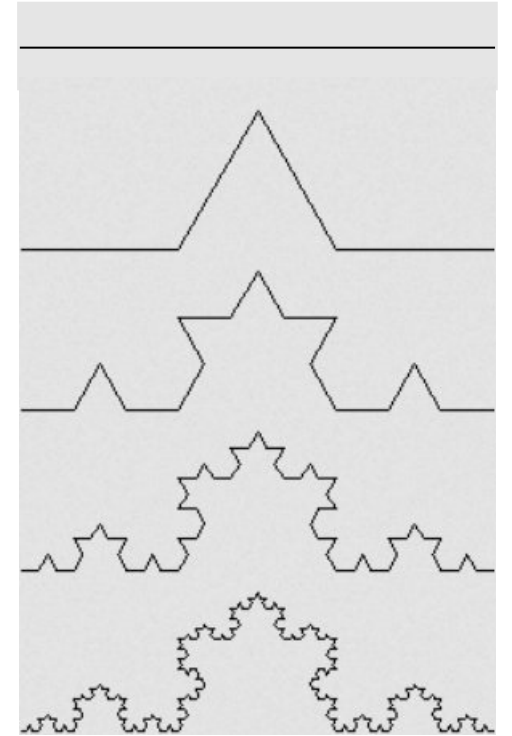
F → F+F--F+F

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

```
int main() {  
    F(2);  
}
```

F
F+F--F+F
F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

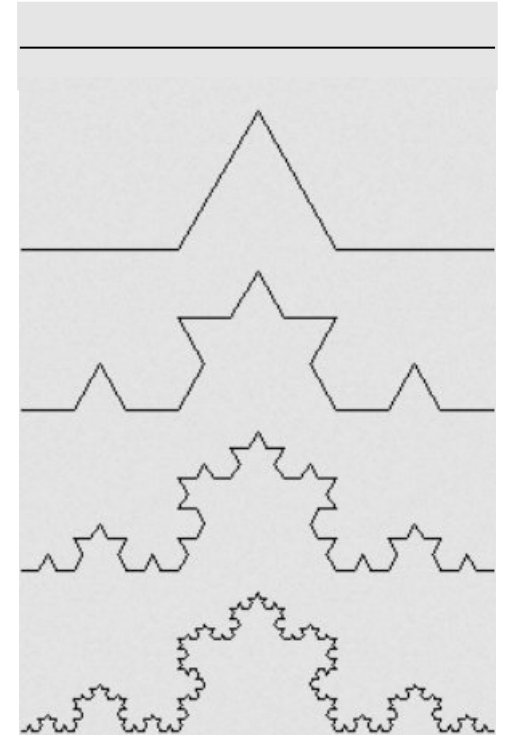
```
int main() {  
    F(2);  
}
```

1. Aufruf: tiefe=2

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

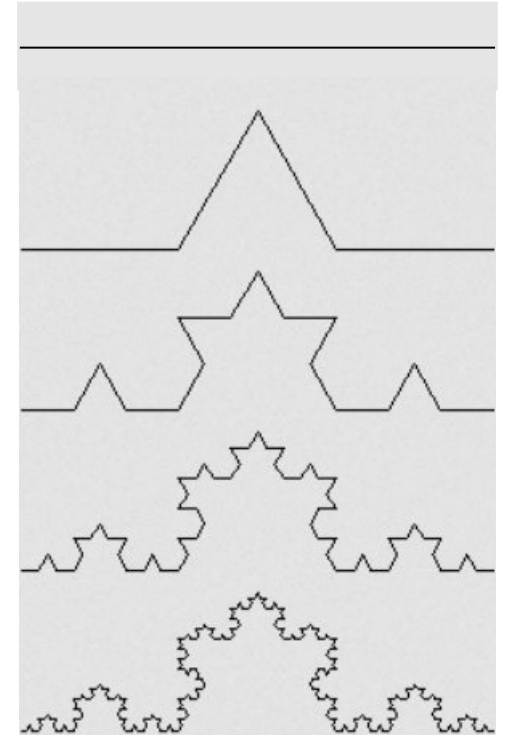
2. Aufruf: tiefe=1

```
int main() {  
    F(2);  
}
```

1. Aufruf: tiefe=2

F
F+F--F+F
F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {
```

```
    if (tiefe == 1) { cout << "F"; return; }
```

```
    F(tiefe - 1);
```

```
    cout << "+";
```

```
    F(tiefe - 1);
```

```
    cout << "--";
```

```
    F(tiefe - 1);
```

```
    cout << "+";
```

```
    F(tiefe - 1);
```

```
}
```

```
int main() {
```

```
    F(2);
```

```
}
```

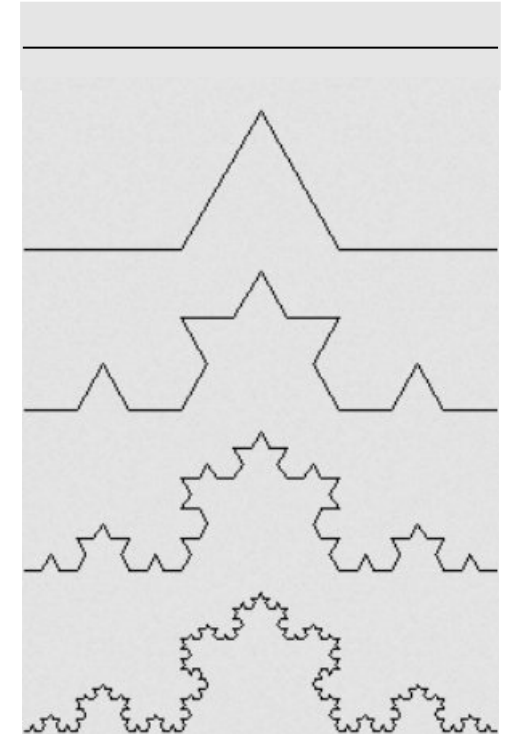
1. Aufruf: tiefe=2

2. Aufruf: tiefe=1

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

Rücksprung zum 1. Aufruf

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

2. Aufruf: tiefe=1

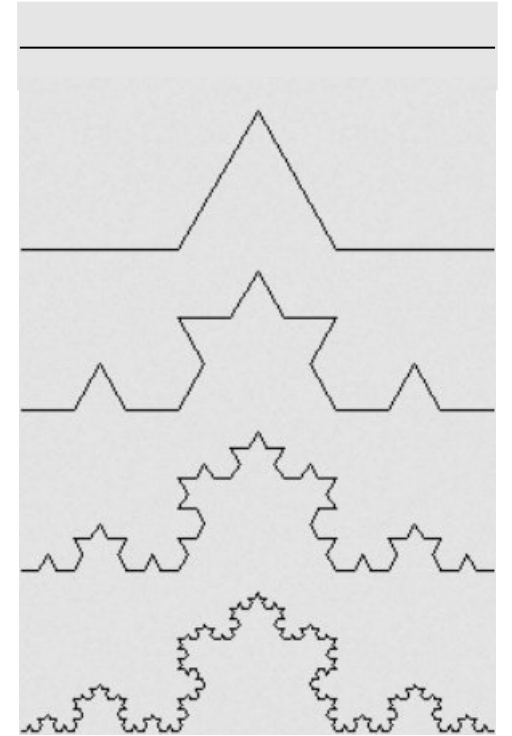
```
int main() {  
    F(2);  
}
```

1. Aufruf: tiefe=2

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

2. Aufruf: tiefe=1

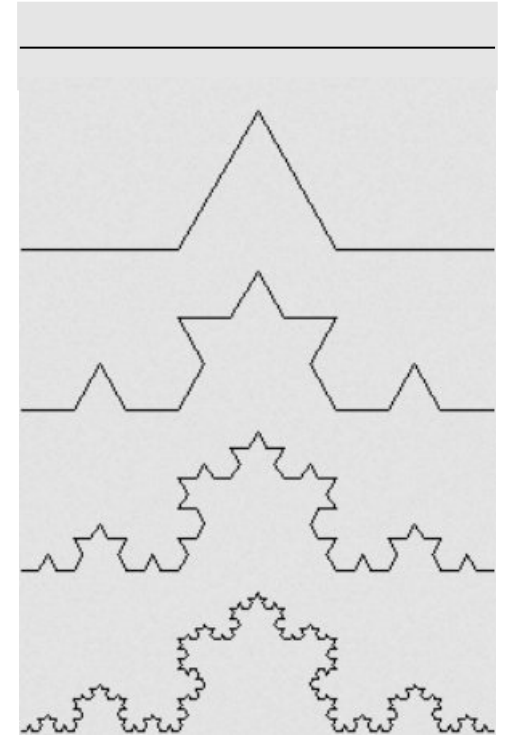
```
int main() {  
    F(2);  
}
```

1. Aufruf: tiefe=2

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {
```

```
    if (tiefe == 1) { cout << "F"; return; }
```

```
    F(tiefe - 1);
```

```
    cout << "+";
```

```
    F(tiefe - 1);
```

```
    cout << "--";
```

```
    F(tiefe - 1);
```

```
    cout << "+";
```

```
    F(tiefe - 1);
```

```
}
```

```
int main() {
```

```
    F(2);
```

```
}
```

2. Aufruf: tiefe=1

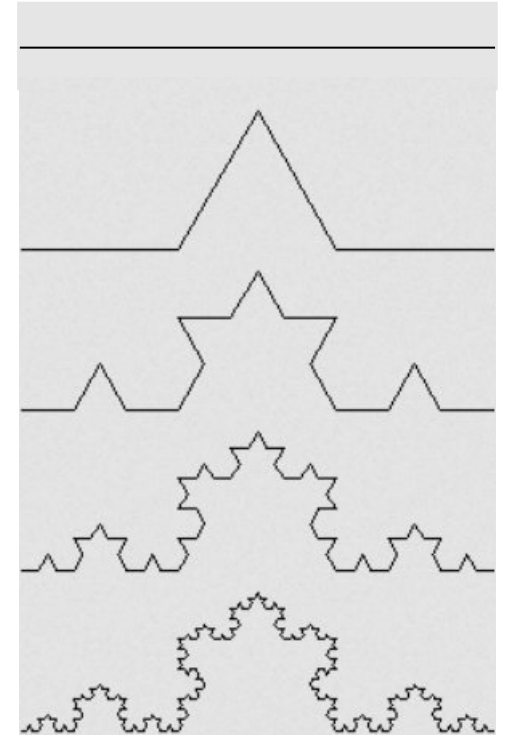
3. Aufruf: tiefe=1

1. Aufruf: tiefe=2

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

2. Aufruf: tiefe=1

3. Aufruf: tiefe=1

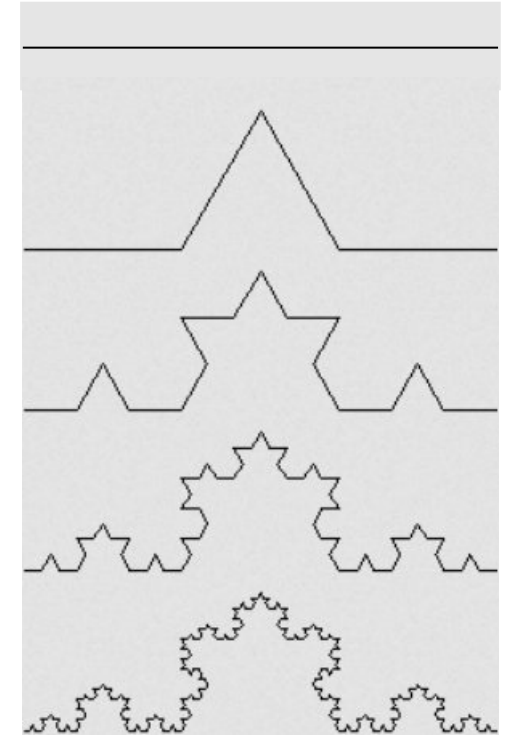
```
int main() {  
    F(2);  
}
```

1. Aufruf: tiefe=2

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

2. Aufruf: tiefe=1

3. Aufruf: tiefe=1

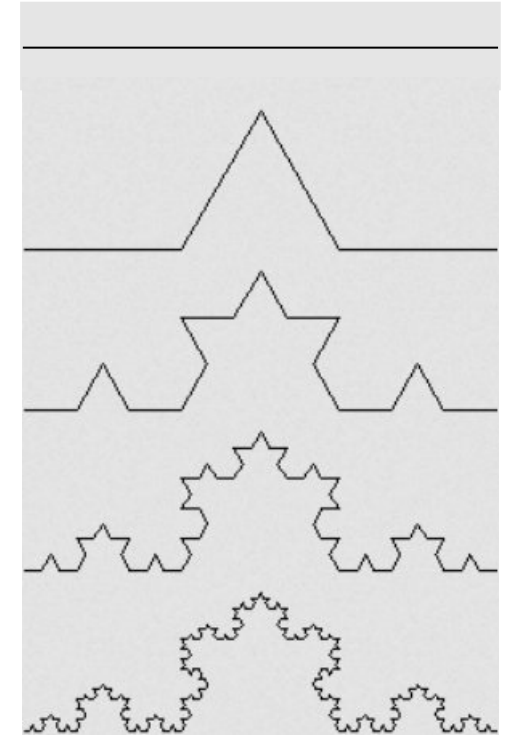
```
int main() {  
    F(2);  
}
```

1. Aufruf: tiefe=2

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

2. Aufruf: tiefe=1

3. Aufruf: tiefe=1

4. Aufruf: tiefe=1

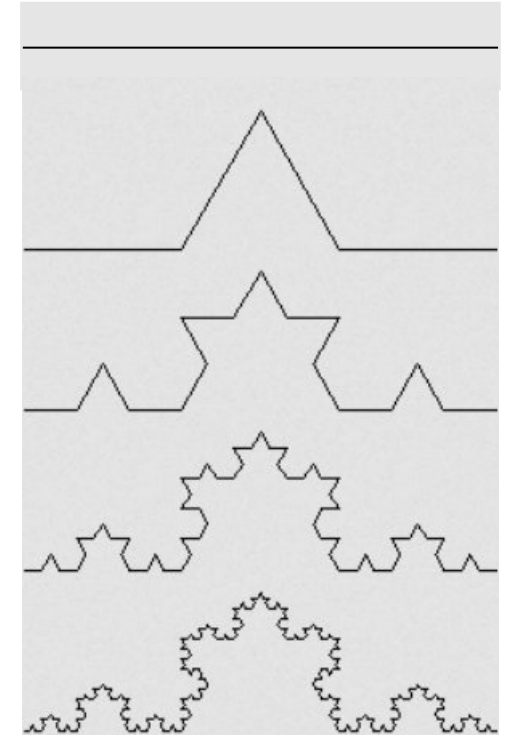
```
int main() {  
    F(2);  
}
```

1. Aufruf: tiefe=2

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



Eine Funktion kann sich auch mehr als einmal selber aufrufen!



Beispiel:

Koch Kurve

Variable:

F

Konstante:

+ -

Start:

F

Regel:

F → F+F--F+F

```
void F(int tiefe) {  
    if (tiefe == 1) { cout << "F"; return; }  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
    cout << "--";  
    F(tiefe - 1);  
    cout << "+";  
    F(tiefe - 1);  
}
```

2. Aufruf: tiefe=1

3. Aufruf: tiefe=1

4. Aufruf: tiefe=1

USW.....

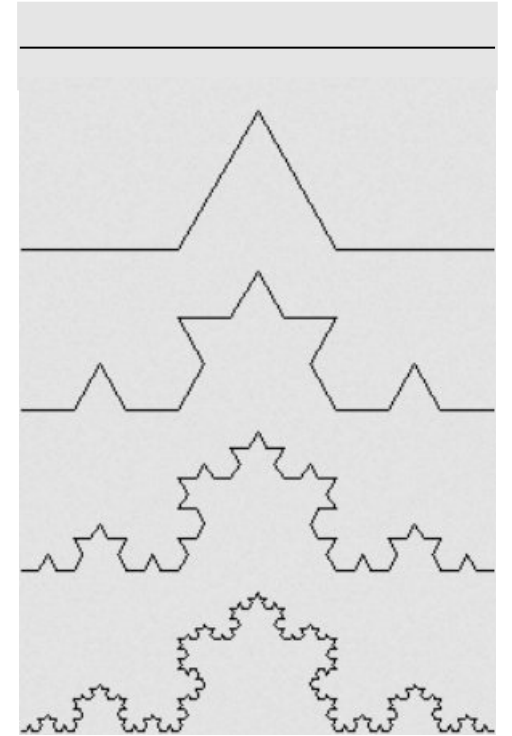
```
int main() {  
    F(2);  
}
```

1. Aufruf: tiefe=2

F
F+F--F+F

F+F--F+F +
F+F--F+F --
F+F--F+F +
F+F--F+F

usw..



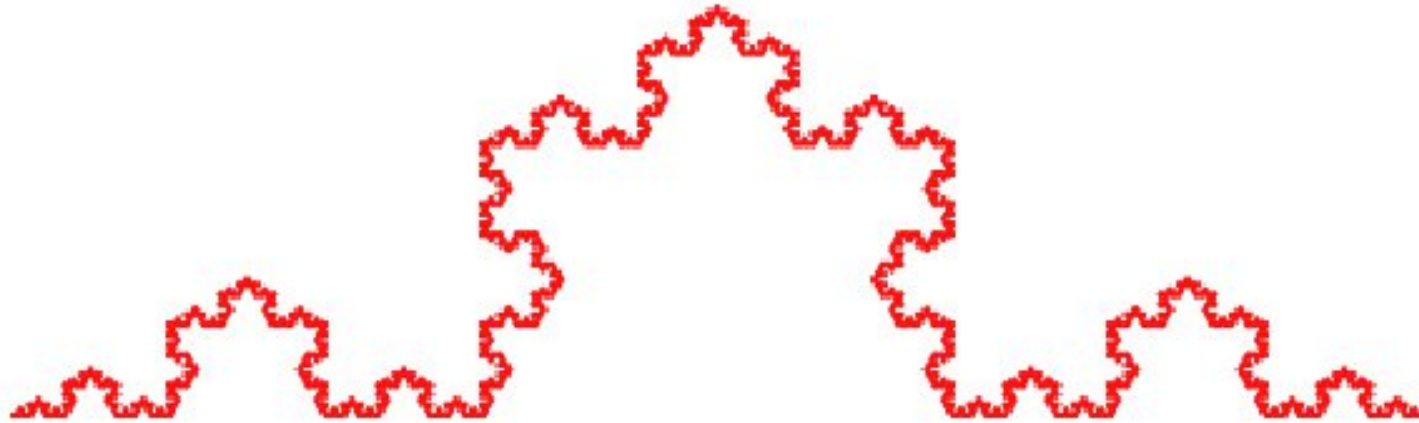
Hierbei bedeutet F „vorwärts ziehen“,
+ drehen um 60° nach links, und -
drehen um 60° nach rechts.



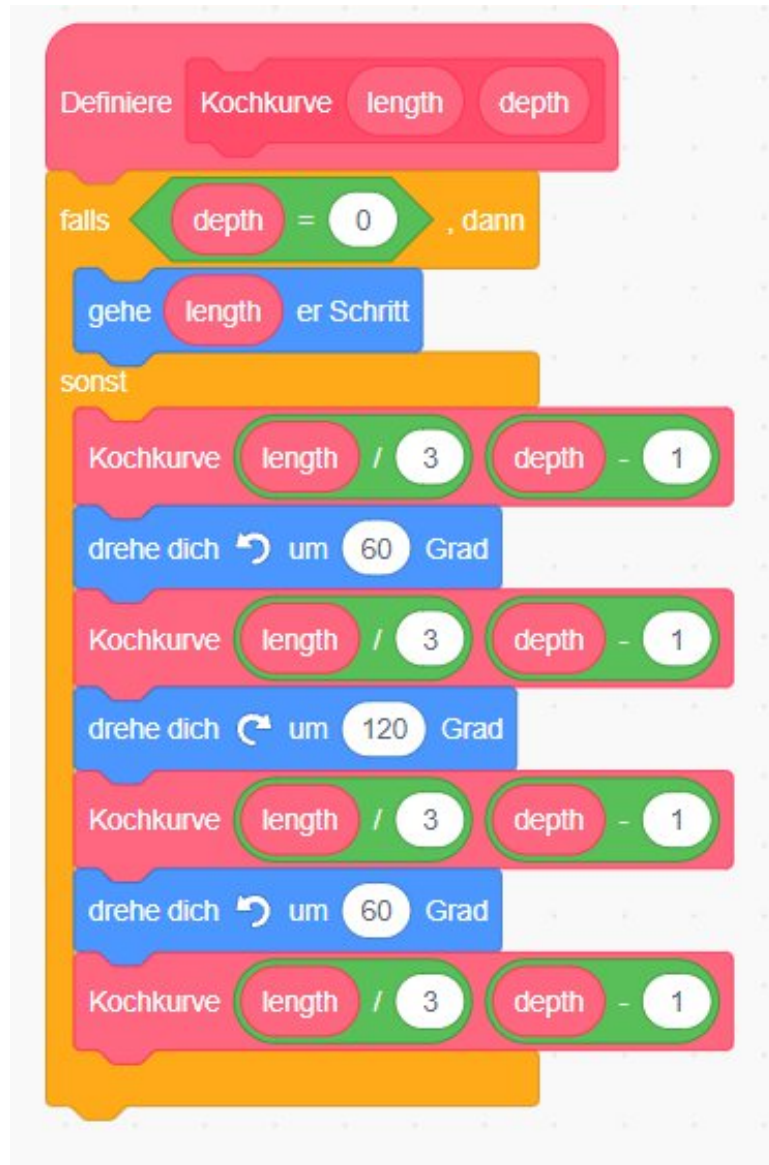
Let's Code: Kochkurve (Scratch)



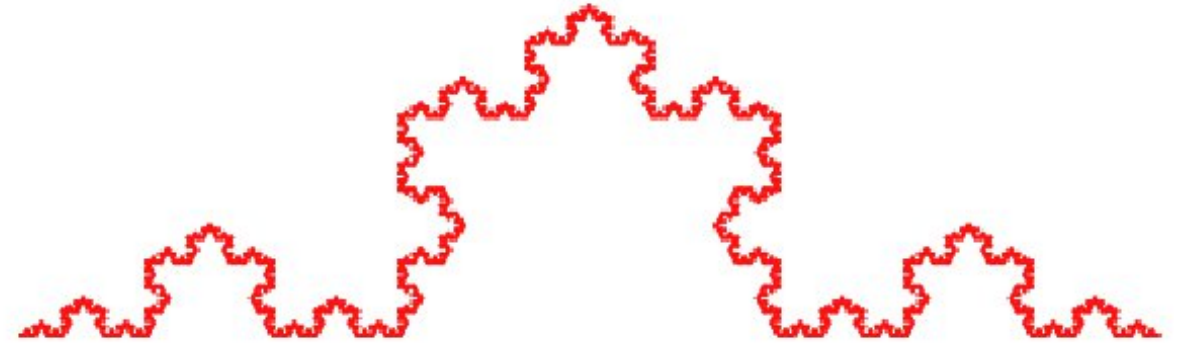
max depth 5



Let's Code: Kochkurve (Scratch)



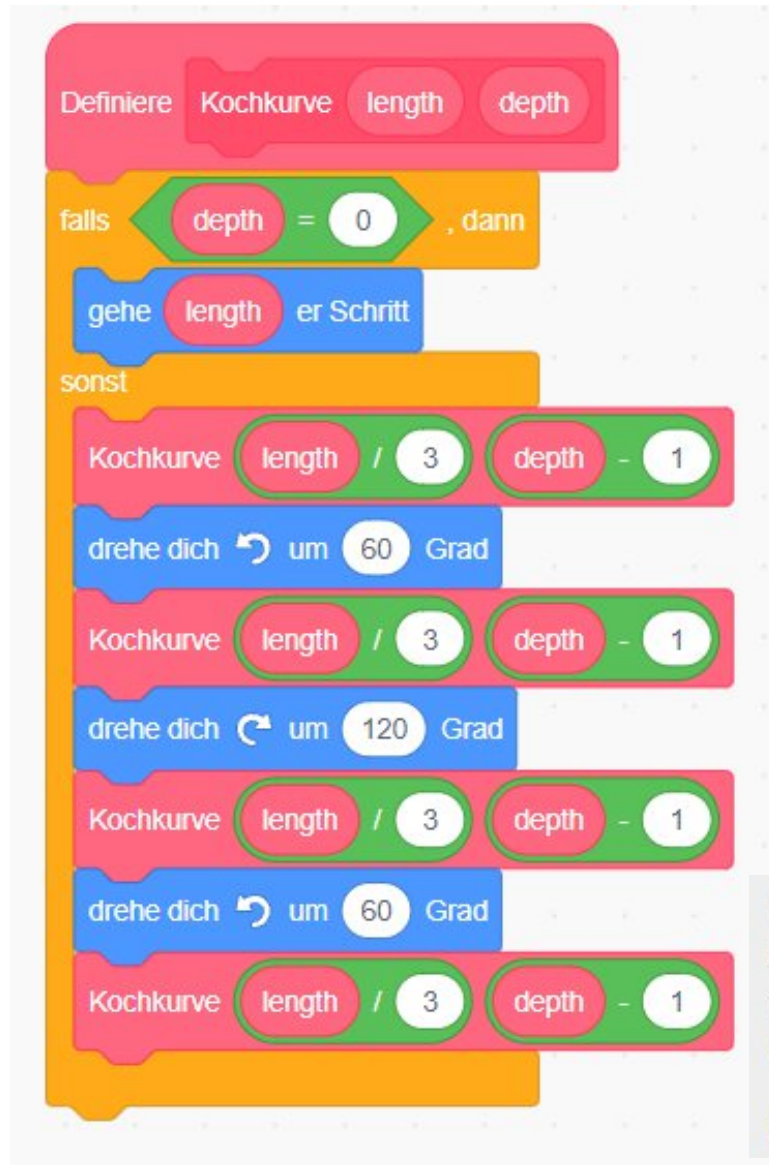
max depth 5



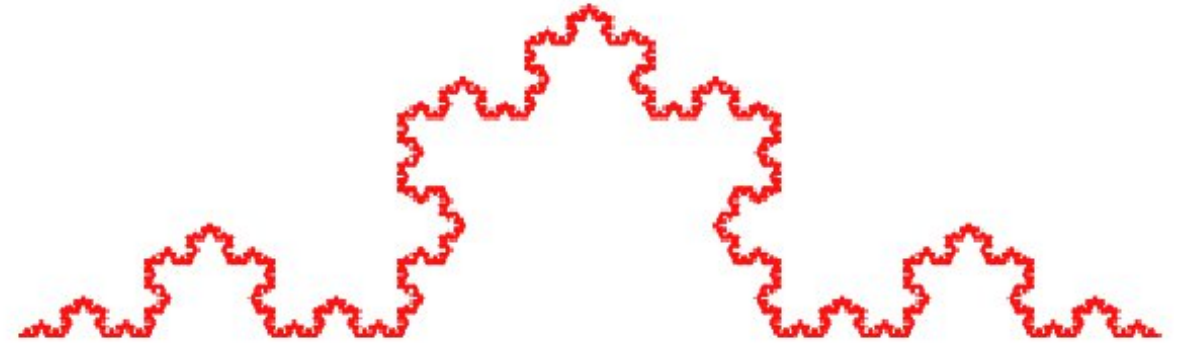
Python code siehe: <http://blog.schockwellenreiter.de/2017/11/2017111703.html>

https://www.pohlig.de/Unterricht/Inf2004/Kap07/7.4_Kochkurve.htm

Let's Code: Kochkurve (Scratch)



max depth 5



Wir dritteln die Strecke und setzen ein Dreieck nach der folgenden Vorschrift darauf: Gehe um die neue Strecke nach vorne (F = forward), drehe um $\alpha = 60$ gegen den Uhrzeigersinn (+), gehe wieder um vorwärts, drehe zweimal um α mit dem Uhrzeigersinn (- -), gehe vorwärts, drehe um α gegen den Uhrzeigersinn und gehe ein letztes Mal vorwärts. Wir können mit der F,+, - Notation also kurz schreiben:

$L := F + F - - F + F$

Python code siehe: <http://blog.schockwellenreiter.de/2017/11/2017111703.html>

https://www.pohlig.de/Unterricht/Inf2004/Kap07/7.4_Kochkurve.htm



Türme von Hanoi

Die Türme von Hanoi



Paradebeispiel für eine Rekursionslösung:

- Rekursion „leicht“ zu entwickeln
- Nicht-rekursive Lösung ist komplizierte und schwierig zu finden

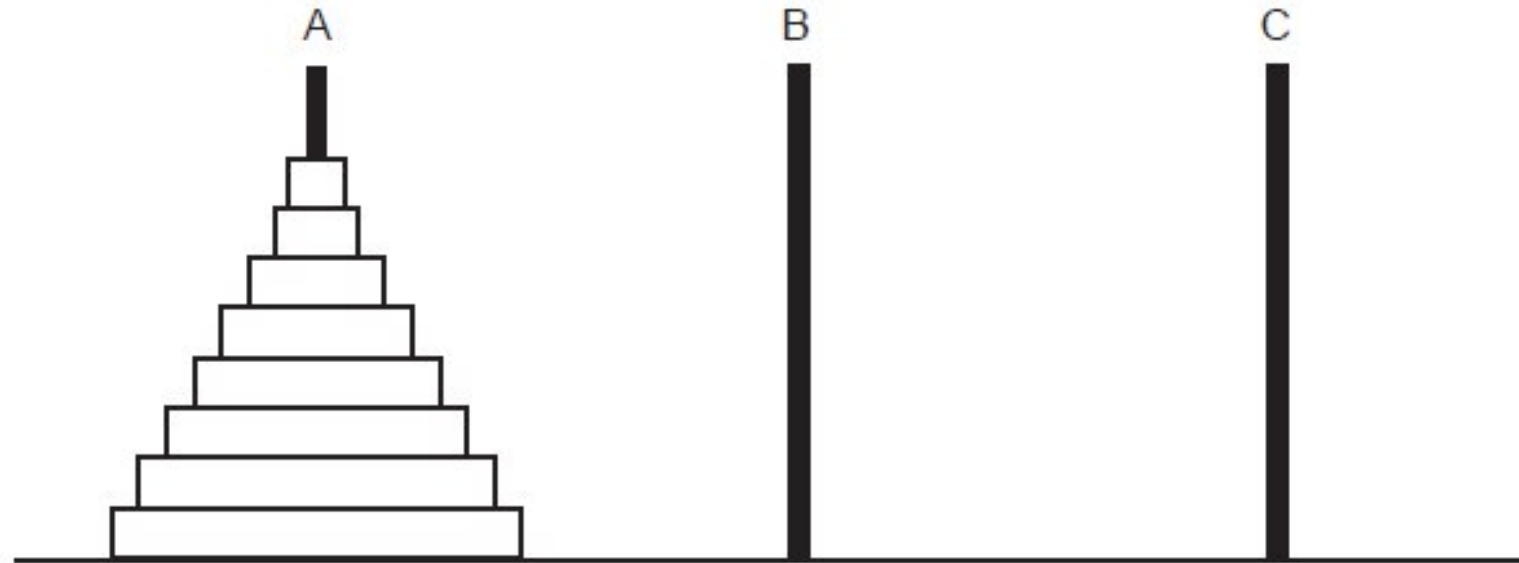


Abbildung 2.5: Türme von Hanoi



Die Türme von Hanoi



Die Geschichte: Buddhistische Mönche des Brahma-Tempels haben die Aufgabe, 64 Scheiben aus Gold, die ein Loch in der Mitte haben, von Stab A nach Stab B zu bringen. Stab C kann als Zwischenablage dienen.

Die Mönche müssen **zwei Regeln** beachten:

1. Es darf nur **eine Scheibe** zurzeit **bewegt** werden.
2. **Nie darf eine größere auf einer kleineren Scheibe** zu liegen kommen.

Die Legende sagt, dass das Ende der Welt kommt, wenn die Mönche ihre Aufgabe

Die Legende* sagt, dass das Ende der Welt kommt, wenn die Mönche ihre Aufgabe beendet haben.

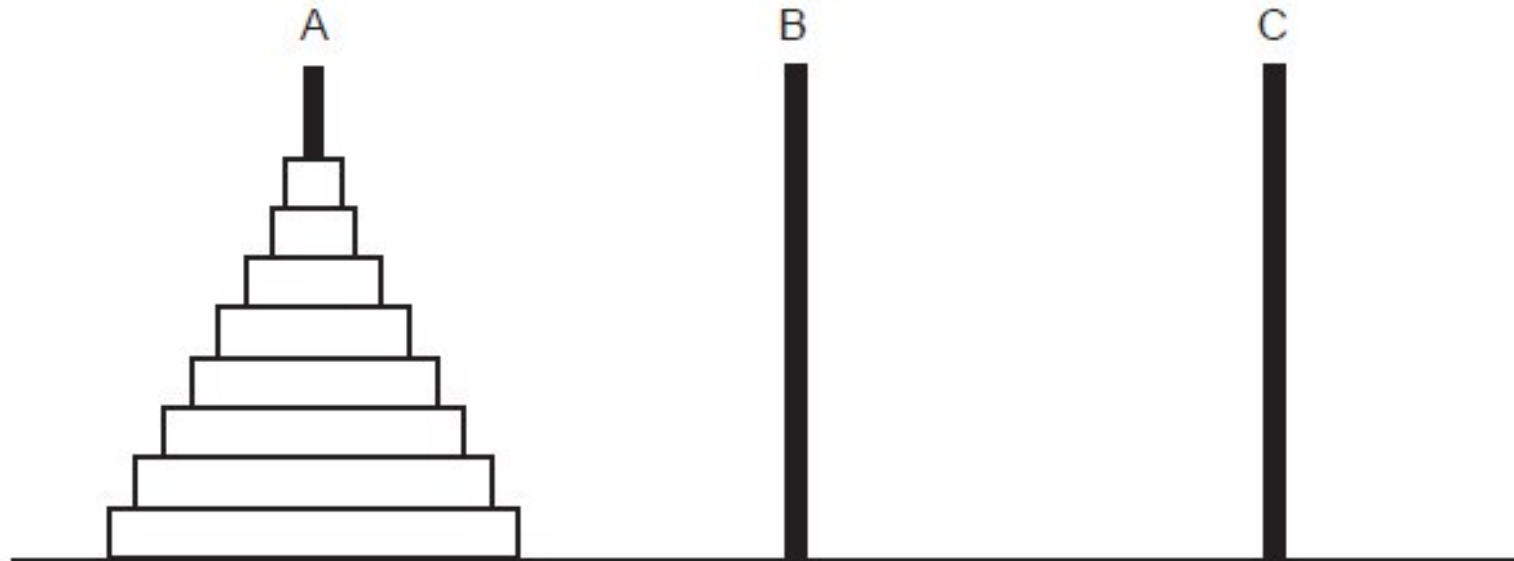


Abbildung 2.5: Türme von Hanoi



(*) Die Wahrheit über den Ursprung der „Legende“ finden Sie hier:
https://de.wikipedia.org/wiki/T%C3%BCrme_von_Hanoi
<http://www.meinelt-online.de/spiele/hanoi.html>



Die Türme von Hanoi (online spielen)



← → ↺ 🏠

🔒 https://www.mathematik.ch/spiele/hanoi_mit_grafik/

☰

 **mathematik.ch**

Türme von Hanoi

1883 erfand der französische Mathematiker Edouard Lucas das Problem der Türme von Hanoi.

Ziel des Spieles: Alle Scheiben vom Turm ganz links sollen auf den Turm ganz rechts bewegt werden.

Bedingungen:

1. Sie können nur eine Scheibe pro Zug verschieben.
2. Eine grössere Scheibe darf nie auf einer kleineren Scheibe liegen.

Zum Verschieben einer Scheibe:
Klicken Sie zuerst auf den Turm, von dem die oberste Scheibe entfernt werden soll.
Klicken Sie dann auf den Turm, auf den die Scheibe platziert werden soll.

Falls Sie Ihre Zeit (in Sekunden) messen wollen, so aktivieren Sie die Checkbox 'mit Stoppuhr'. Bei Ihrem **ersten** Zug wird die Uhr dann gestartet, beim Erreichen des Zieles gestoppt.

Anzahl Scheiben (3 bis 10):



Anzahl Züge: 0

Bewegen Sie alle Scheiben auf den Turm ganz rechts.

☐ mit Stoppuhr

Stoppuhr: 0.0

Stellt Anfangszustand her

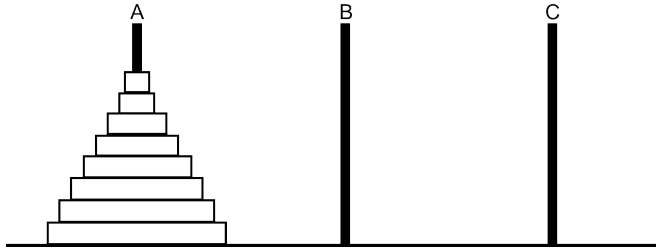
©1997 - 2024 www.mathematik.ch | 

https://www.mathematik.ch/spiele/hanoi_mit_grafik/

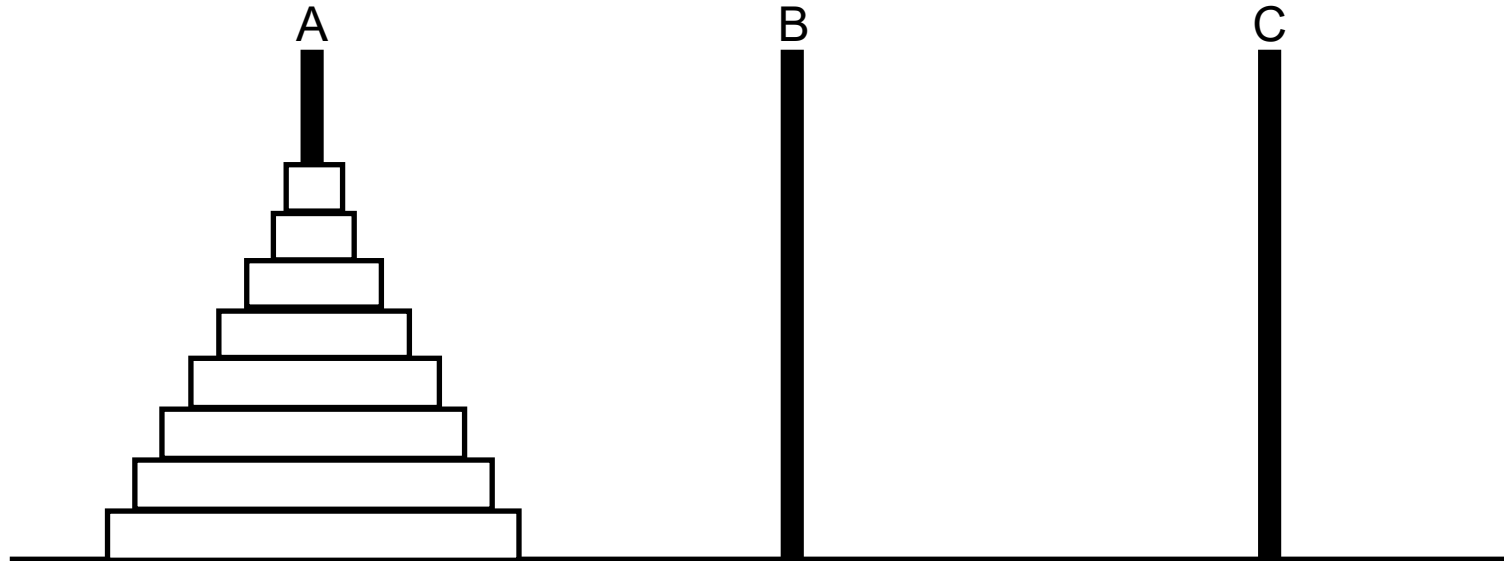
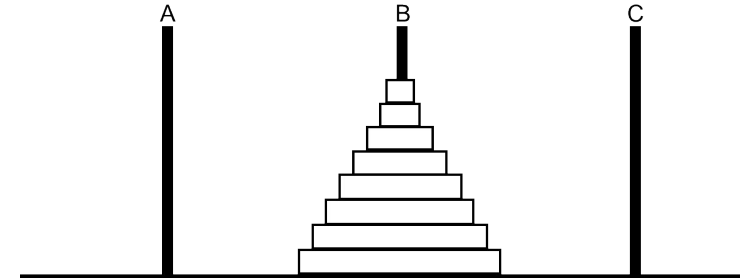
Rekursive Lösung: Türme von Hanoi



Start:



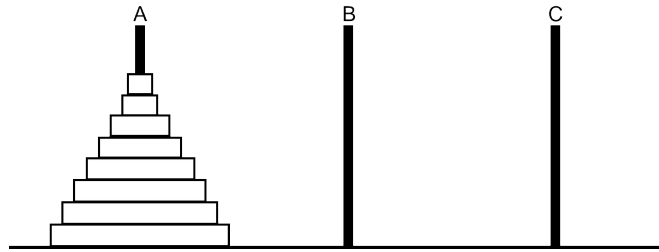
Ziel



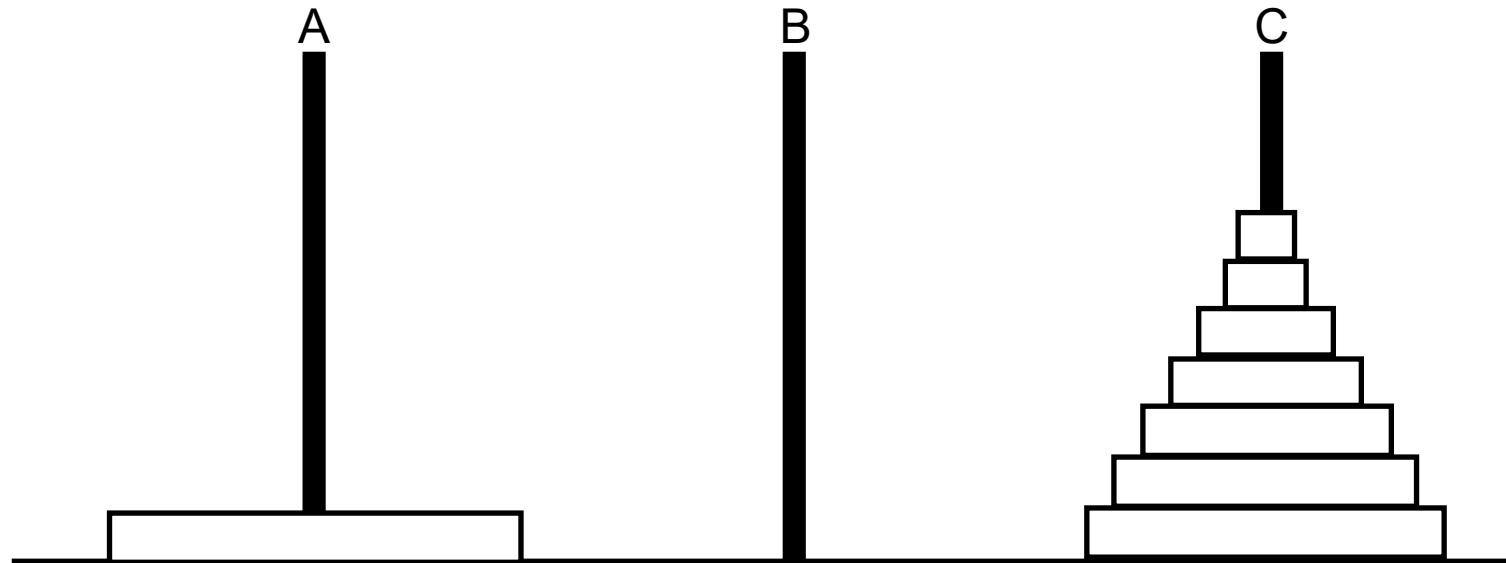
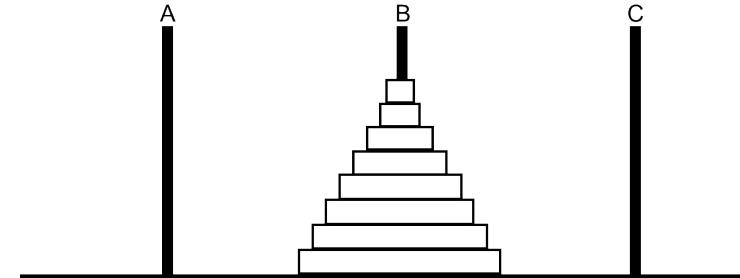
Rekursive Lösung: Türme von Hanoi



Start:



Ziel

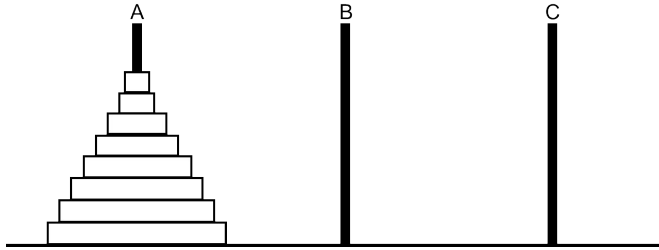


Wenn wir wüssten, wie man die oberen 7 bis auf die letzte Schreibe auf Stab C bringen könnte, wäre das Problem schon einfacher...

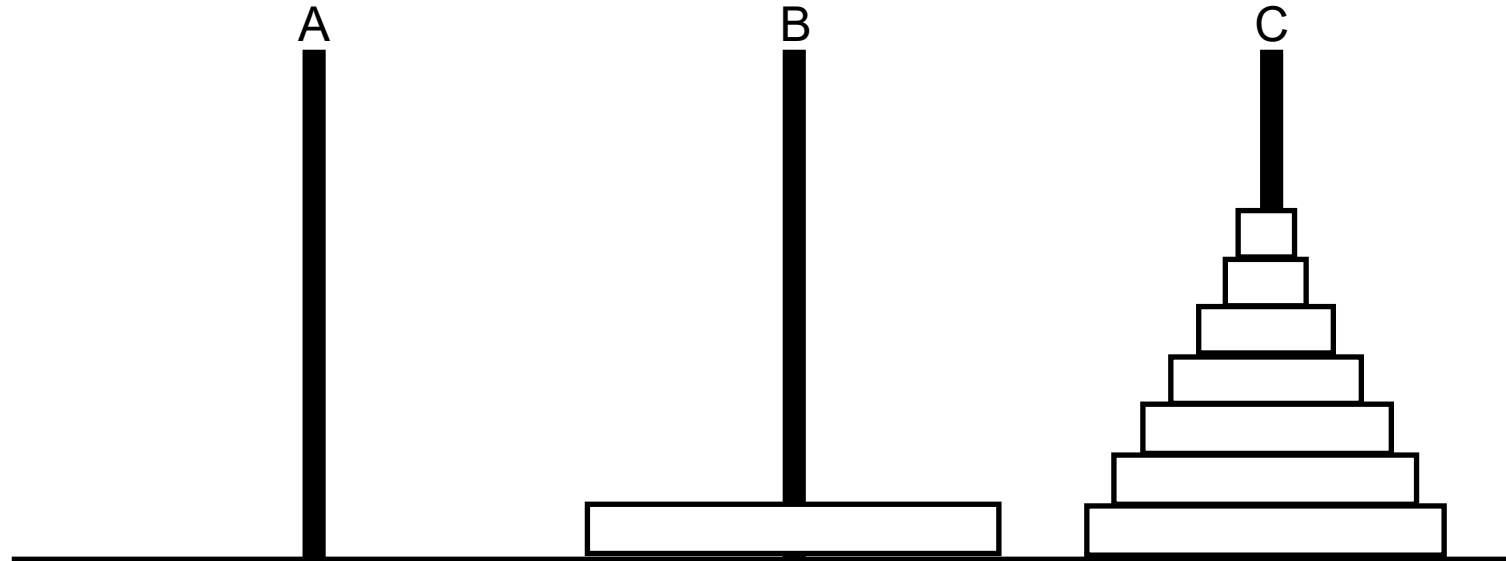
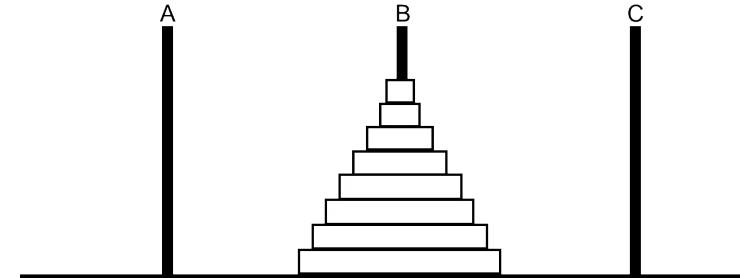
Rekursive Lösung: Türme von Hanoi



Start:



Ziel

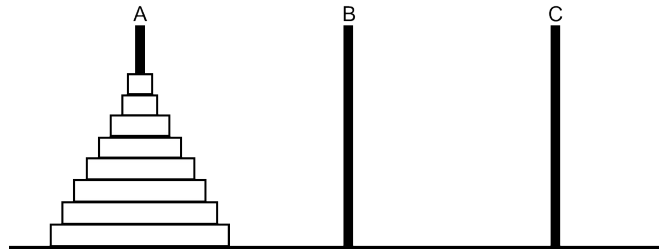


Wenn wir wüssten, wie man die oberen 7 bis auf die letzte Schreibe auf Stab C bringen könnte, wäre das Problem schon einfacher... dann nur Scheibe 8 nach B

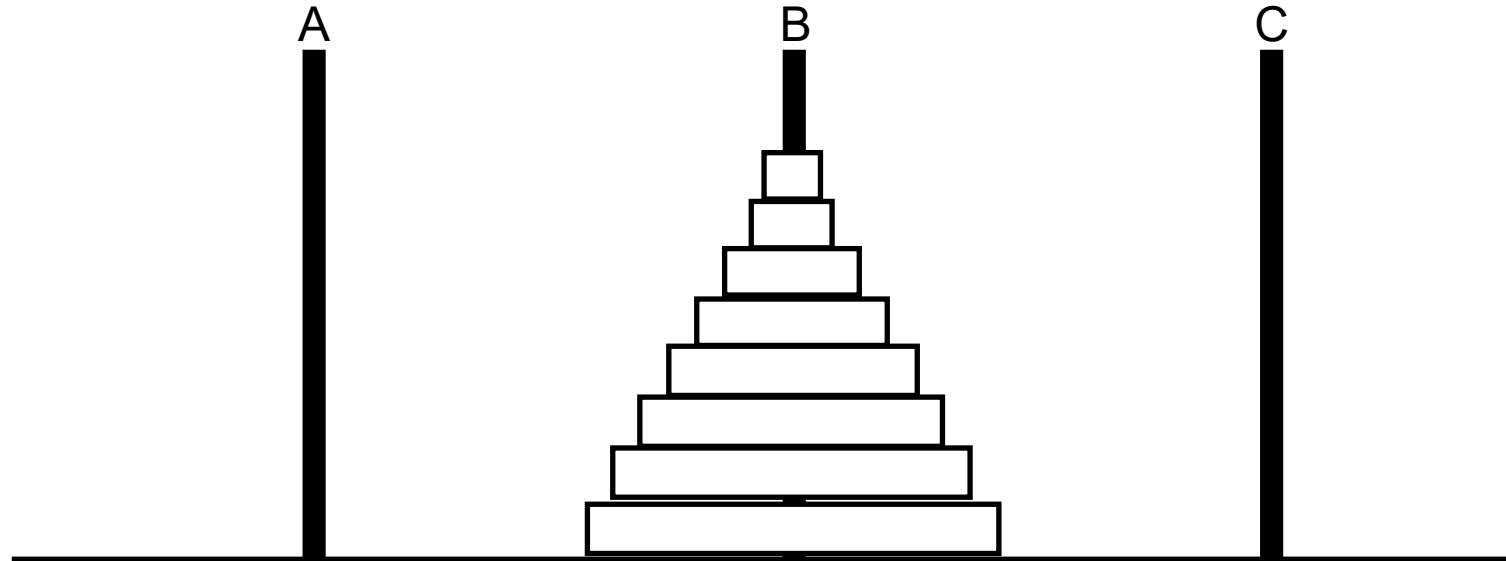
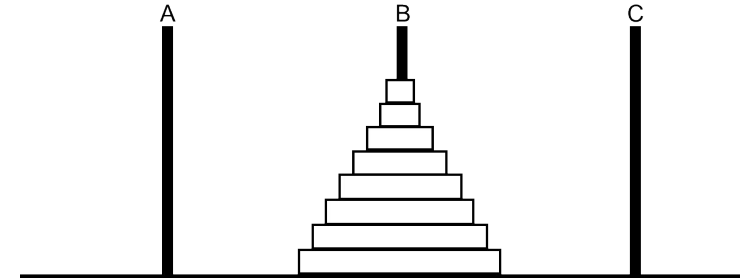
Rekursive Lösung: Türme von Hanoi



Start:



Ziel

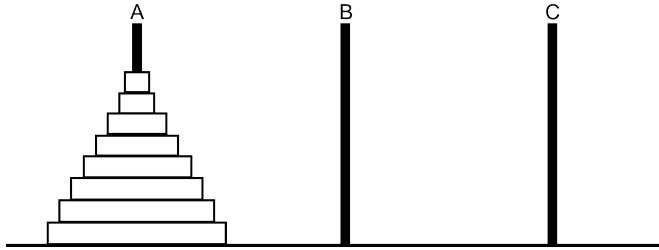


Wenn wir wüssten, wie man die oberen 7 bis auf die letzte Scheibe auf Stab C bringen könnte, wäre das Problem schon einfacher... dann nur Scheibe 8 nach B .. dann 7 Scheiben nach B.

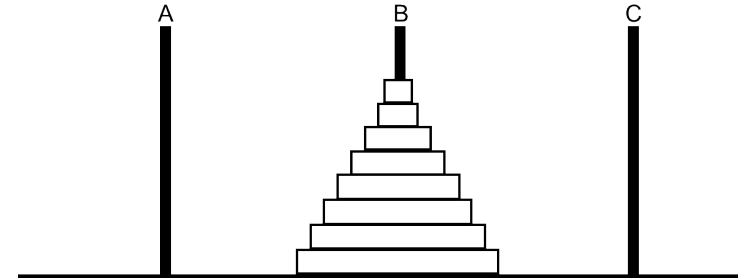
Rekursive Lösung: Türme von Hanoi



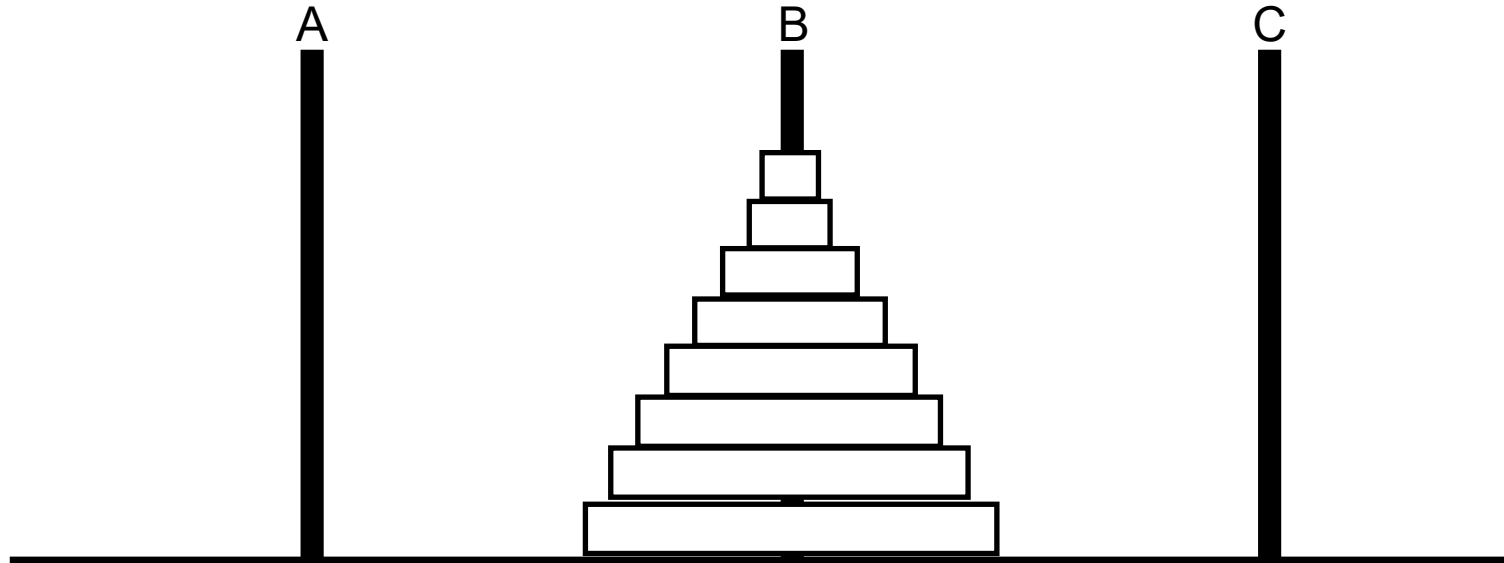
Start:



Ziel



Start:
8 Scheiben nach B



Wenn wir wüssten, wie man die oberen 7 bis auf die letzte Schreibe auf Stab C bringen könnte, wäre das Problem schon einfacher... dann nur Scheibe 8 nach B .. dann 7 Scheiben nach B.

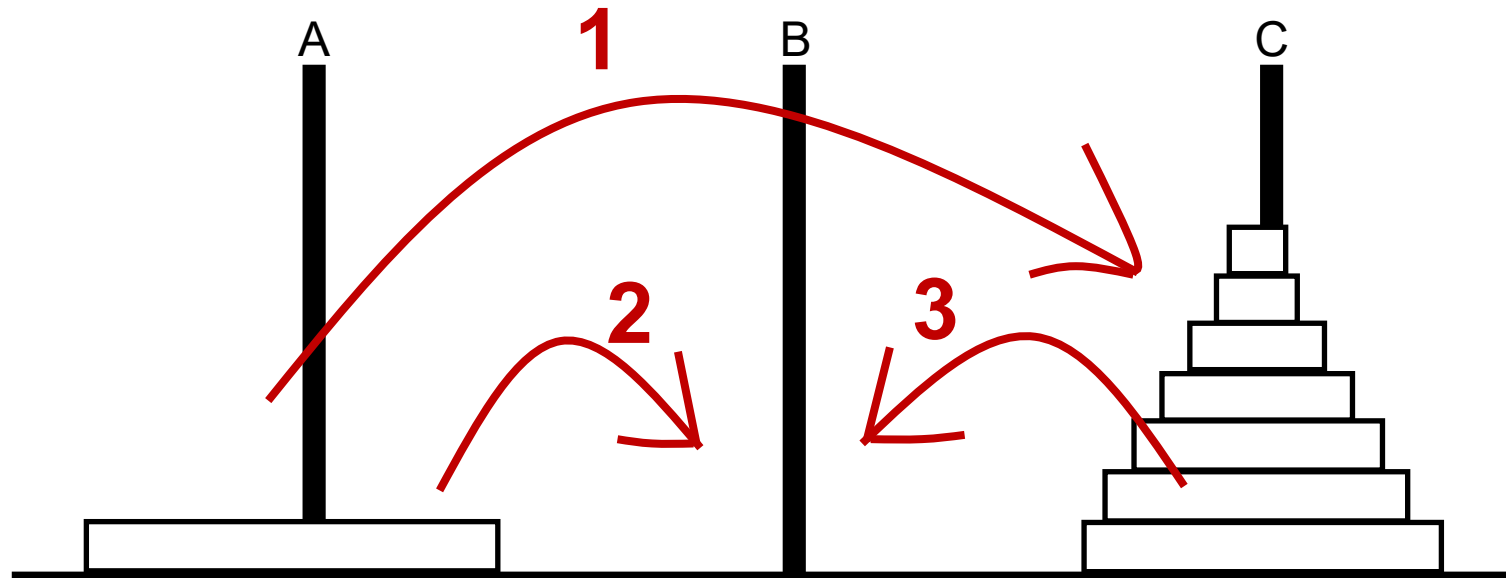


Rekursive Lösung: Türme von Hanoi



Aufgabe: 8 Scheiben von A nach B:

- 7 bis auf die letzte Schreibe von Stab A auf Stab C; 8te von Stab A nach Stab B; 7 Scheiben von Stab C nach B.

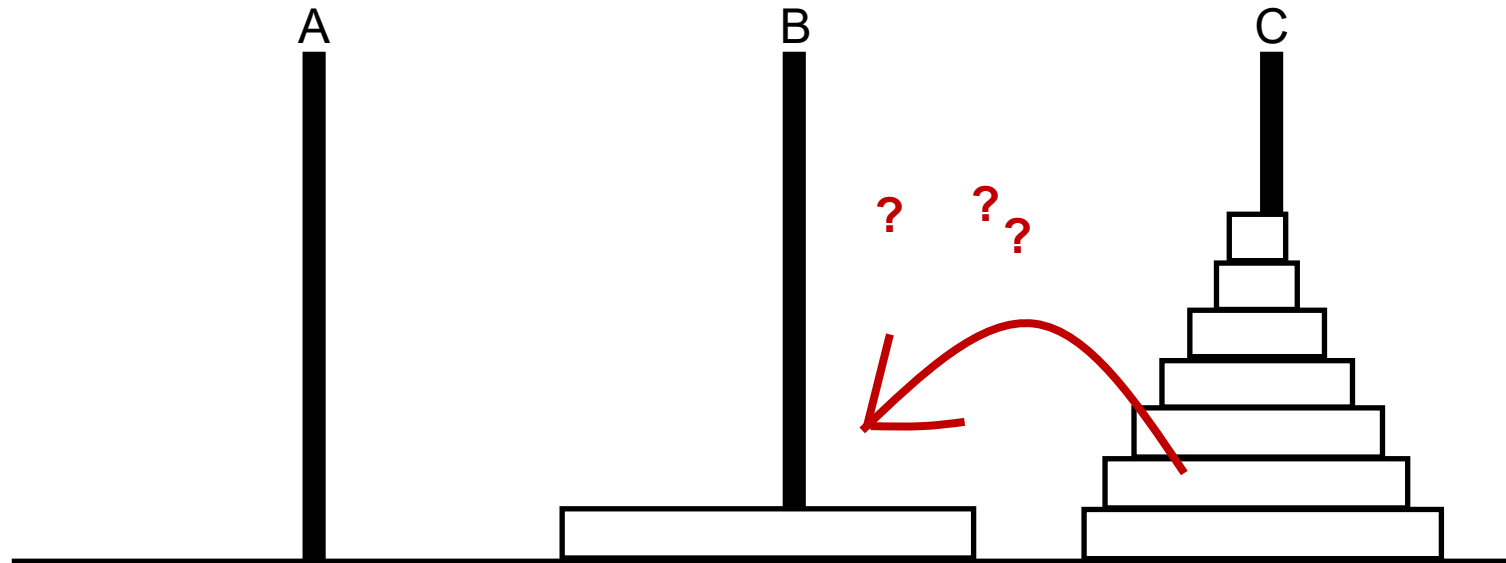


Rekursive Lösung: Türme von Hanoi



Aufgabe: 8 Scheiben von A nach B:

- 7 bis auf die letzte Schreibe von Stab A auf Stab C; 8te von Stab A nach Stab B; 7 Scheiben von Stab C nach B.

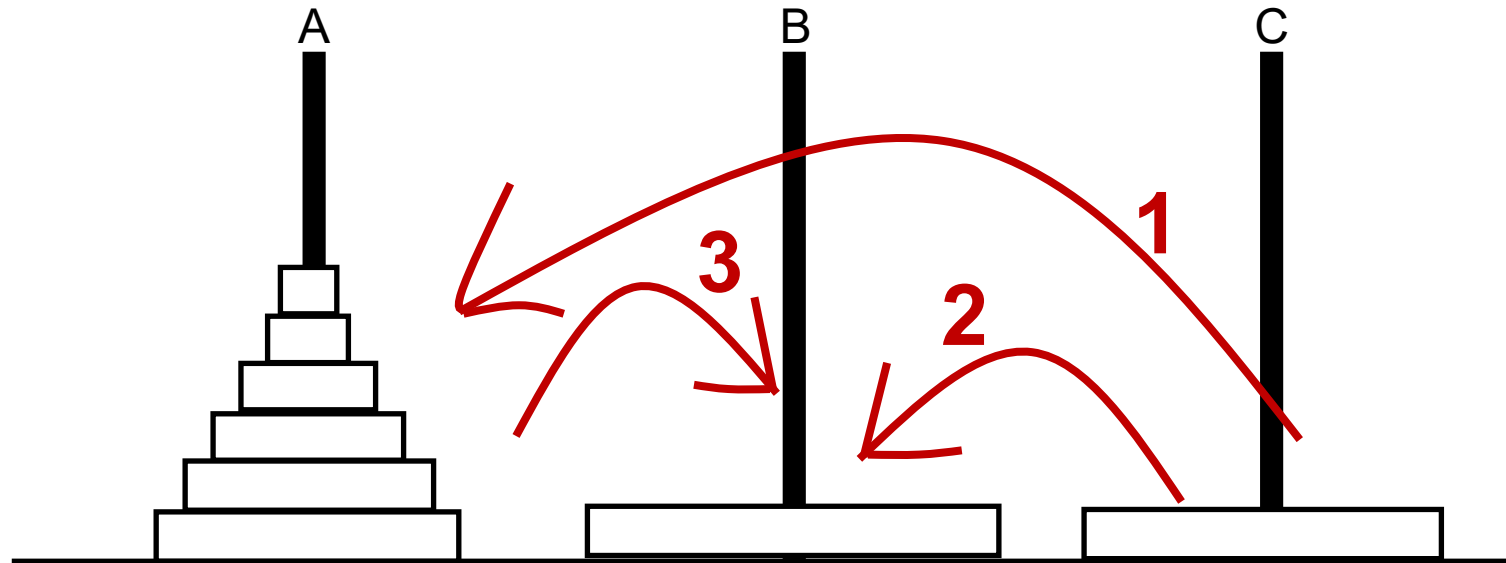


Rekursive Lösung: Türme von Hanoi



Aufgabe: 8 Scheiben von A nach B:

- 7 bis auf die letzte Schreibe von Stab A auf Stab C; 8te von Stab A nach Stab B; 7 Scheiben von Stab C nach B.
- 6 bis auf die letzte Schreibe von Stab C auf Stab A; 7te von Stab C nach Stab B; 6 Scheiben von Stab A nach B

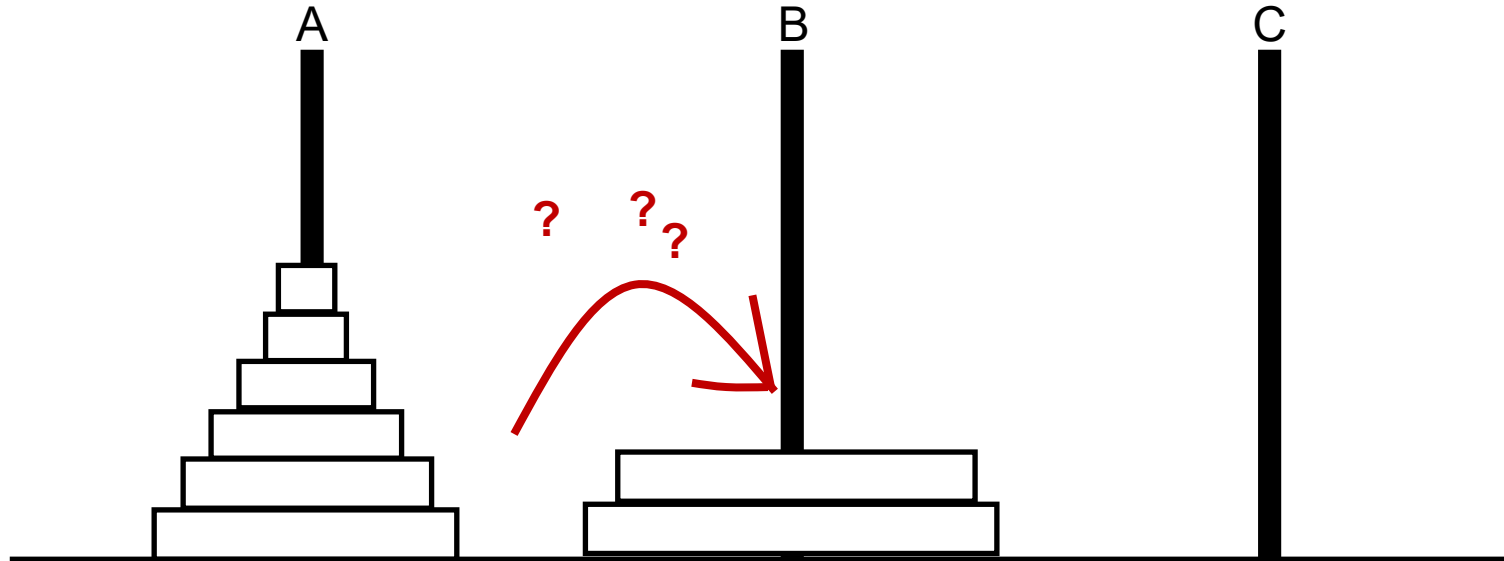


Rekursive Lösung: Türme von Hanoi



Aufgabe: 8 Scheiben von A nach B:

- 7 bis auf die letzte Schreibe von Stab A auf Stab C; 8te von Stab A nach Stab B; 7 Scheiben von Stab C nach B.
- 6 bis auf die letzte Schreibe von Stab C auf Stab A; 7te von Stab C nach Stab B; 6 Scheiben von Stab A nach B

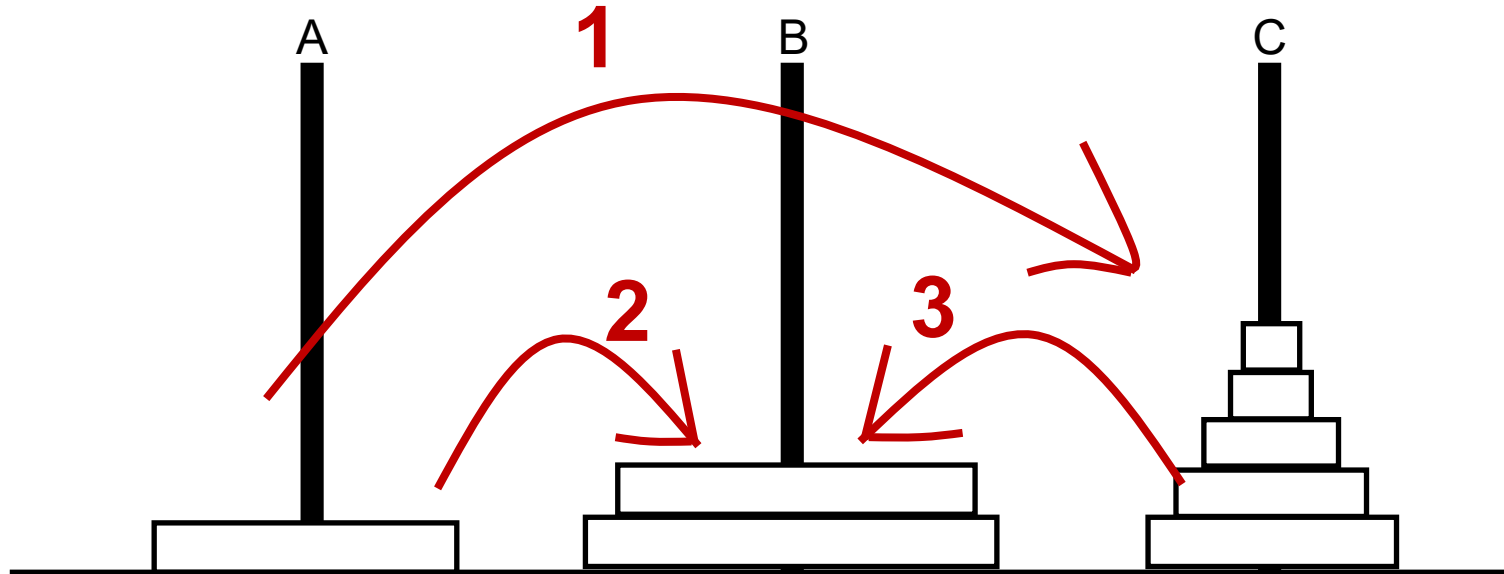


Rekursive Lösung: Türme von Hanoi



Aufgabe: 8 Scheiben von A nach B:

- 7 bis auf die letzte Schreibe von Stab A auf Stab C; 8te von Stab A nach Stab B; 7 Scheiben von Stab C nach B.
- 6 bis auf die letzte Schreibe von Stab C auf Stab A; 7te von Stab C nach Stab B; 6 Scheiben von Stab A nach B
- 5 bis auf die letzte Schreibe von Stab A auf Stab C; 6te von Stab A nach Stab B; 5 Scheiben von Stab C nach B



Rekursive Lösung: Türme von Hanoi



Aufgabe: 8 Scheiben von A nach B:

- 7 bis auf die letzte Schreibe von Stab A auf Stab C; 8te von Stab A nach Stab B; 7 Scheiben von Stab C nach B.
 - 6 bis auf die letzte Schreibe von Stab C auf Stab A; 7te von Stab C nach Stab B; 6 Scheiben von Stab A nach B
 - 5 bis auf die letzte Schreibe von Stab A auf Stab C; 6te von Stab A nach Stab B; 5 Scheiben von Stab C nach B
- USW...

Abbruchbedingung: Nur noch eine Scheibe muss bewegt werden, das geht nach Regeln.

Rekursive Lösung: Türme von Hanoi



Aufgabe: 8 Scheiben von A nach B:

- 7 bis auf die letzte Schreibe von Stab A auf Stab C; 8te von Stab A nach Stab B; 7 Scheiben von Stab C nach B.
 - 6 bis auf die letzte Schreibe von Stab C auf Stab A; 7te von Stab C nach Stab B; 6 Scheiben von Stab A nach B
 - 5 bis auf die letzte Schreibe von Stab A auf Stab C; 6te von Stab A nach Stab B; 5 Scheiben von Stab C nach B
- USW...

1 Abbruchbedingung: Nur noch eine Scheibe muss bewegt werden, das geht nach Regeln.

```
if (wieviele == 1) {  
    cout << "Bewege Scheibe von " << StartStapel << " nach " << ZielStapel << endl;  
    return;  
}
```

Rekursive Lösung: Türme von Hanoi



2

Aufgabe: 8 Scheiben von A nach B:

- 7 bis auf die letzte Scheibe von Stab A auf Stab C; 8te von Stab A nach Stab B; 7 Scheiben von Stab C nach B.
 - 6 bis auf die letzte Scheibe von Stab C auf Stab A; 7te von Stab C nach Stab B; 6 Scheiben von Stab A nach B
 - 5 bis auf die letzte Scheibe von Stab A auf Stab C; 6te von Stab A nach Stab B; 5 Scheiben von Stab C nach B
- USW...

1 Abbruchbedingung: Nur noch eine Scheibe muss bewegt werden, das geht nach Regeln.

```
if (wieviele == 1) {  
    cout << "Bewege Scheibe von " << StartStapel << " nach " << ZielStapel << endl;  
    return;  
}  
Bewege(StartStapel, ArbeitsStapel, wieviele-1);
```

1

2

Rekursive Lösung: Türme von Hanoi



Aufgabe: 8 Scheiben von A nach B:

2

3

- 7 bis auf die letzte Schreibe von Stab A auf Stab C; 8te von Stab A nach Stab B; 7 Scheiben von Stab C nach B.
 - 6 bis auf die letzte Schreibe von Stab C auf Stab A; 7te von Stab C nach Stab B; 6 Scheiben von Stab A nach B
 - 5 bis auf die letzte Schreibe von Stab A auf Stab C; 6te von Stab A nach Stab B; 5 Scheiben von Stab C nach B
- USW...

1 Abbruchbedingung: Nur noch eine Scheibe muss bewegt werden, das geht nach Regeln.

```
if (wieviele == 1) {  
    cout << "Bewege Scheibe von " << StartStapel << " nach " << ZielStapel << endl;  
    return;  
}  
Bewege(StartStapel, ArbeitsStapel, wieviele-1);  
Bewege(StartStapel, ZielStapel, 1);
```

1

2

3

Rekursive Lösung: Türme von Hanoi



2

3

4

Aufgabe: 8 Scheiben von A nach B:

- 7 bis auf die letzte Scheibe von Stab A auf Stab C; 8te von Stab A nach Stab B; 7 Scheiben von Stab C nach B.
 - 6 bis auf die letzte Scheibe von Stab C auf Stab A; 7te von Stab C nach Stab B; 6 Scheiben von Stab A nach B
 - 5 bis auf die letzte Scheibe von Stab A auf Stab C; 6te von Stab A nach Stab B; 5 Scheiben von Stab C nach B
- USW...

1 Abbruchbedingung: Nur noch eine Scheibe muss bewegt werden, das geht nach Regeln.

```
if (wieviele == 1) {  
    cout << "Bewege Scheibe von " << StartStapel << " nach " << ZielStapel << endl;  
    return;  
}  
Bewege(StartStapel, ArbeitsStapel, wieviele-1);  
Bewege(StartStapel, ZielStapel, 1);  
Bewege(ArbeitsStapel, ZielStapel, wieviele-1);  
}
```

Rekursive Lösung: Türme von Hanoi



2

3

4

Aufgabe: 8 Scheiben von A nach B:

- 7 bis auf die letzte Scheibe von Stab A auf Stab C; 8te von Stab A nach Stab B; 7 Scheiben von Stab C nach B.
 - 6 bis auf die letzte Scheibe von Stab C auf Stab A; 7te von Stab C nach Stab B; 6 Scheiben von Stab A nach B
 - 5 bis auf die letzte Scheibe von Stab A auf Stab C; 6te von Stab A nach Stab B; 5 Scheiben von Stab C nach B
- USW...

1 Abbruchbedingung: Nur noch eine Scheibe muss bewegt werden, das geht nach Regeln.

5

Information über den jeweiligen Arbeitsstapel

```
void Bewege(char StartStapel, char ZielStapel, char ArbeitsStapel, int wieviele) {  
    if (wieviele == 1) {  
        cout << "Bewege Scheibe von " << StartStapel << " nach " << ZielStapel << endl;  
        return;  
    }  
    Bewege(StartStapel, ArbeitsStapel, ZielStapel, wieviele-1);  
    Bewege(StartStapel, ZielStapel, ArbeitsStapel, 1);  
    Bewege(ArbeitsStapel, ZielStapel, StartStapel, wieviele-1);  
}
```

Rekursive Lösung: Türme von Hanoi



Lösung für 3 Scheiben:

```
C:\EigeneDaten\Vorlesung_2024_SS\Informatik2\Uebungsaufgaben>TuermeVonHanoi.exe  
Bewege Scheibe von A nach B  
Bewege Scheibe von A nach C  
Bewege Scheibe von B nach C  
Bewege Scheibe von A nach B  
Bewege Scheibe von C nach A  
Bewege Scheibe von C nach B  
Bewege Scheibe von A nach B
```

Die Gesamtzahl der Bewegungen für k Scheiben ist gegeben durch $N = 2^k - 1$.

Somit bei 64 Goldscheiben bräuchten die Mönche (1s pro Zug) ungefähr $5,86 \cdot 10^{11}$ Jahre. Das entspricht etwa dem 50-fachen Alter unseres Universums.

Rekursive Lösung: Türme von Hanoi



Lösung für 3 Scheiben:

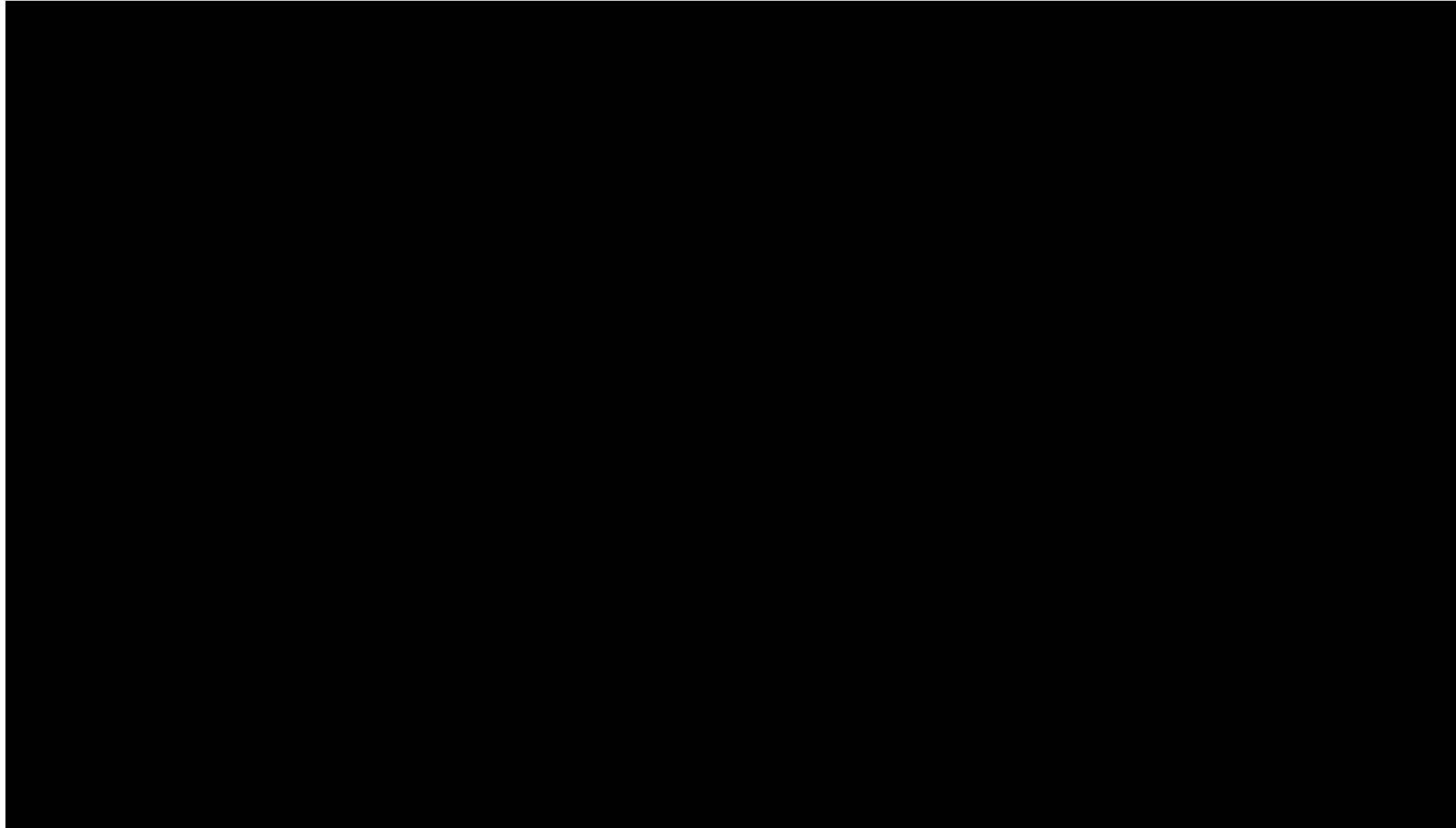
```
C:\EigeneDaten\Vorlesung_2024_SS\Informatik2\Uebungsaufgaben>TuermeVonHanoi.exe
Bewege Scheibe von A nach B
Bewege Scheibe von A nach C
Bewege Scheibe von B nach C
Bewege Scheibe von A nach B
Bewege Scheibe von C nach A
Bewege Scheibe von C nach B
Bewege Scheibe von A nach B

Bewege 3 Scheibe(n) von A nach B
Bewege 2 Scheibe(n) von A nach C
*** Bewege einzelne Scheibe von A nach B
*** Bewege einzelne Scheibe von A nach C
Bewege 2 Scheibe(n) von B nach C
*** Bewege einzelne Scheibe von B nach C
*** Bewege einzelne Scheibe von A nach B
Bewege 3 Scheibe(n) von C nach B
Bewege 2 Scheibe(n) von C nach B
*** Bewege einzelne Scheibe von C nach A
*** Bewege einzelne Scheibe von C nach B
Bewege 2 Scheibe(n) von A nach B
*** Bewege einzelne Scheibe von A nach B
```

Die Gesamtzahl der Bewegungen für k Scheiben ist gegeben durch $N = 2^k - 1$.

Somit bei 64 Goldscheiben bräuchten die Mönche (1s pro Zug) ungefähr $5,86 \cdot 10^{11}$ Jahre. Das entspricht etwa dem 50-fachen Alter unseres Universums.

Rekursive Lösung: Türme von Hanoi



Rekursive Baumsuche/- ausgabe

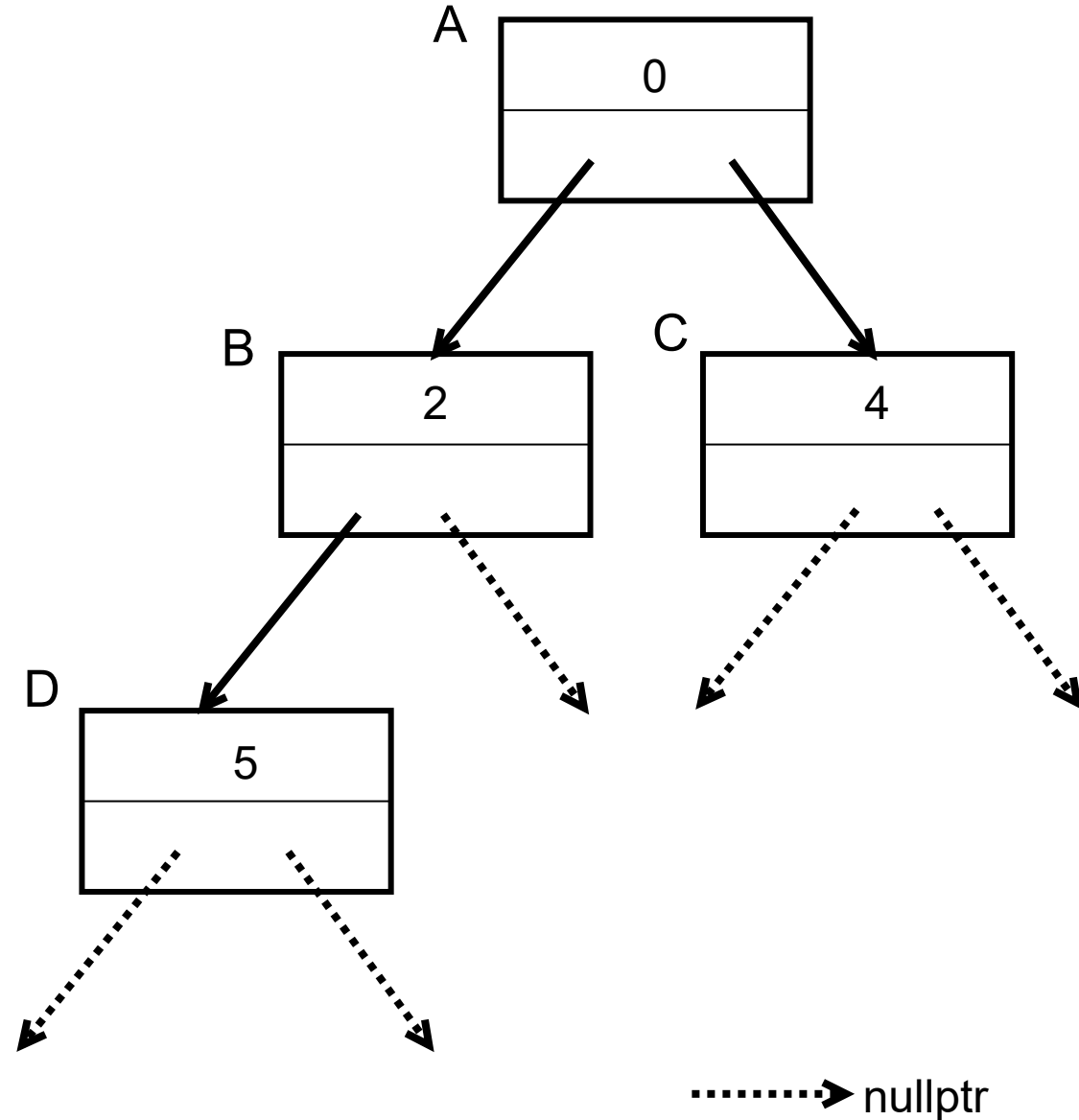
Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;

struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
}
```



Rekursive Baumsuche/-ausgabe



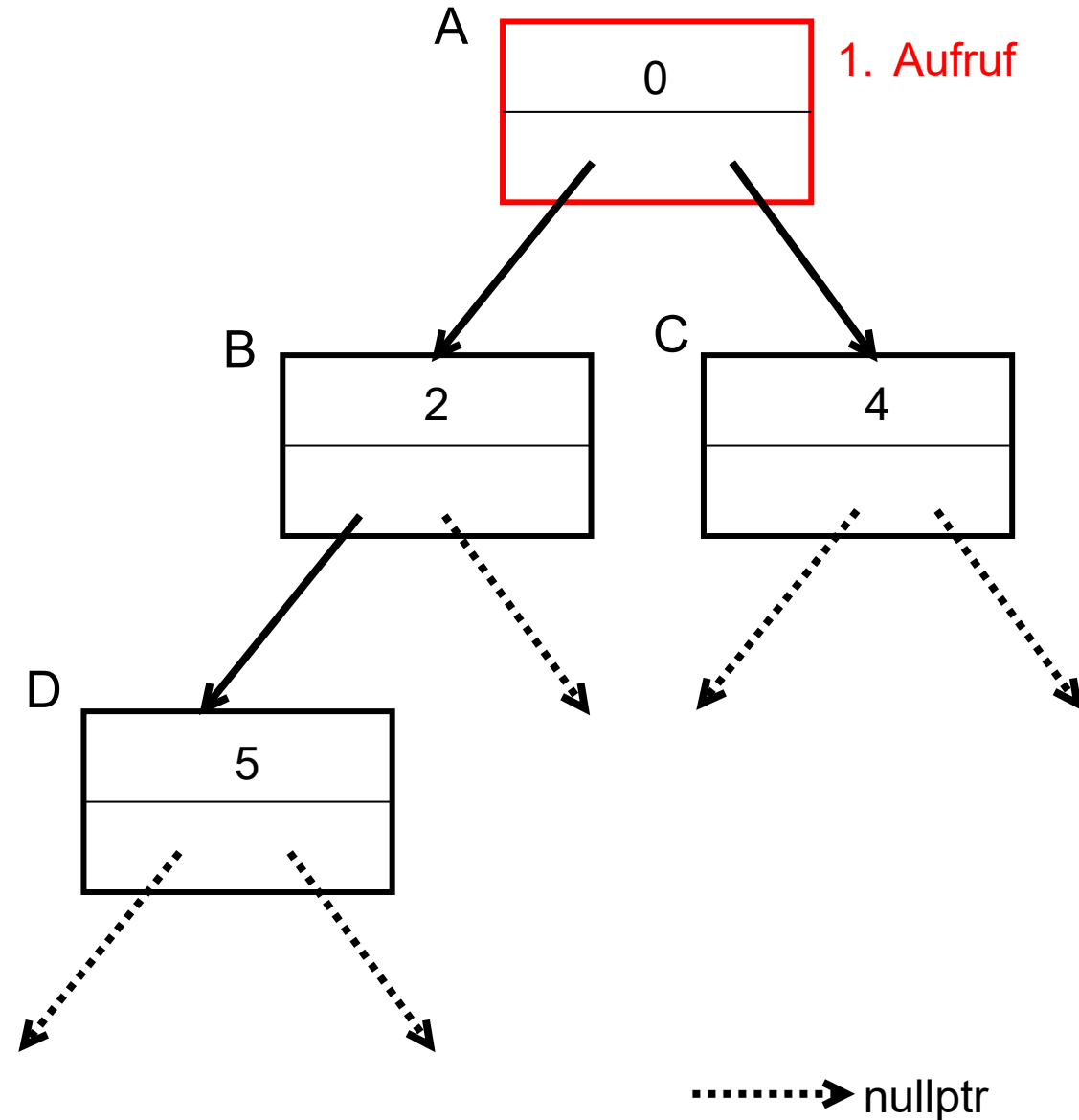
```
#include <iostream>
using namespace std;

struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

void Baumausgabe(Node* wobinich) {

}

int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



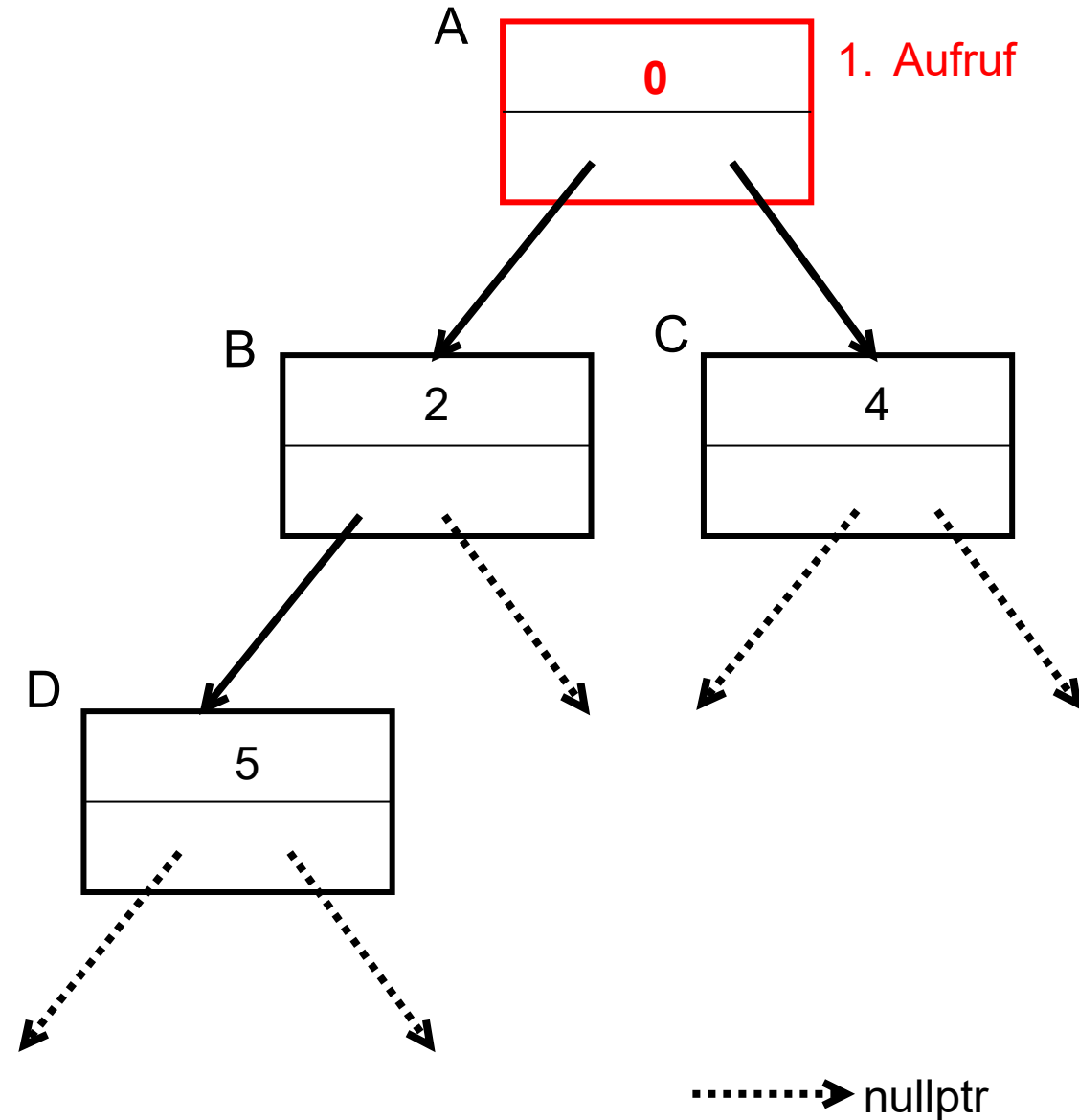
```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
```



```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

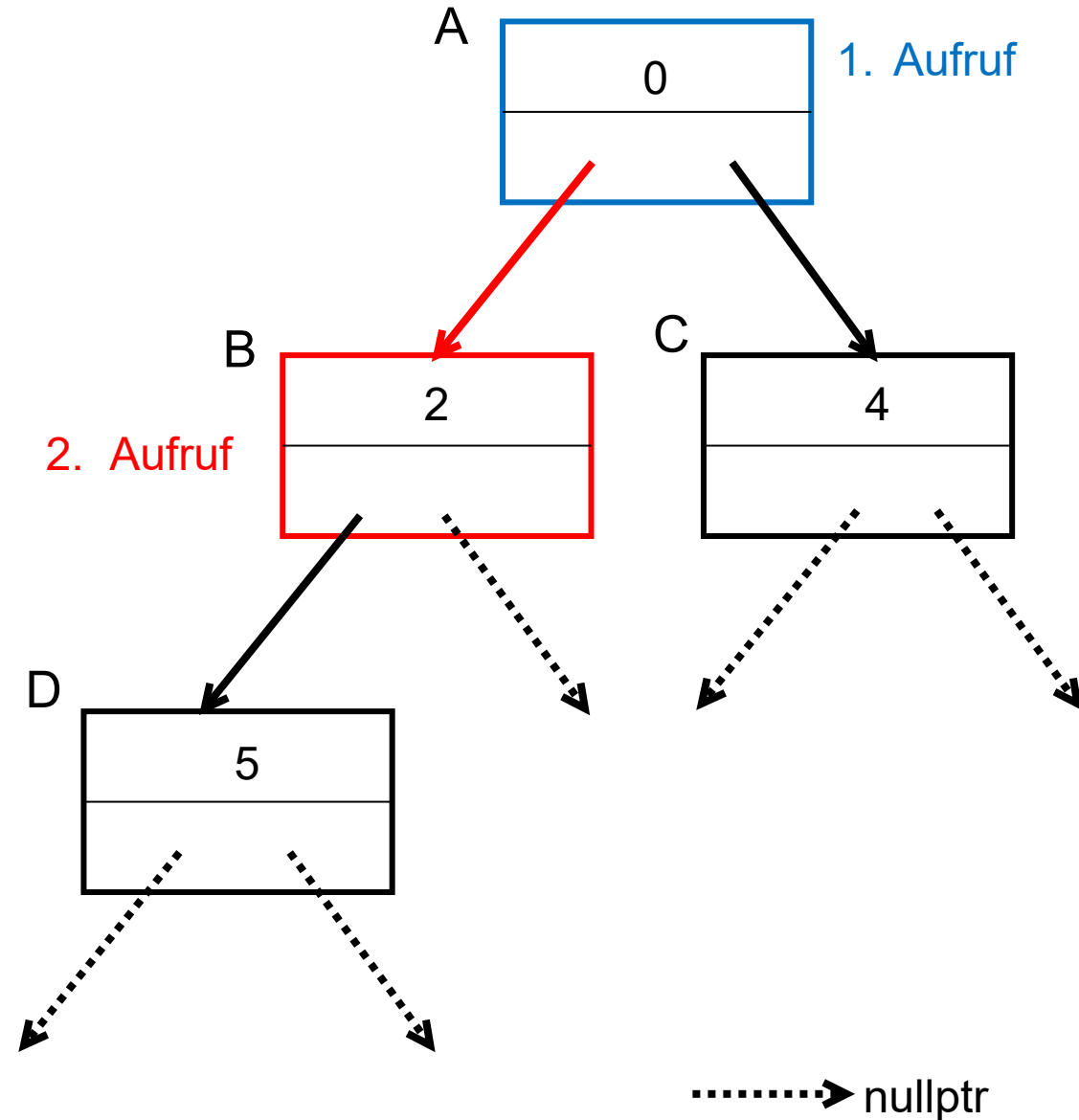


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



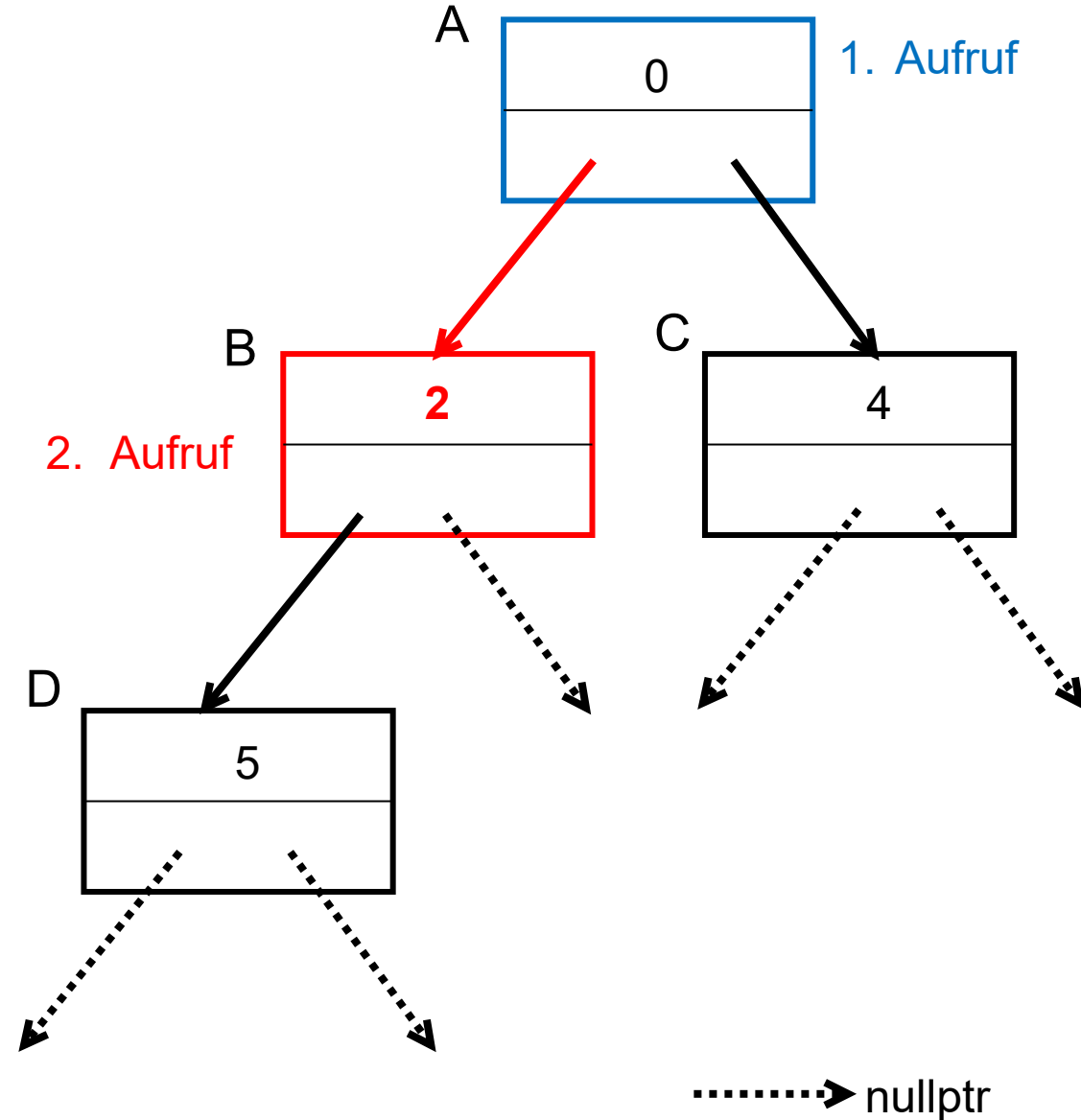
```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
```



```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

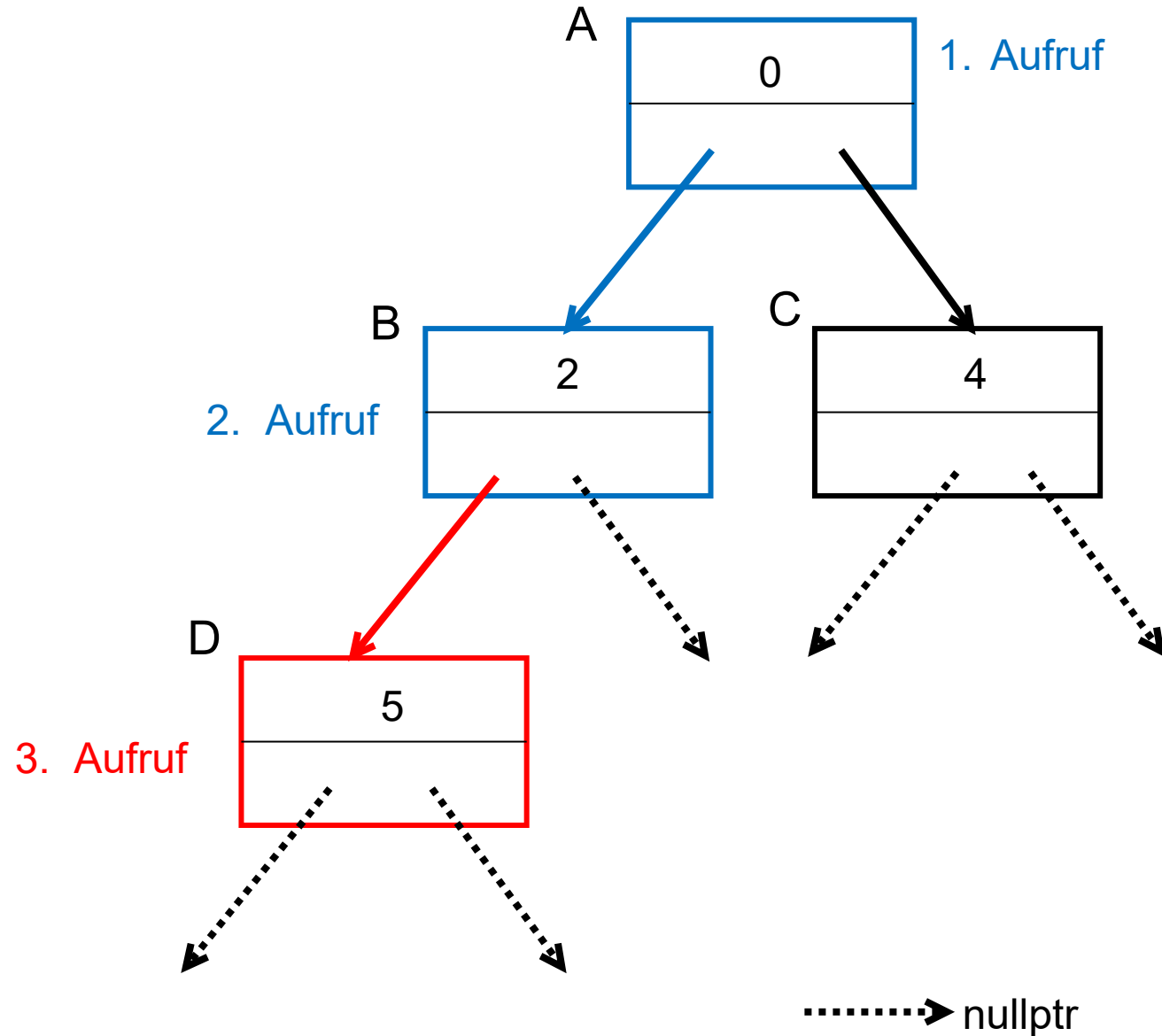


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

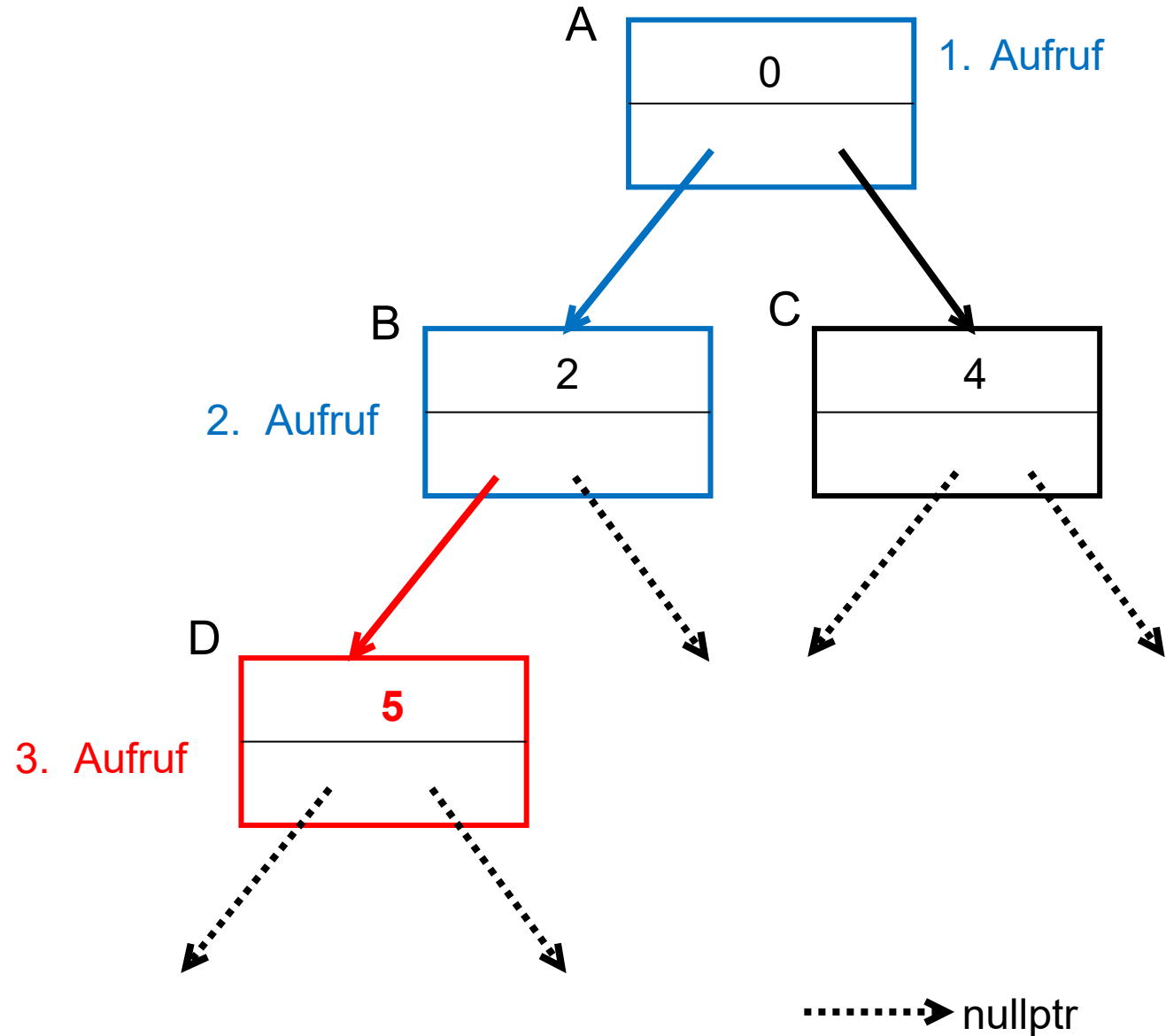


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

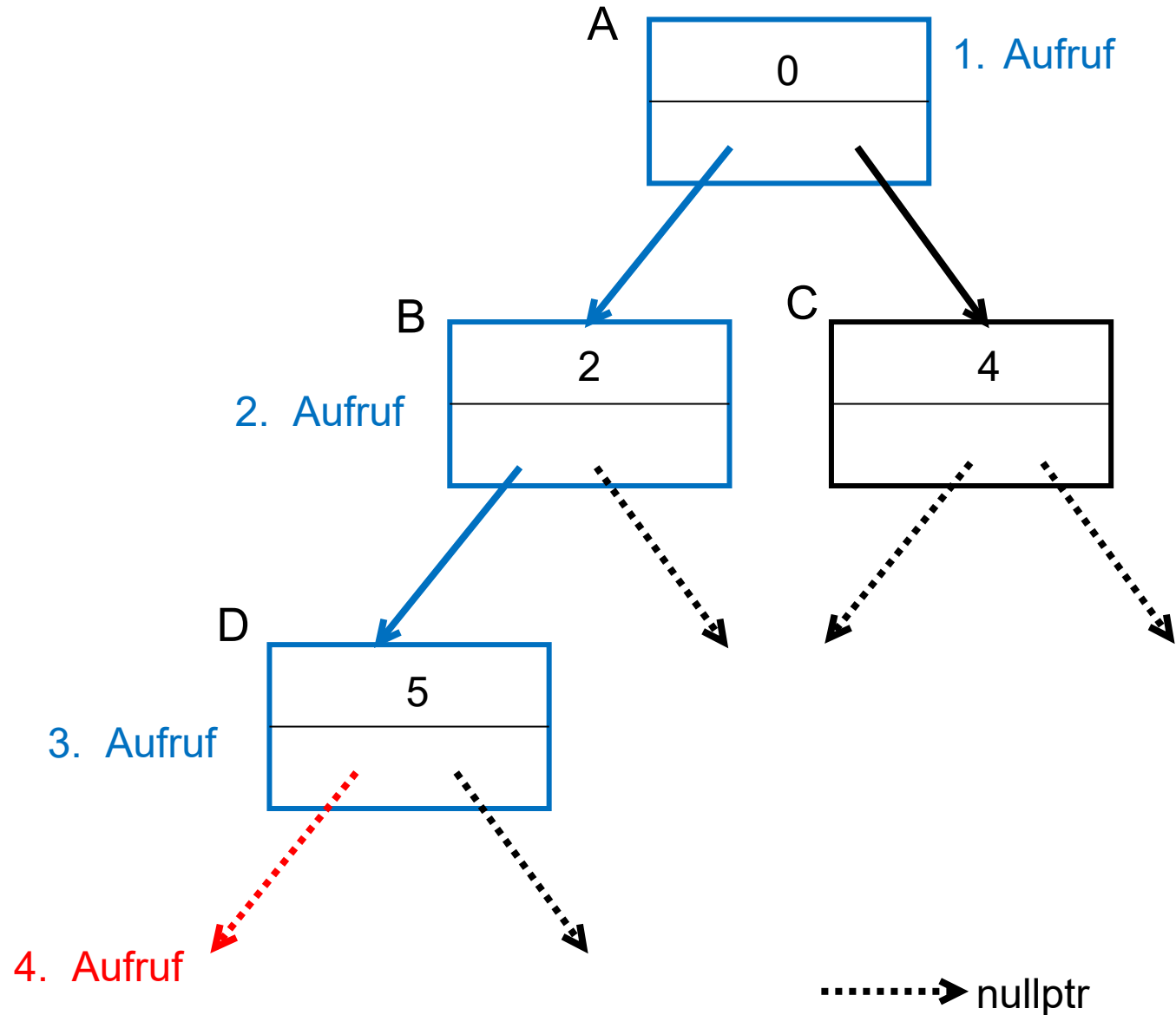


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

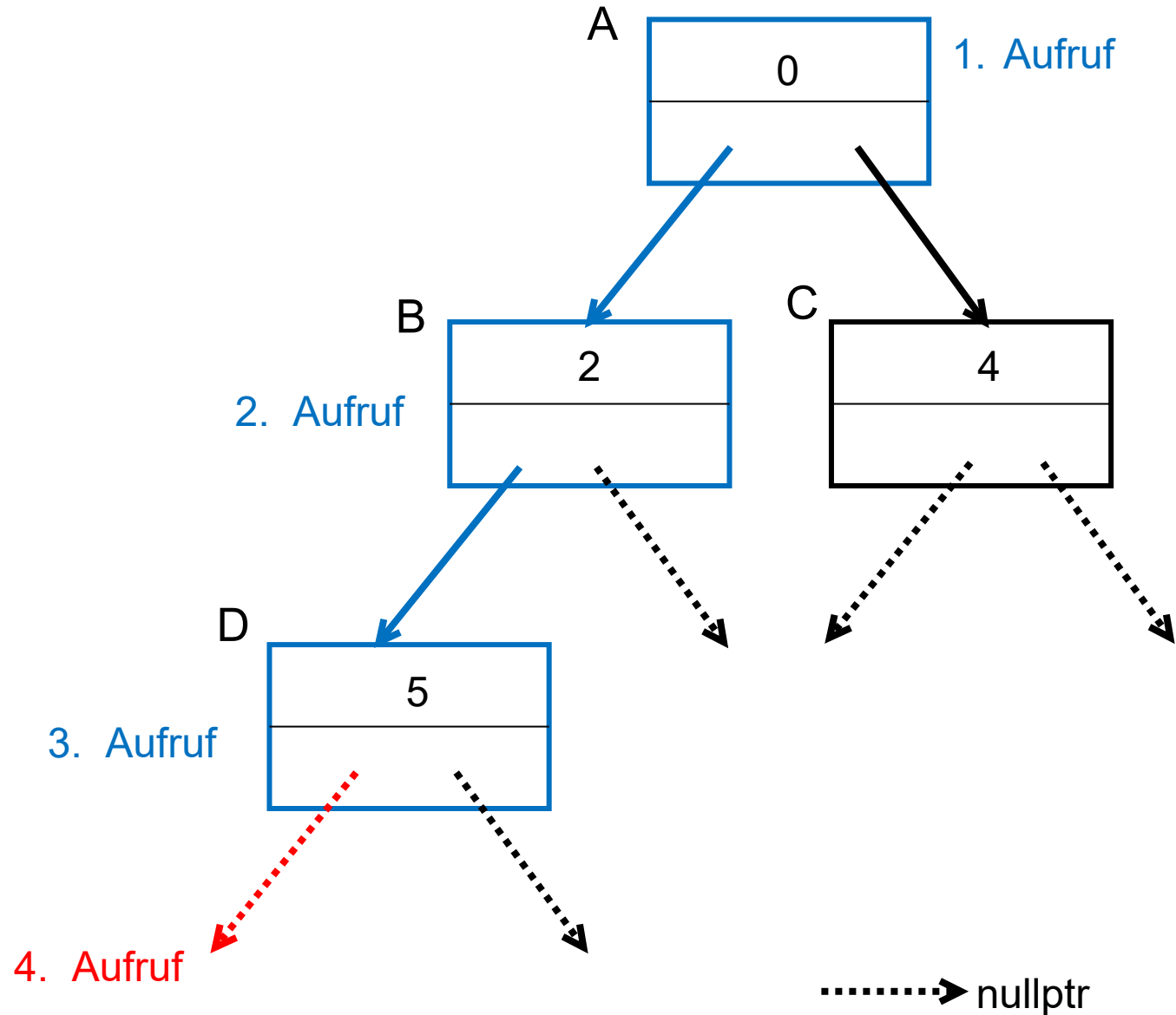


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) { ←
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
    else { return; }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}

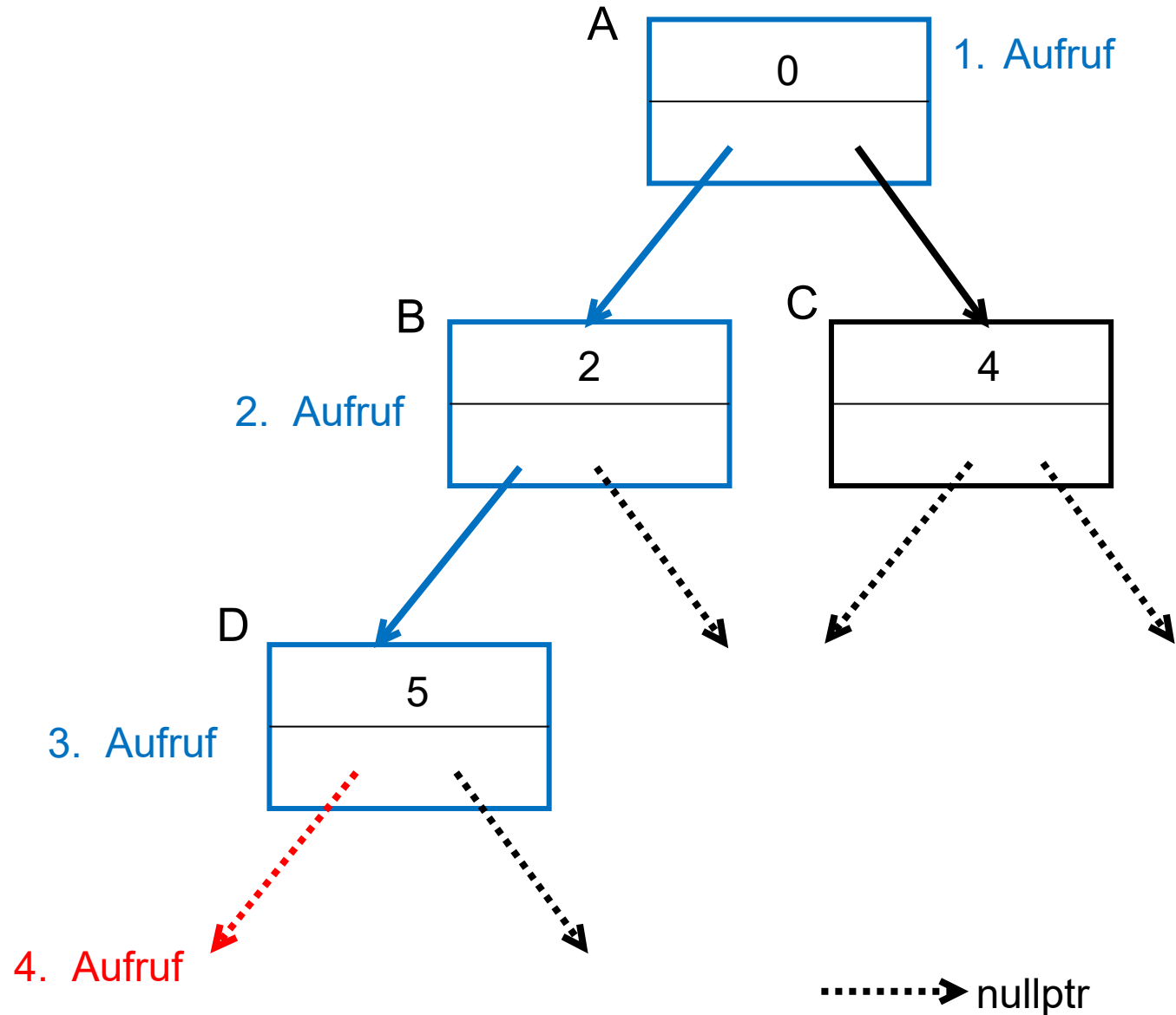
```

else { return; } ←

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

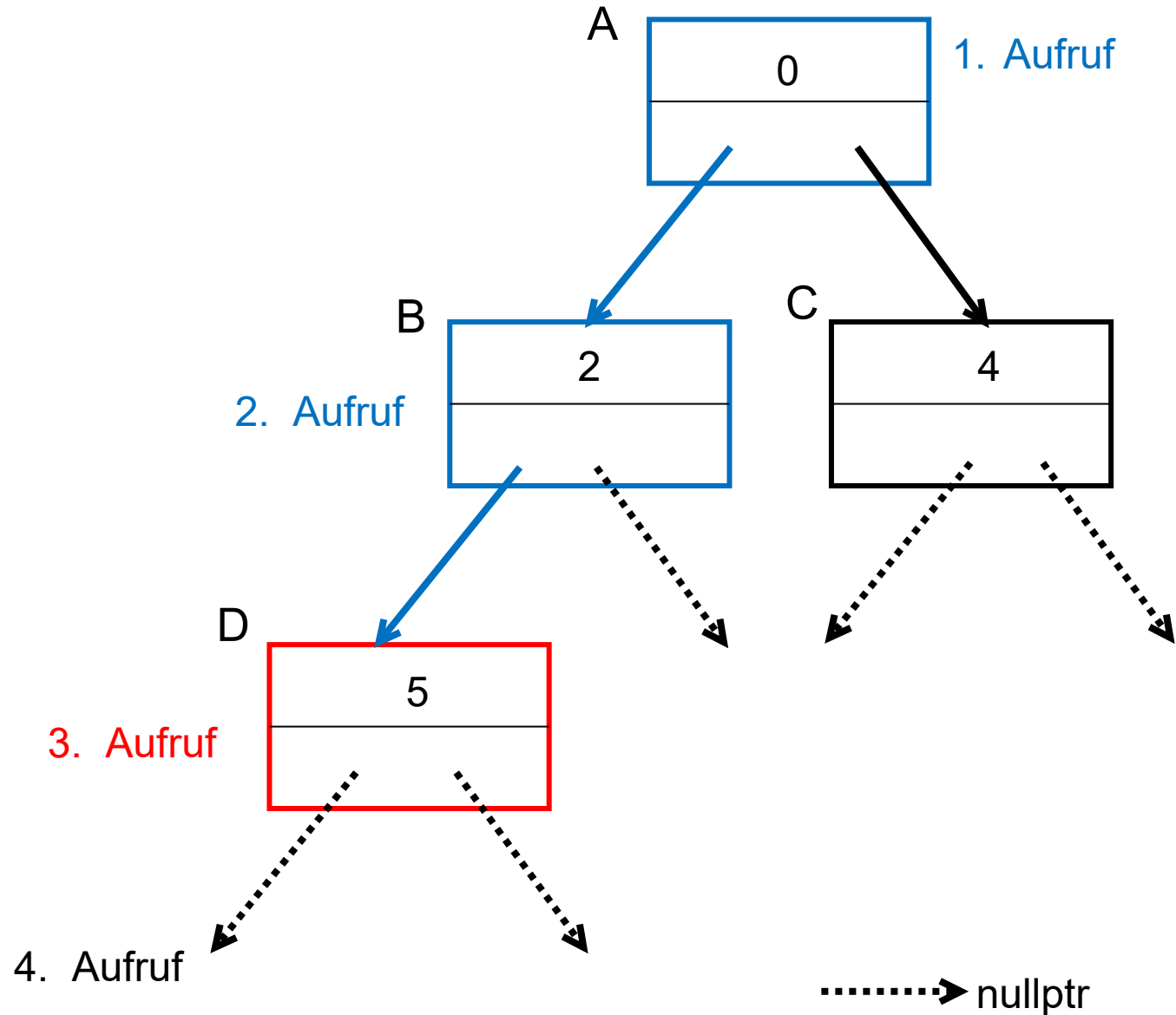
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}

```

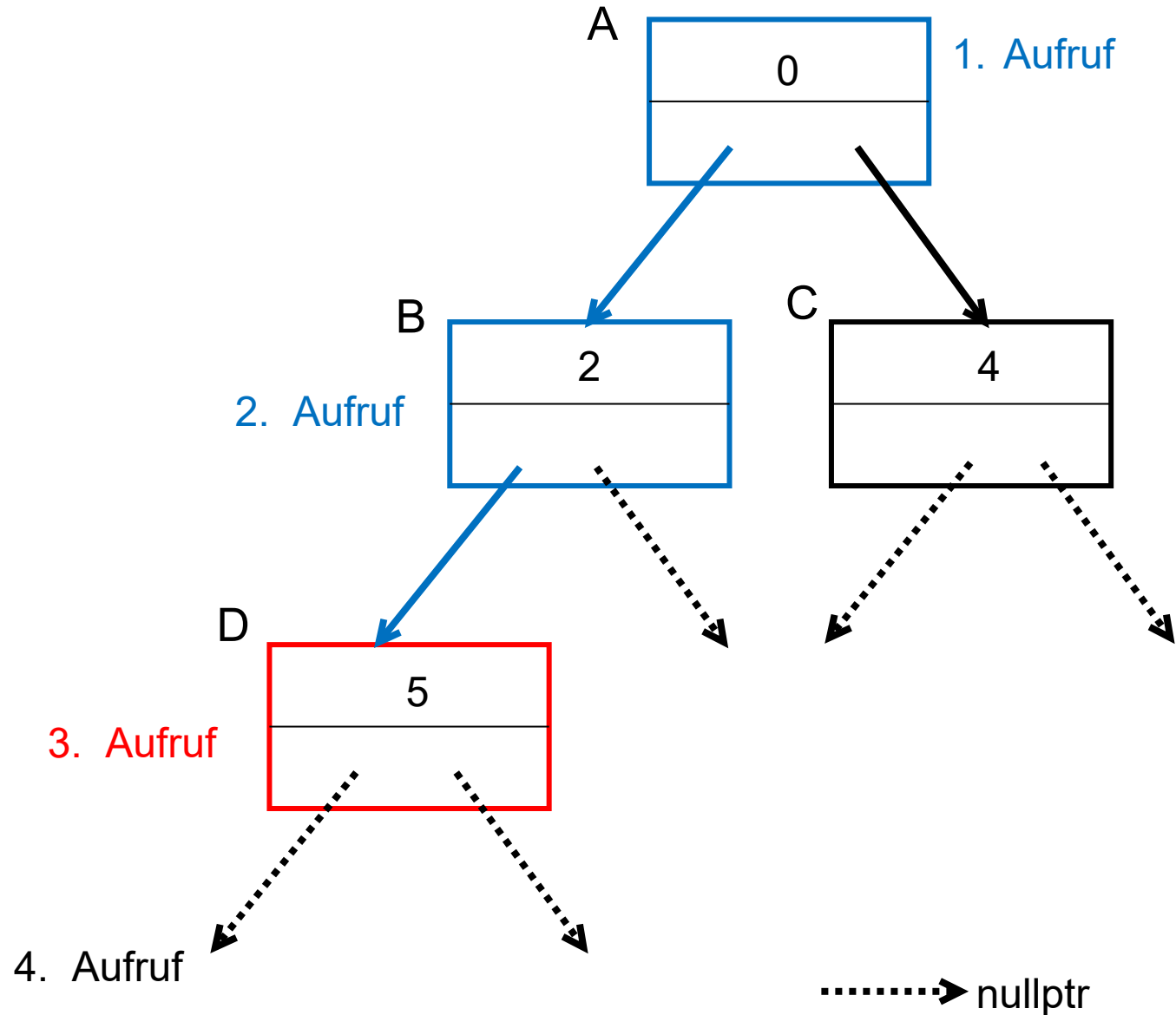
```
else { return; }

```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

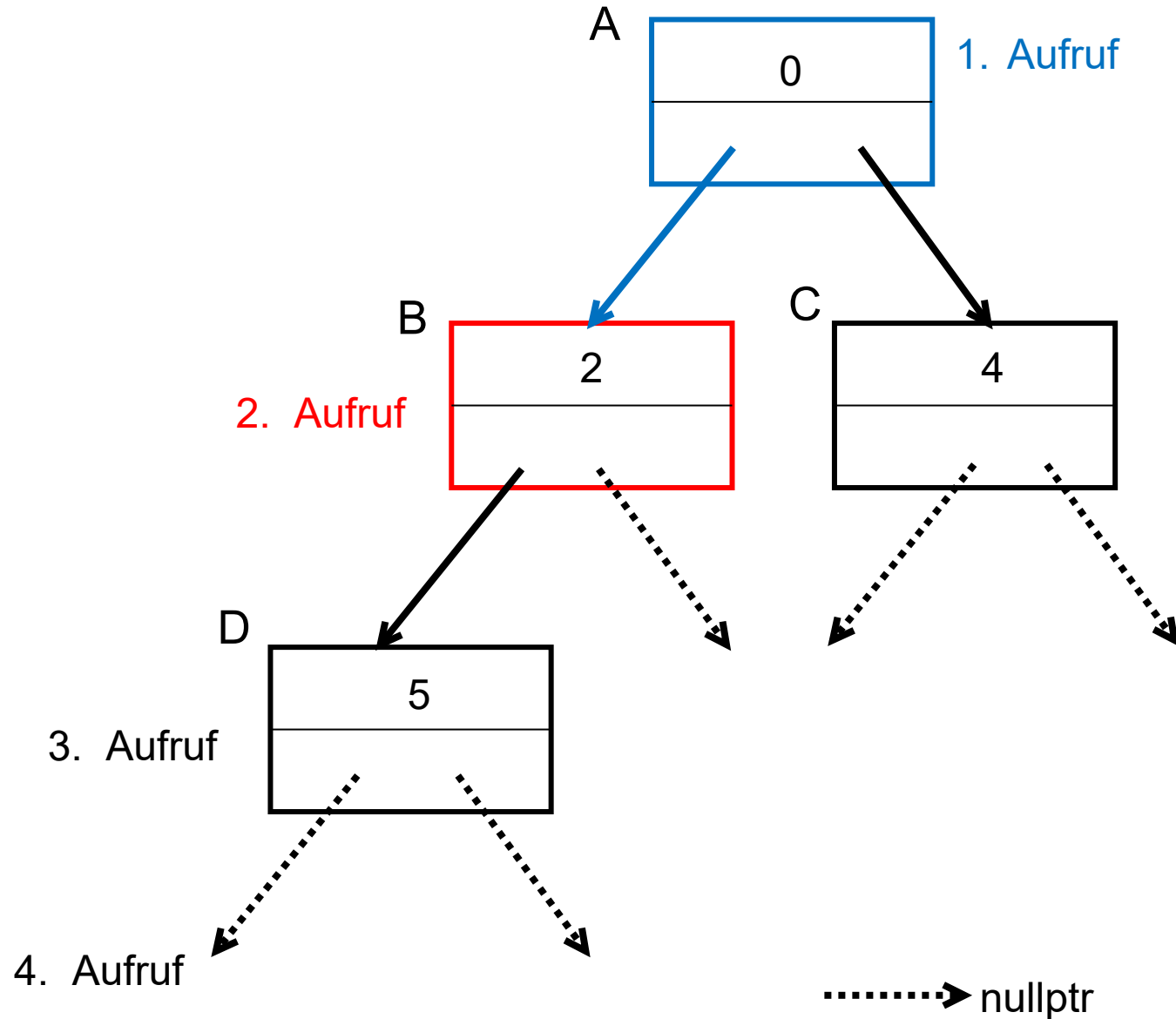
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
```

```
    else { return; }
}
```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}

```

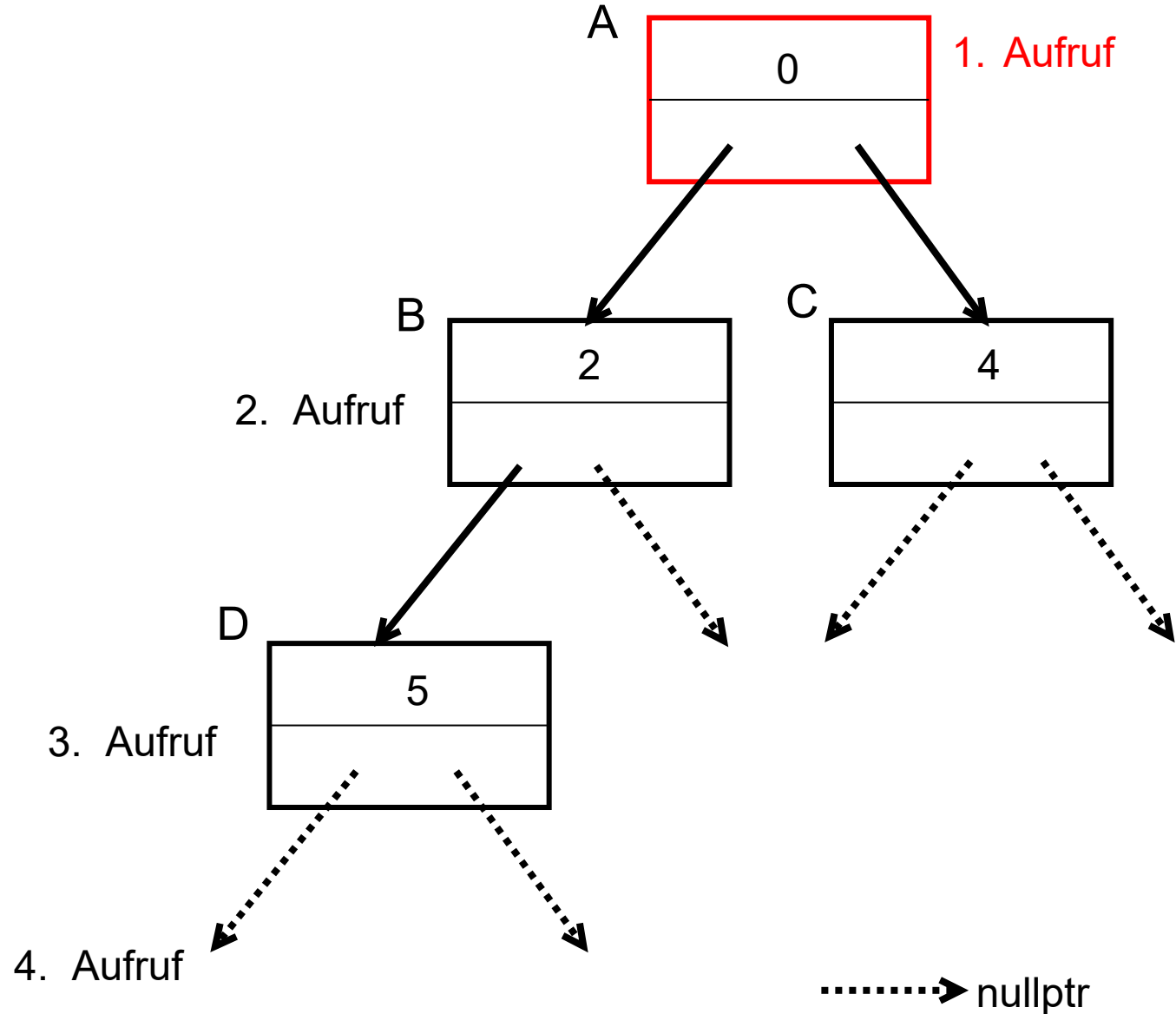
```
else { return; }

```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

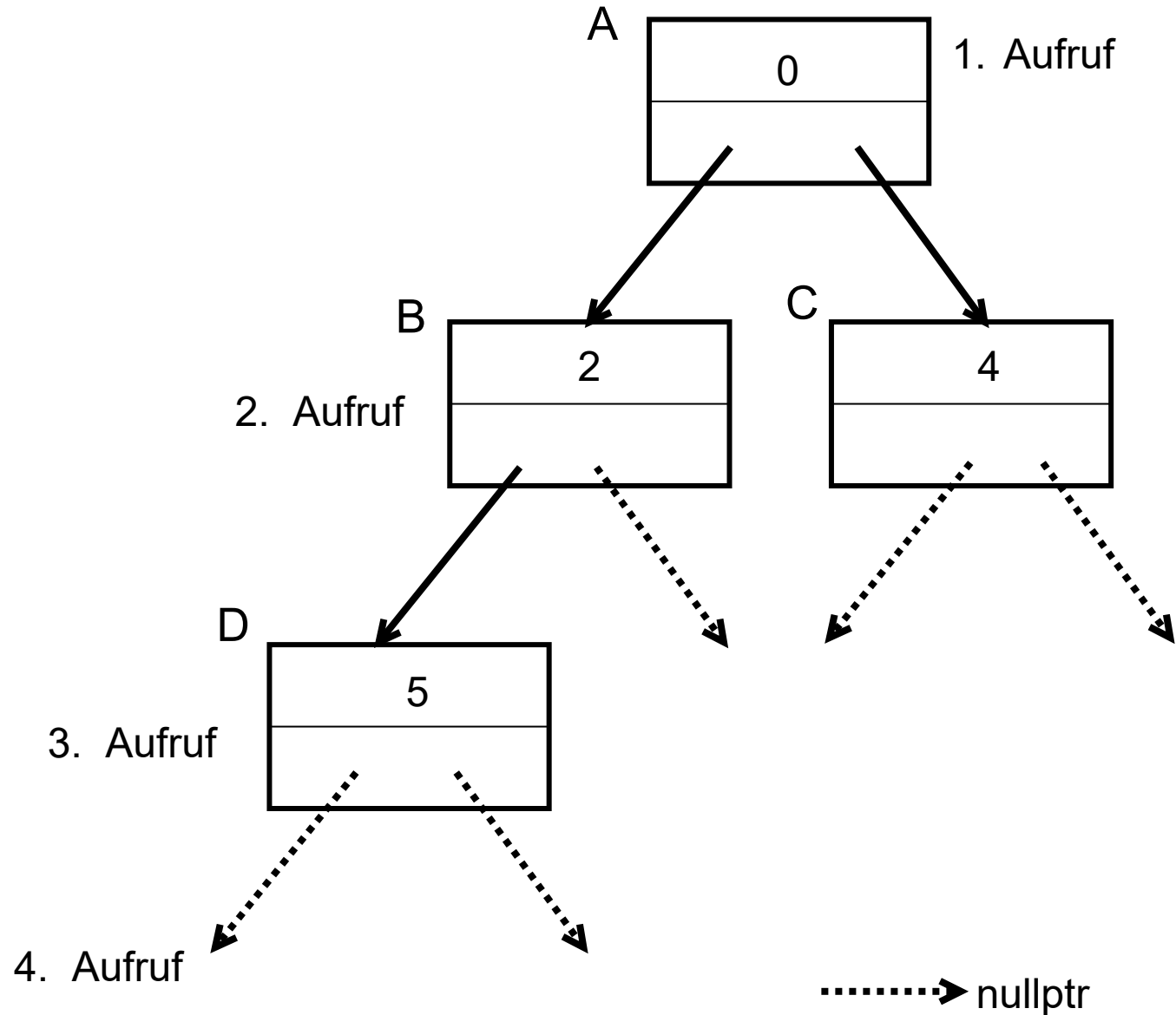
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}

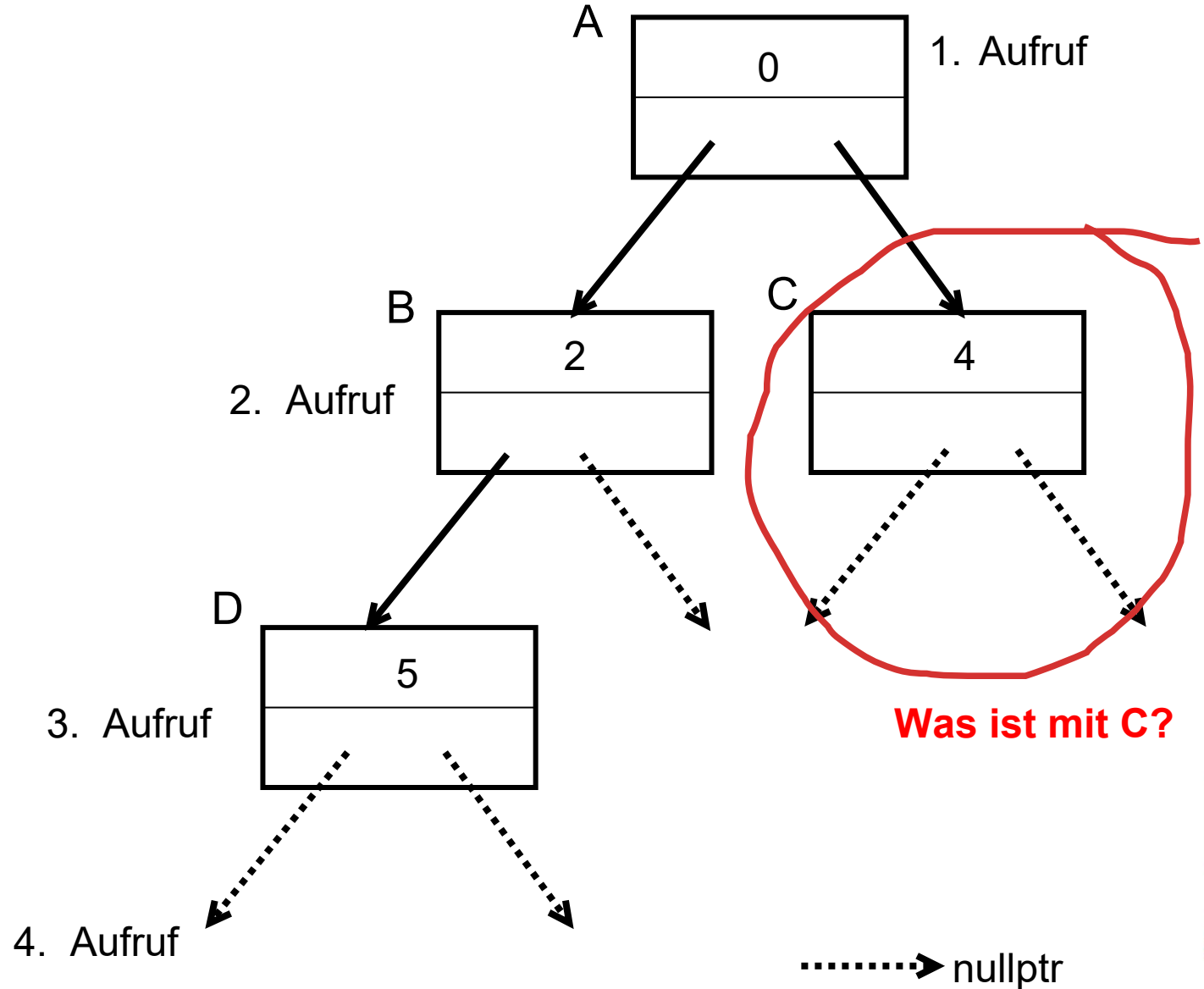
```

```
else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

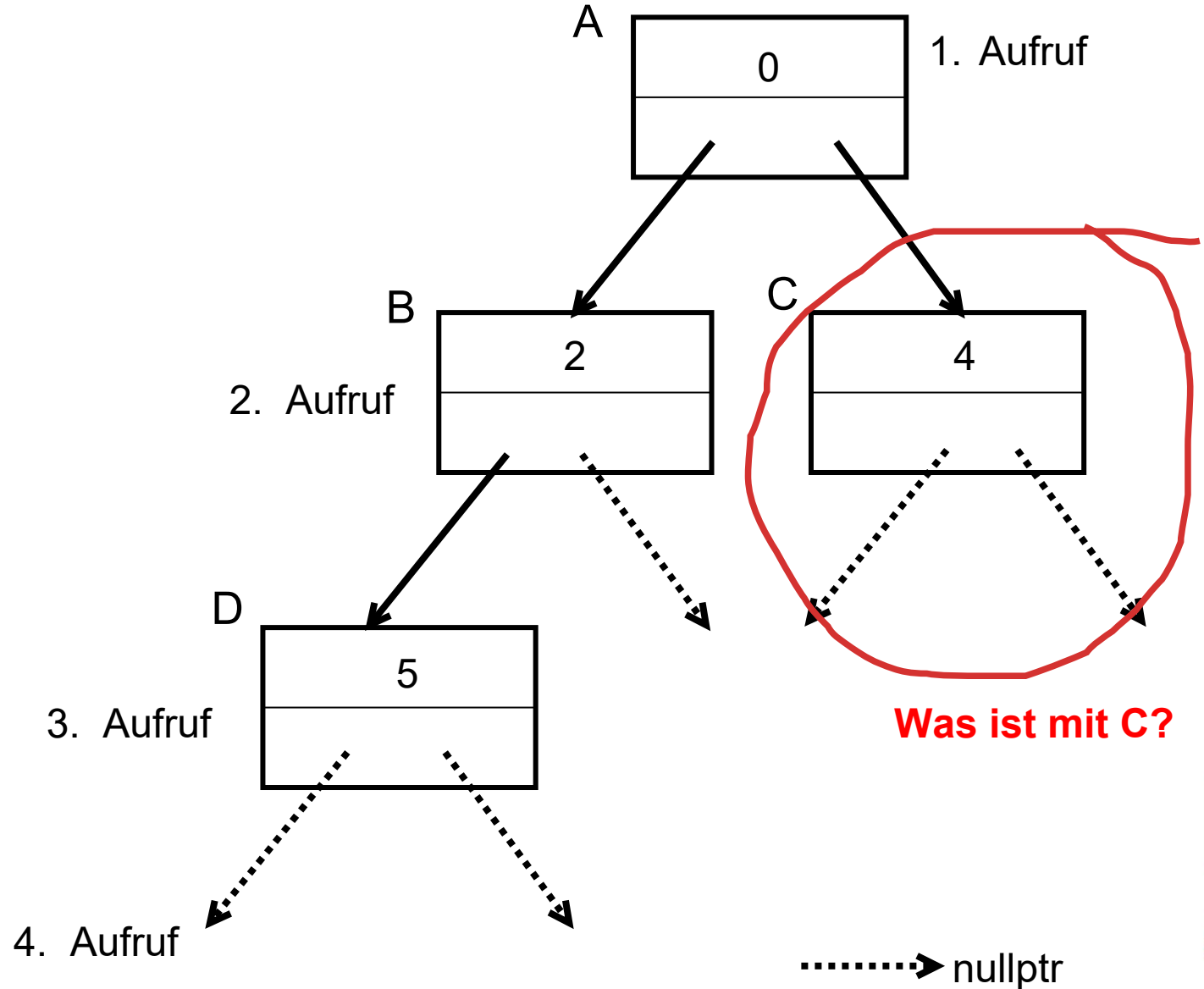
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

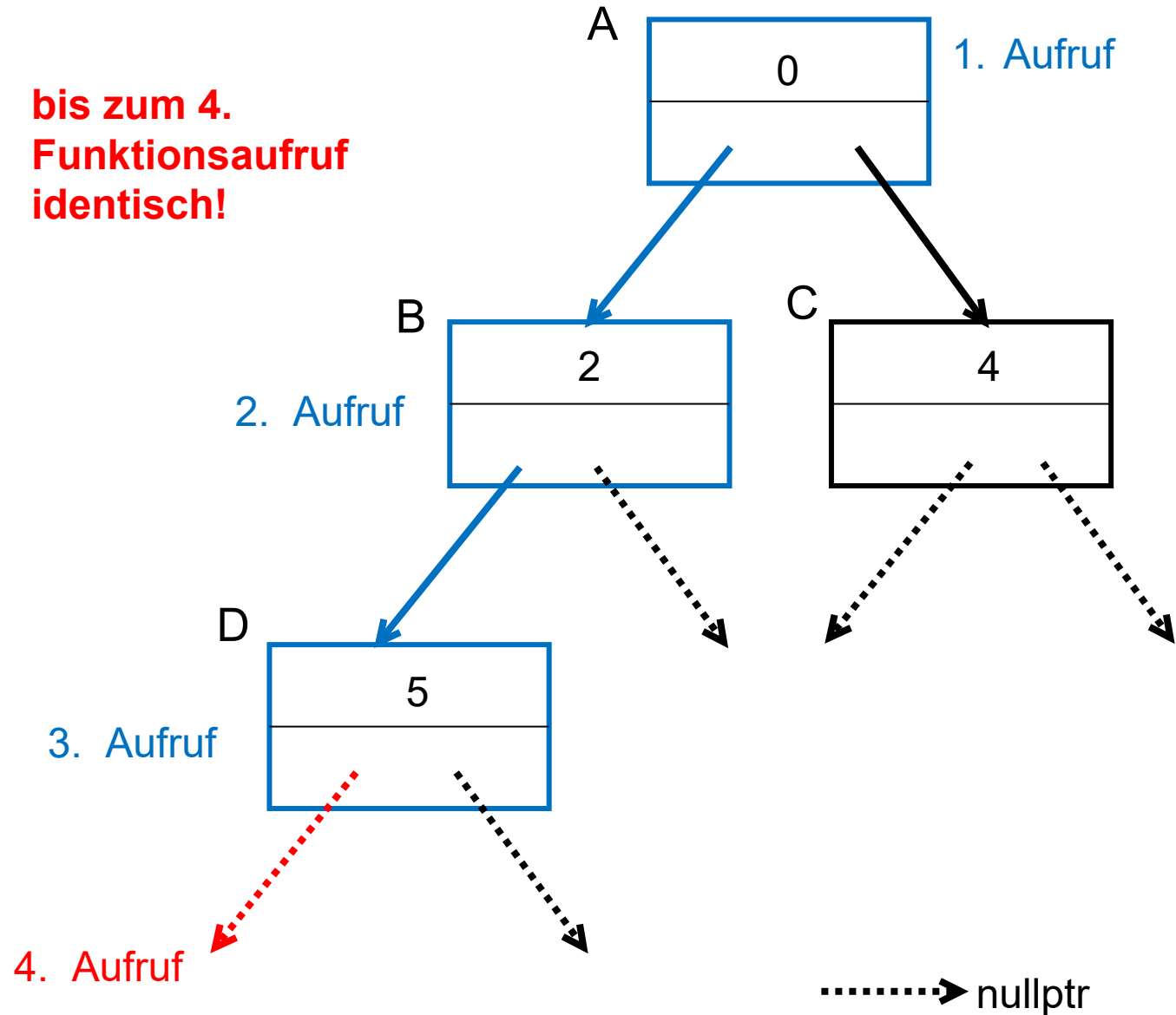
```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) { ←
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```

**bis zum 4.
Funktionsaufruf
identisch!**



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

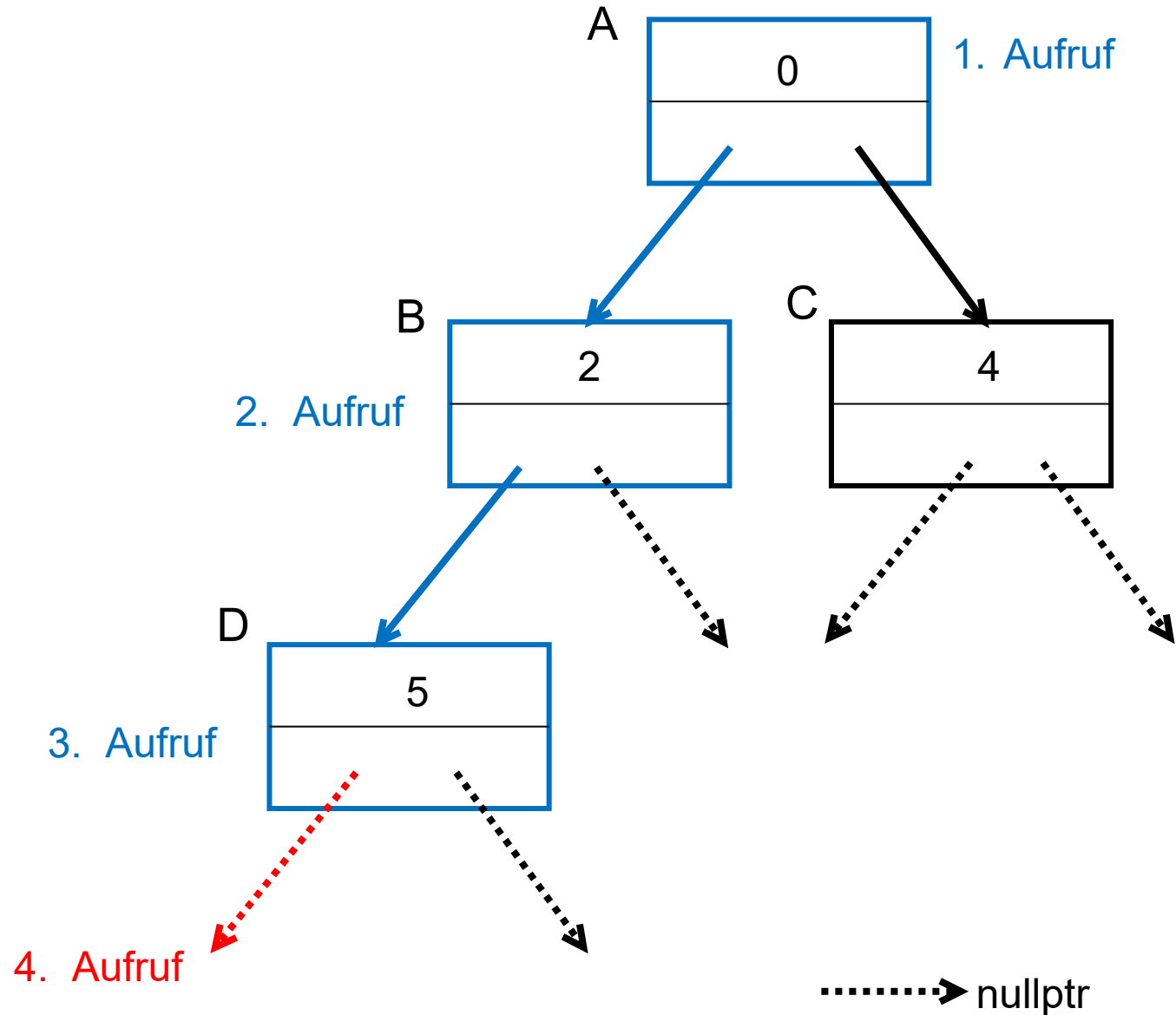
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; } ←
}
```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

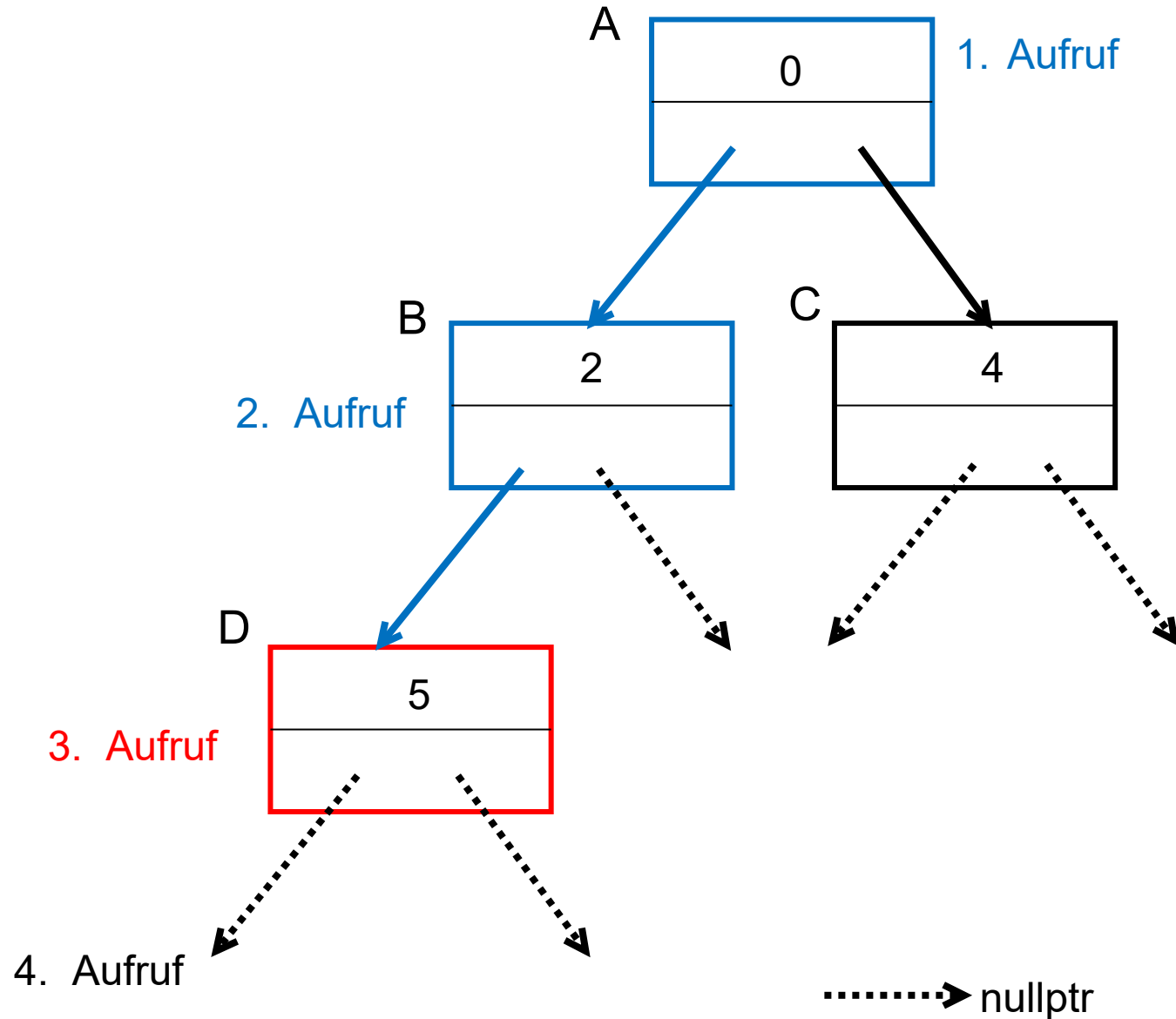
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

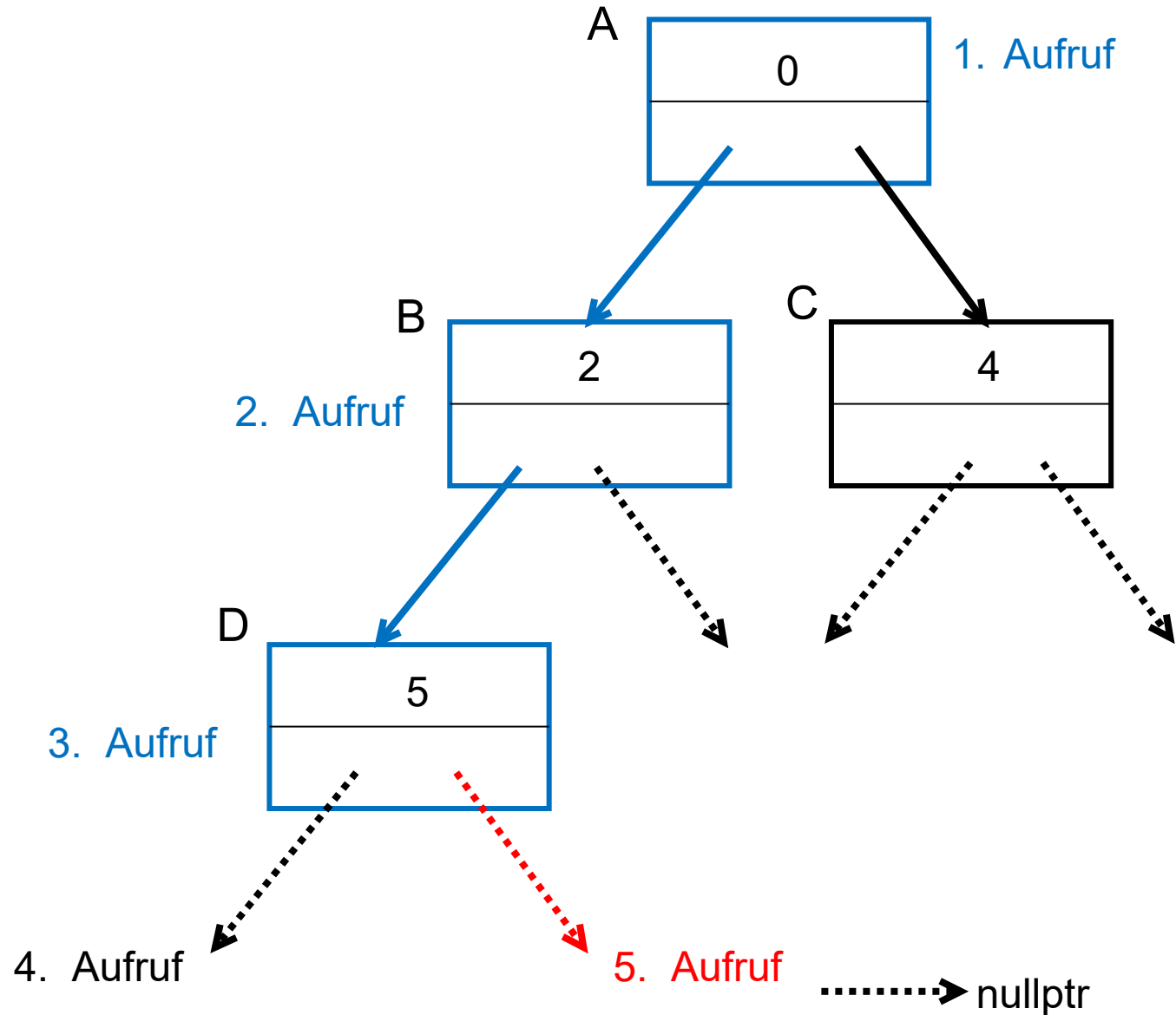
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);
    } else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

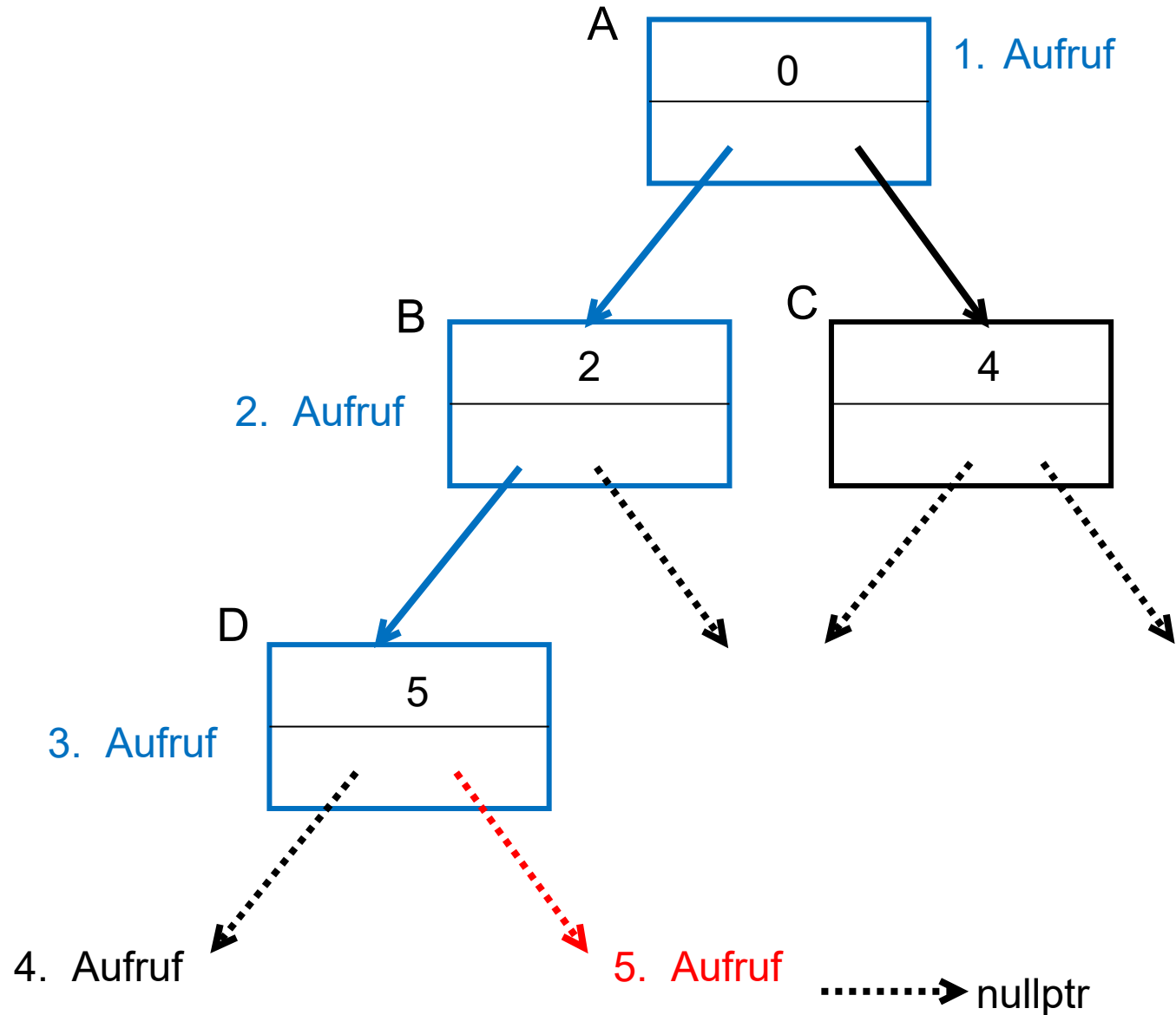
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

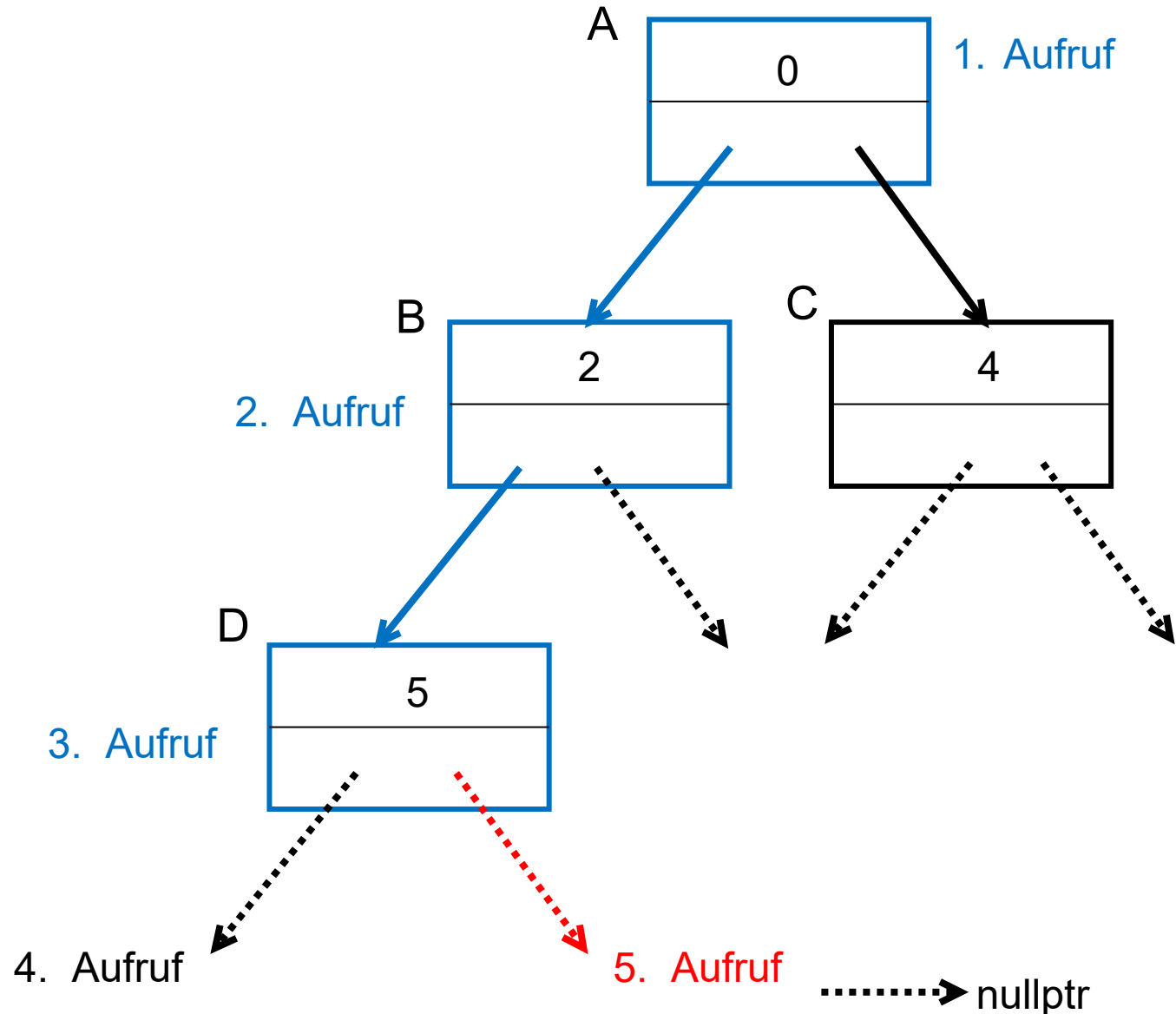
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; } ←
}
```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



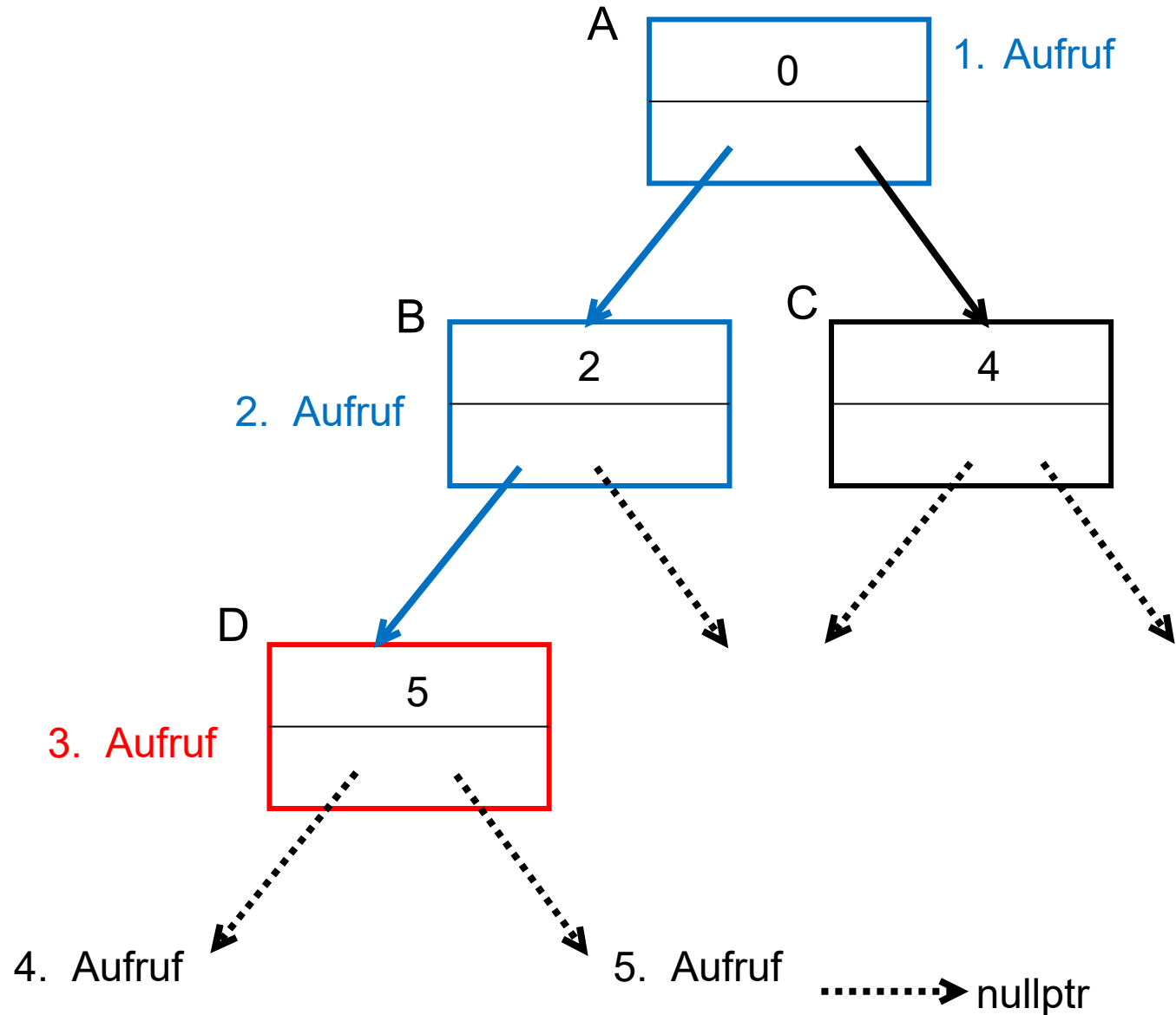
```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

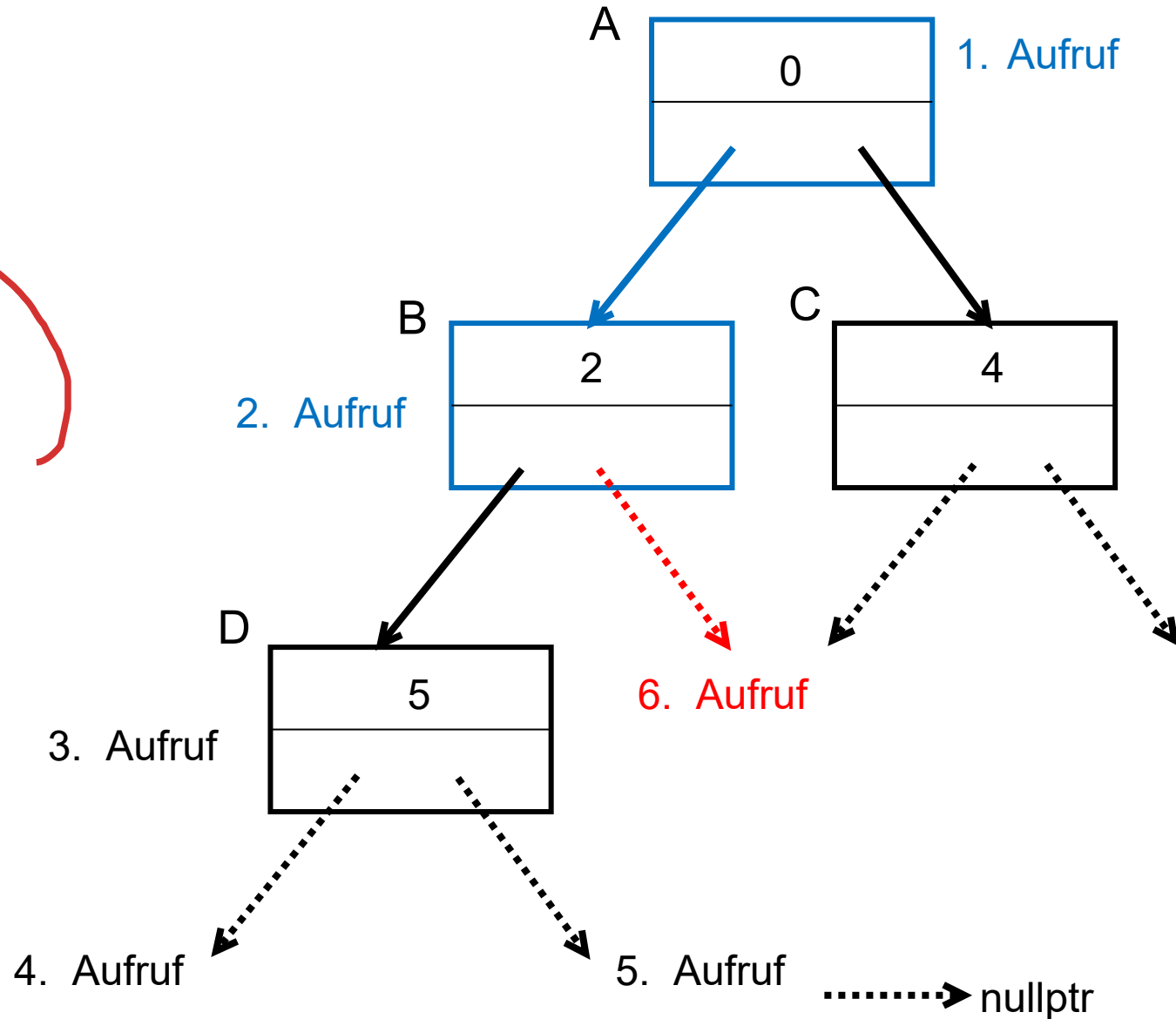


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

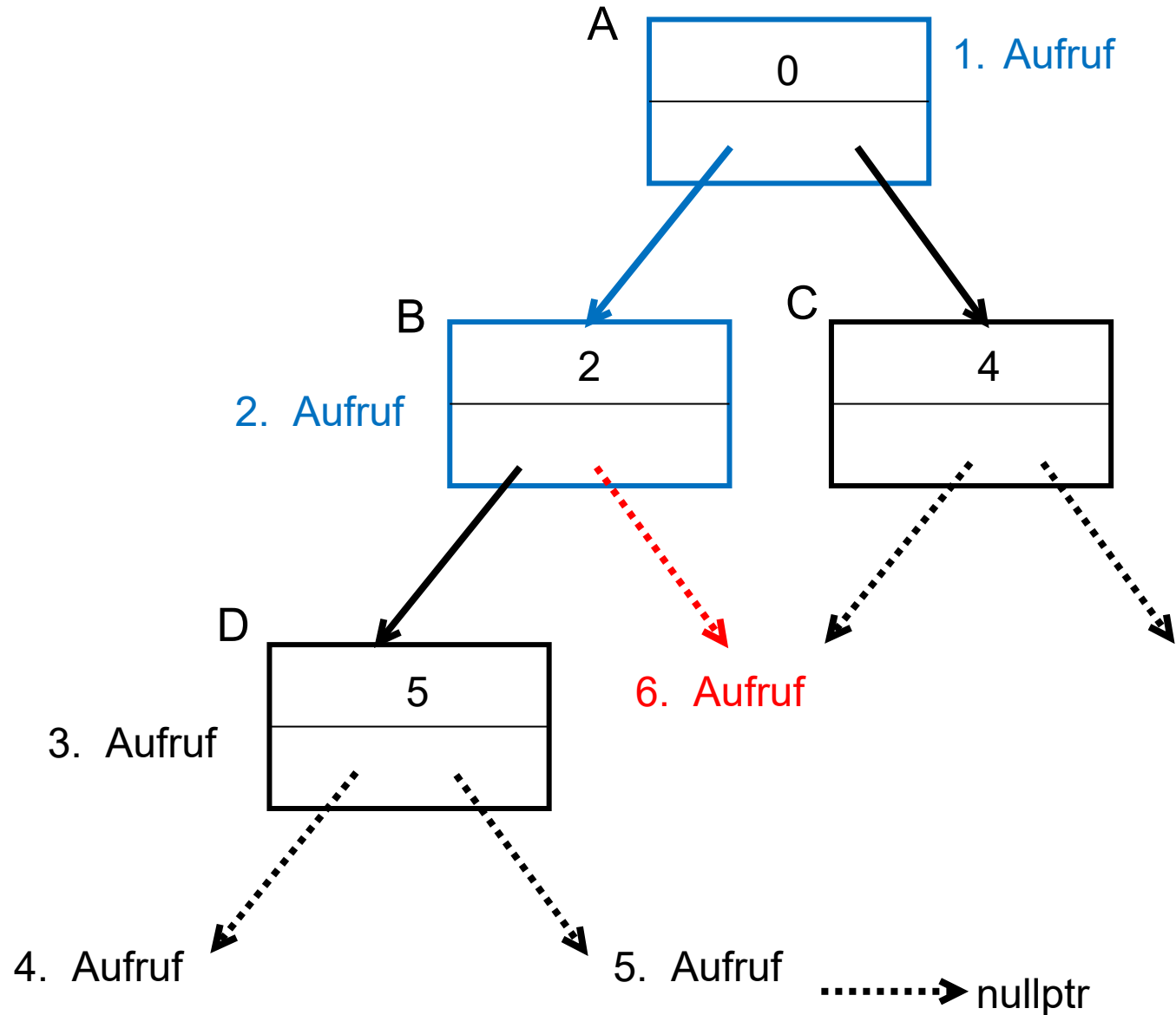
```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);
    }
    else { return; }
}

```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

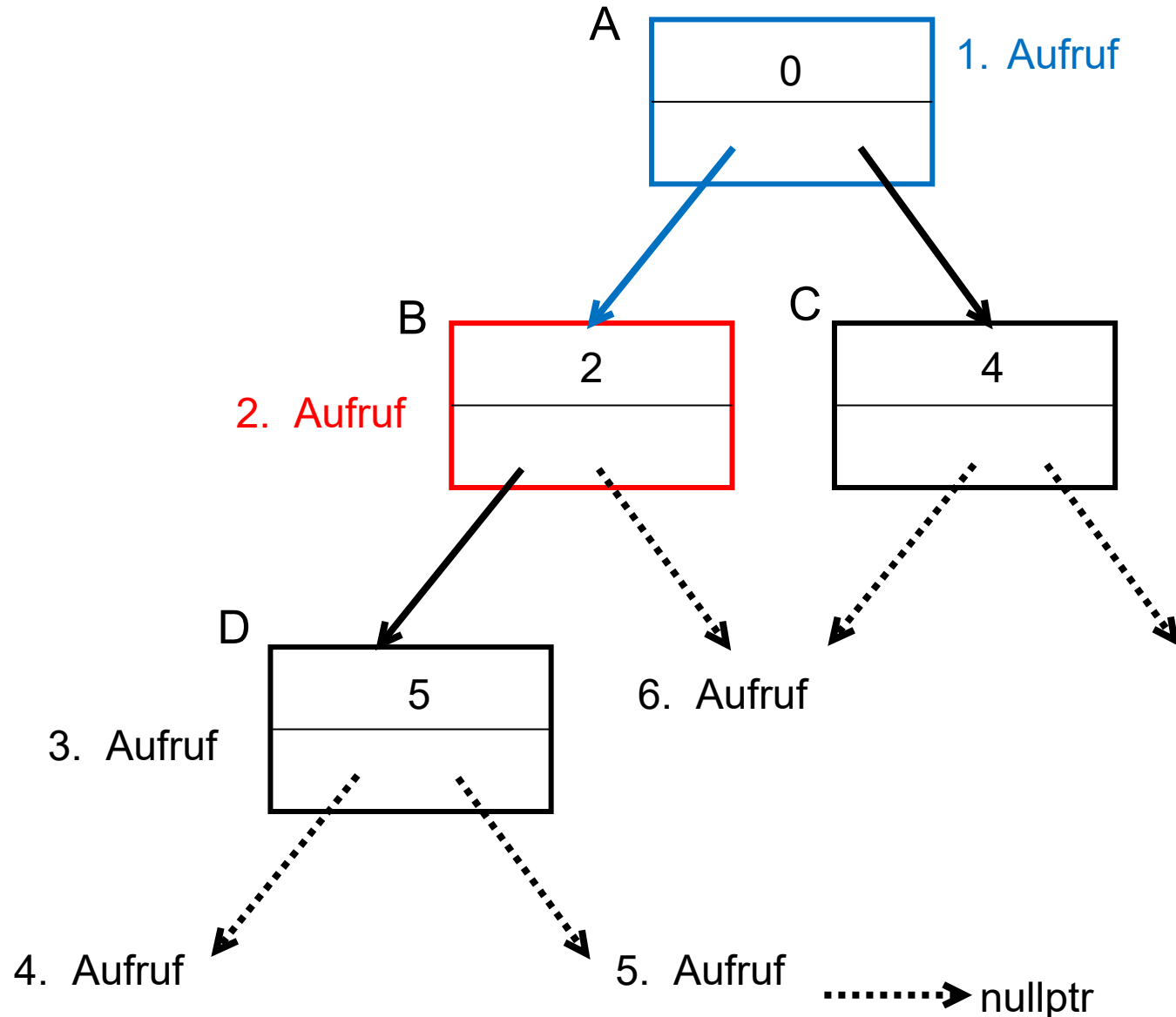
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

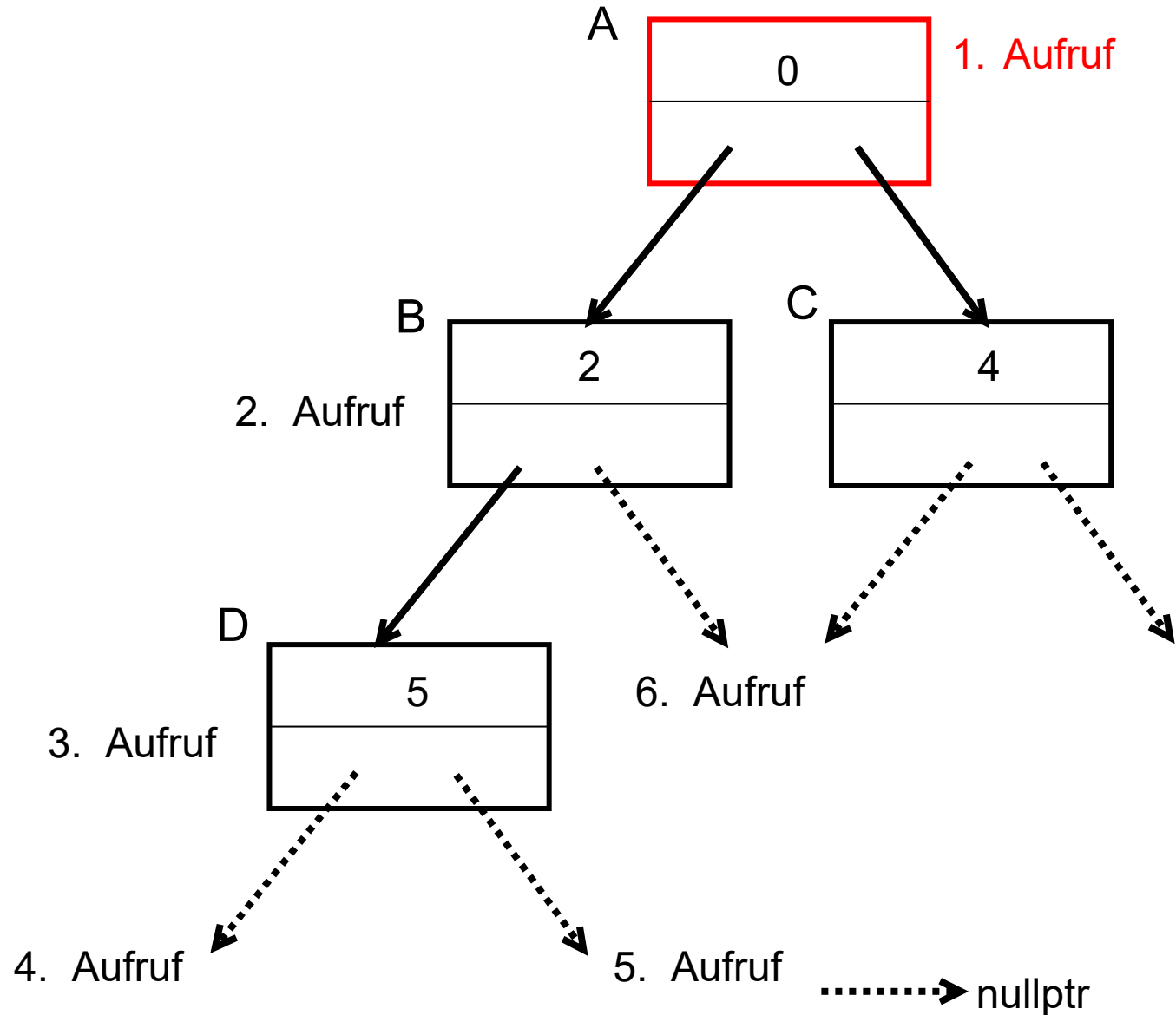
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
```



```
}

int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

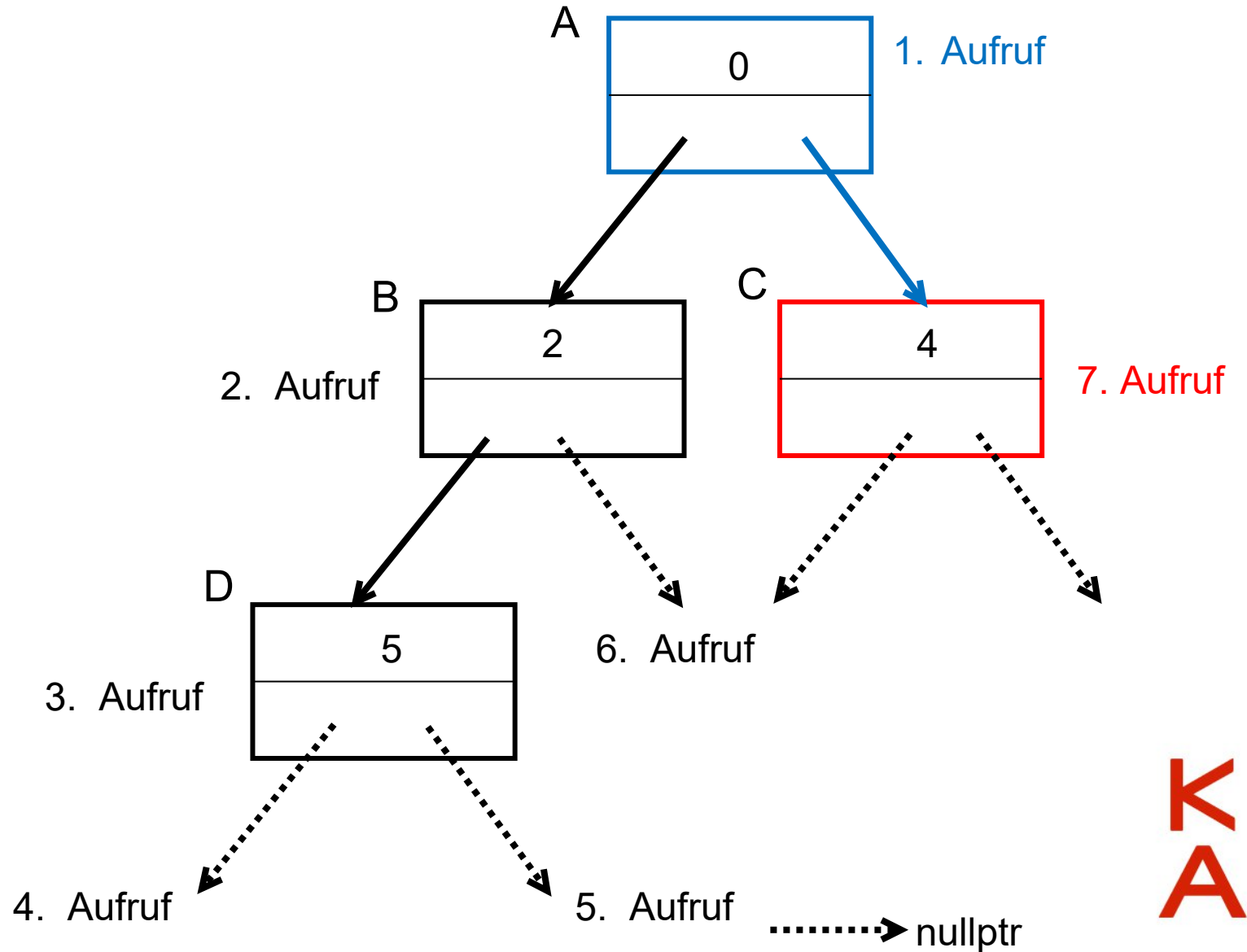


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

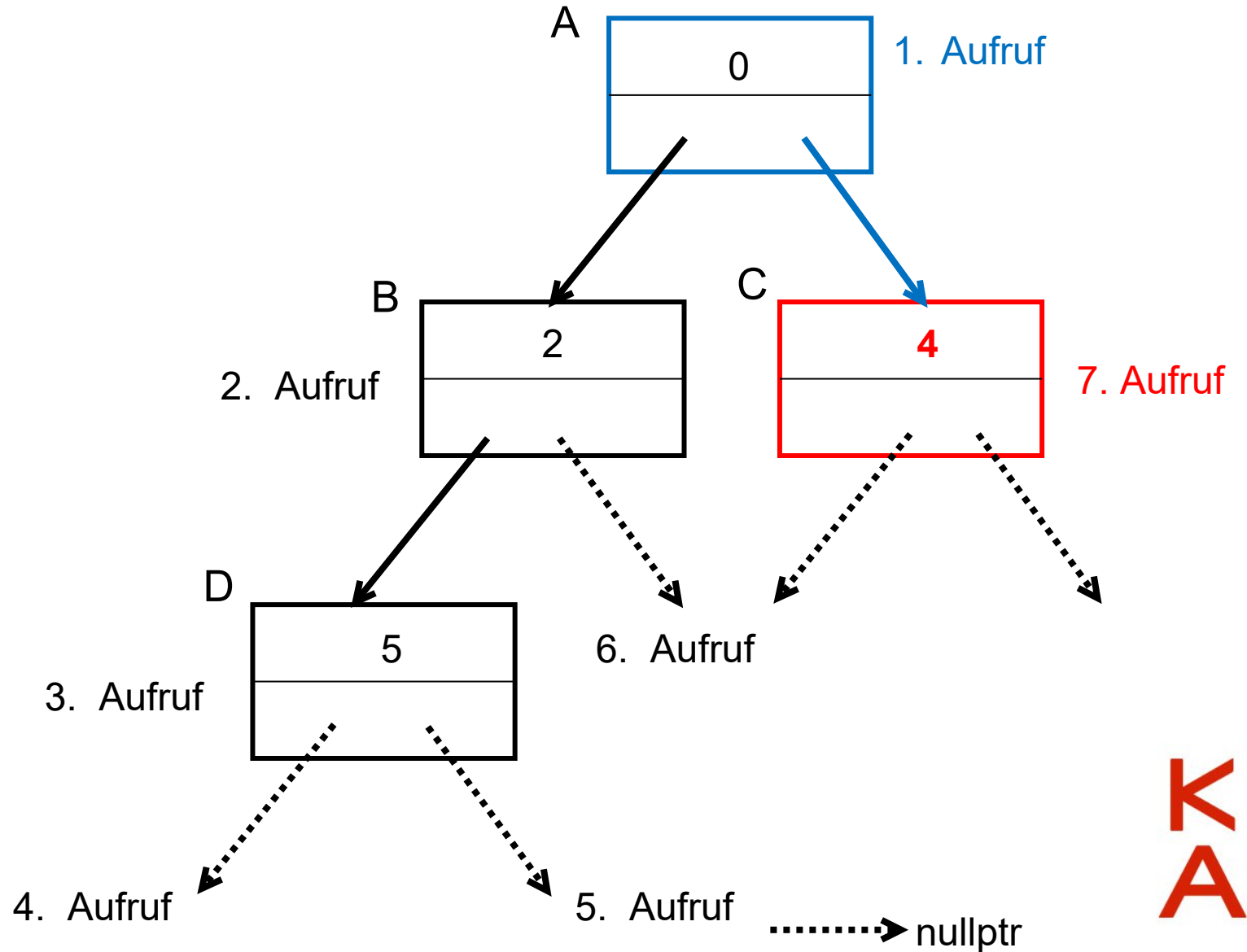


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

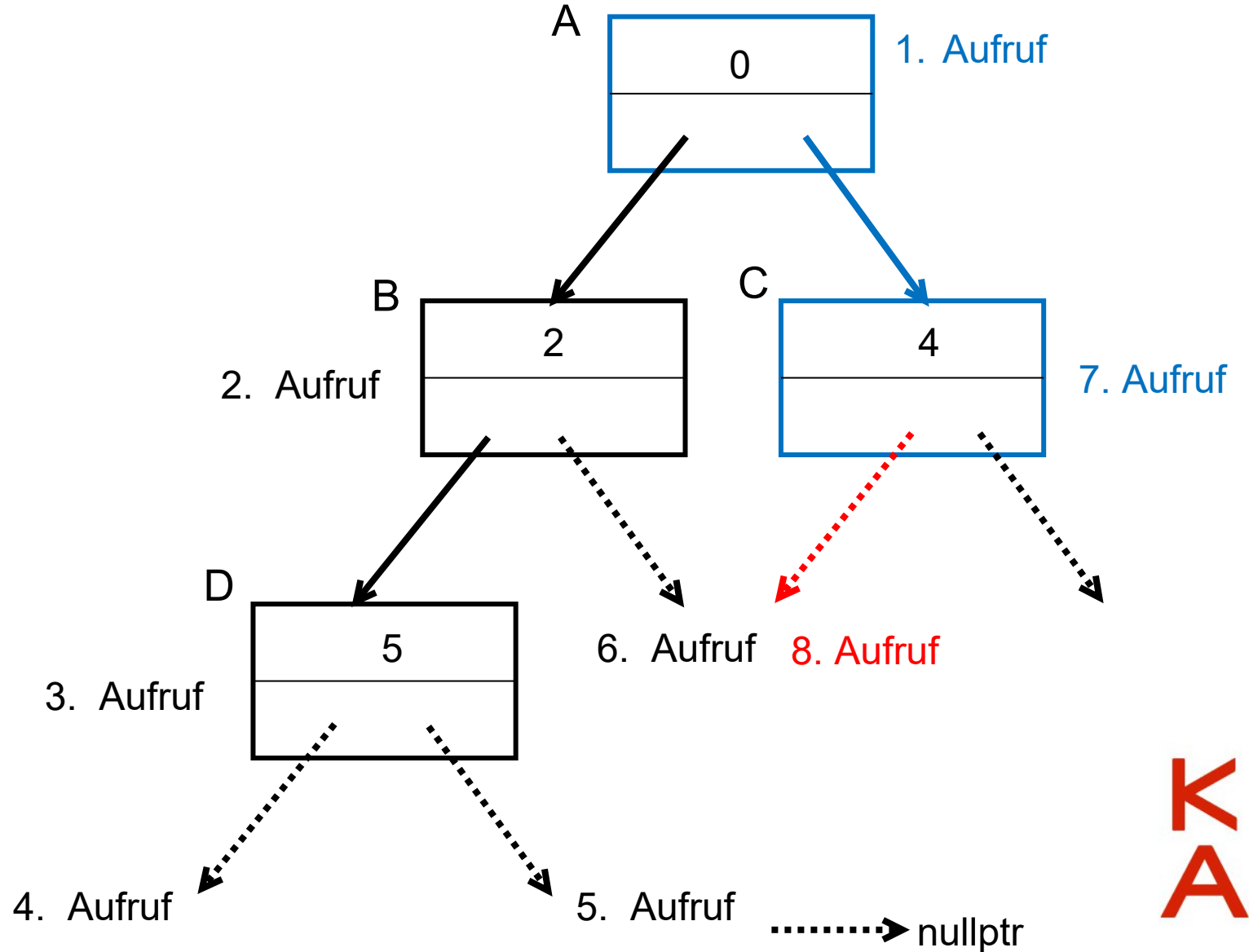
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);
    } else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

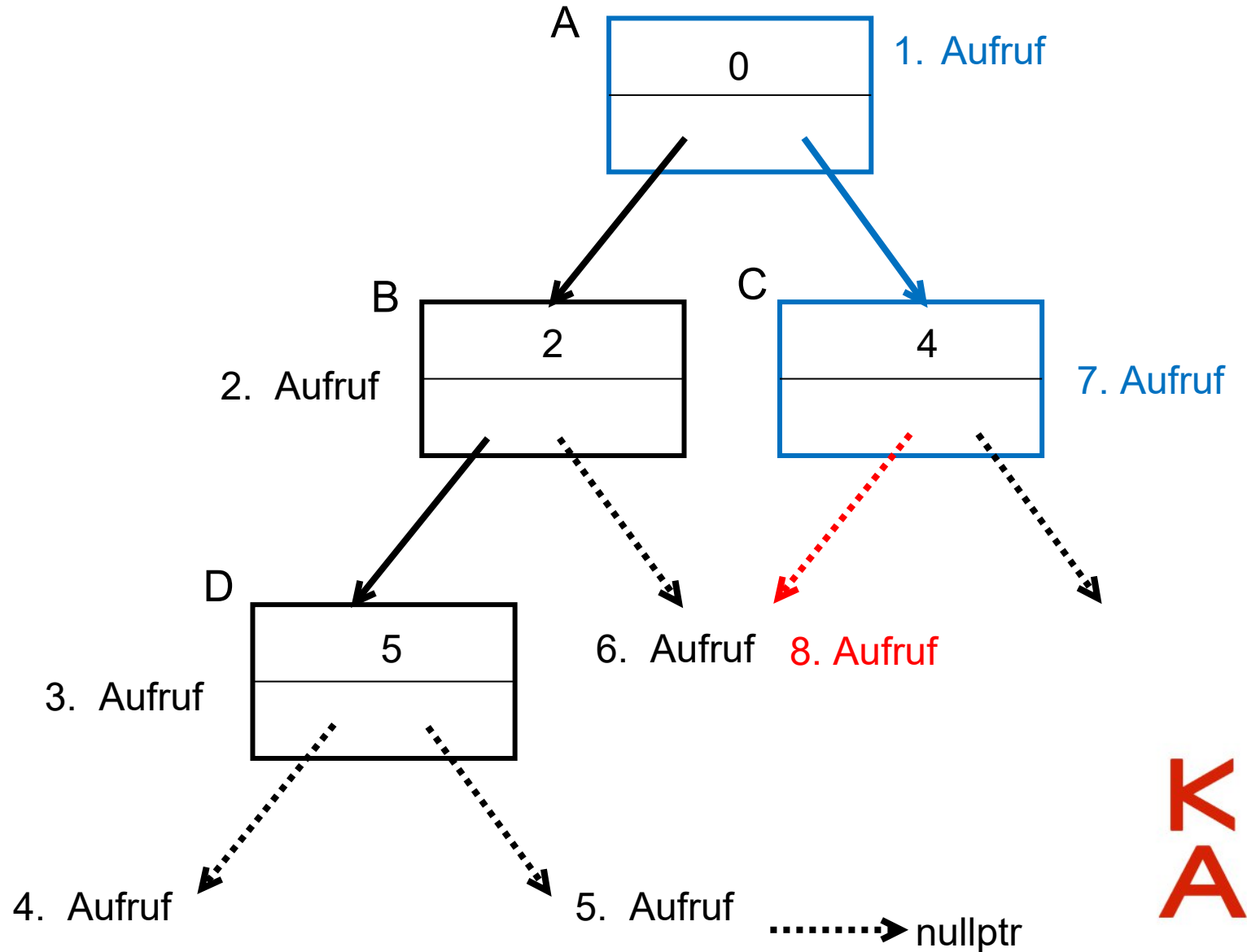
```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) { ←
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

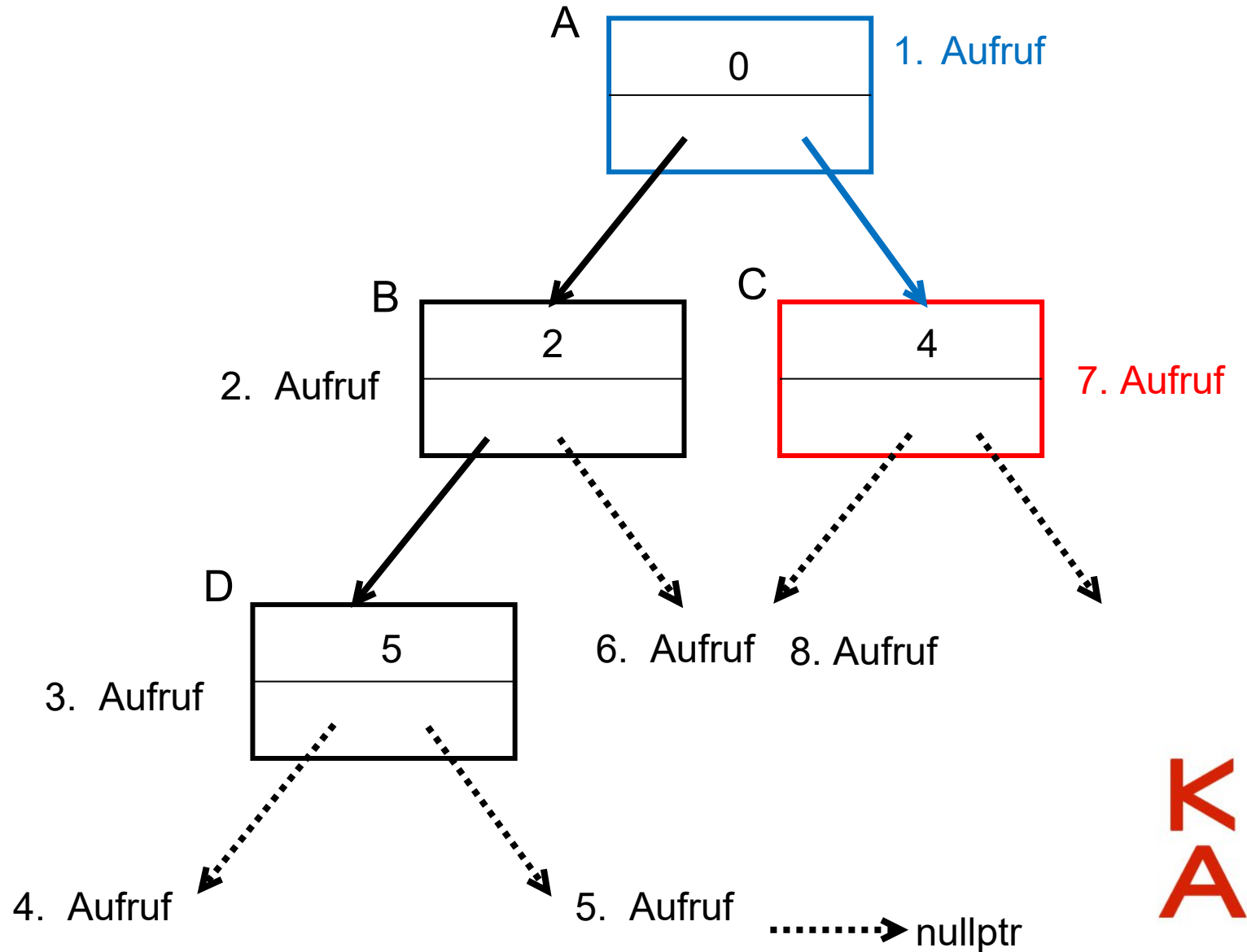
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



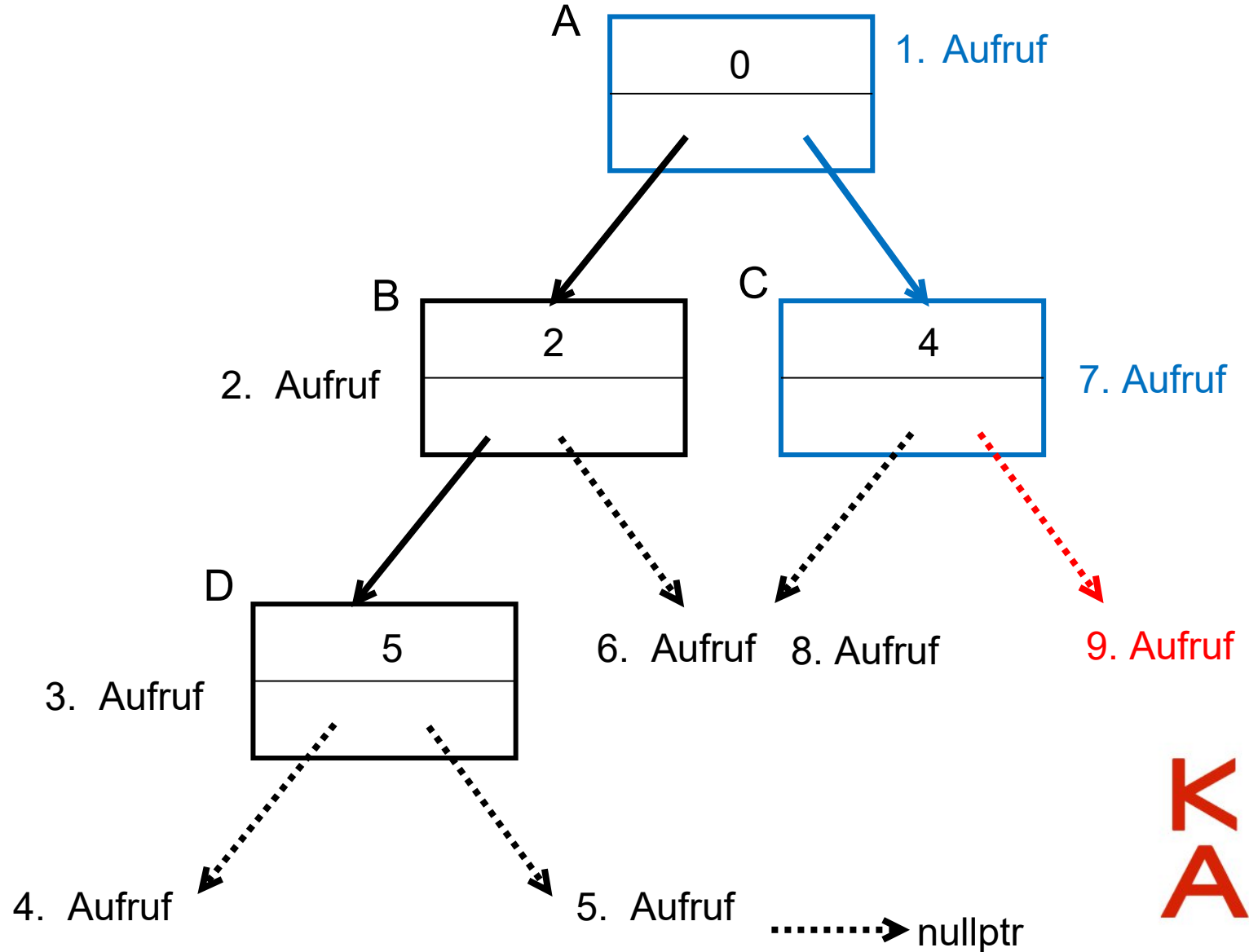
```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

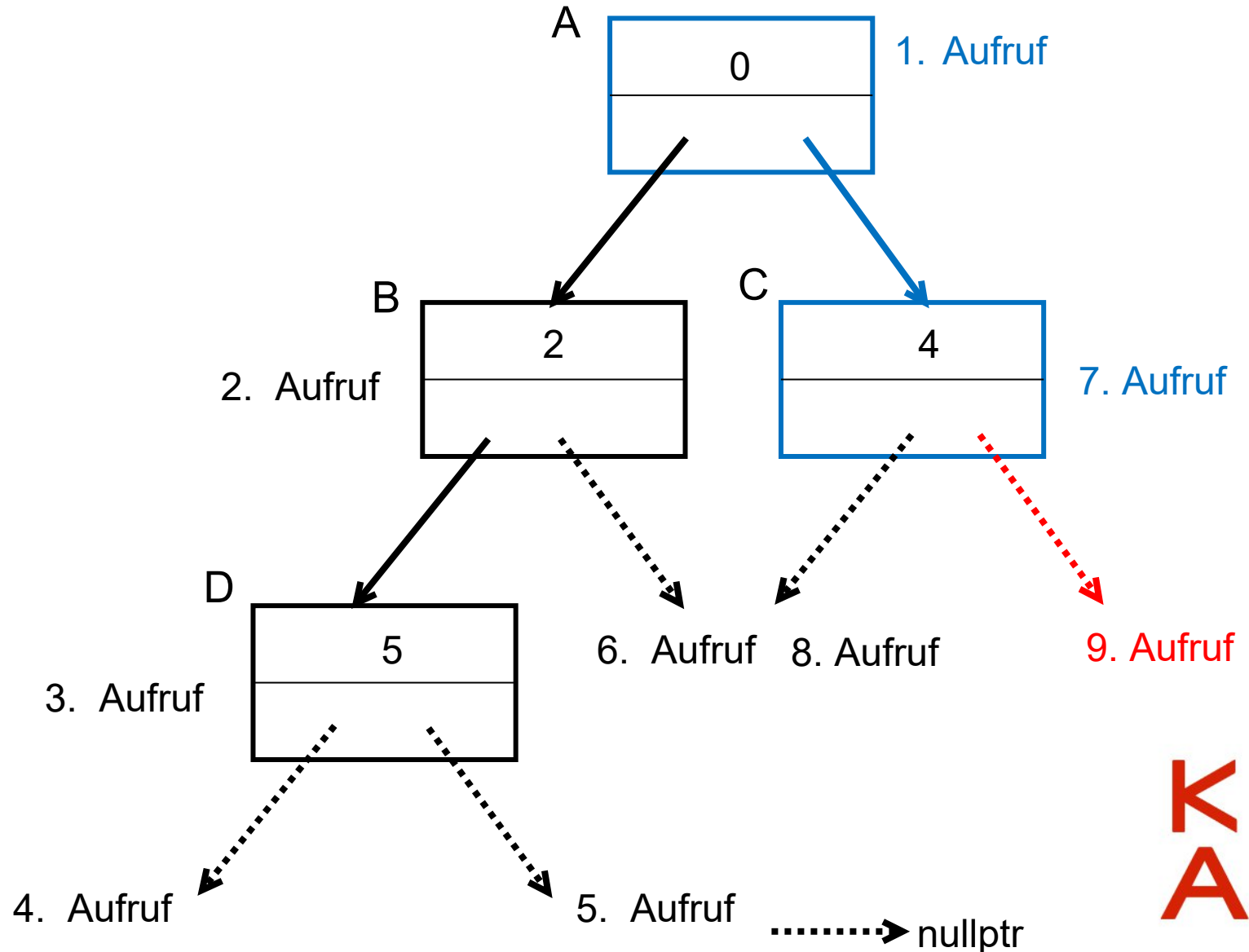
```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) { ←
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

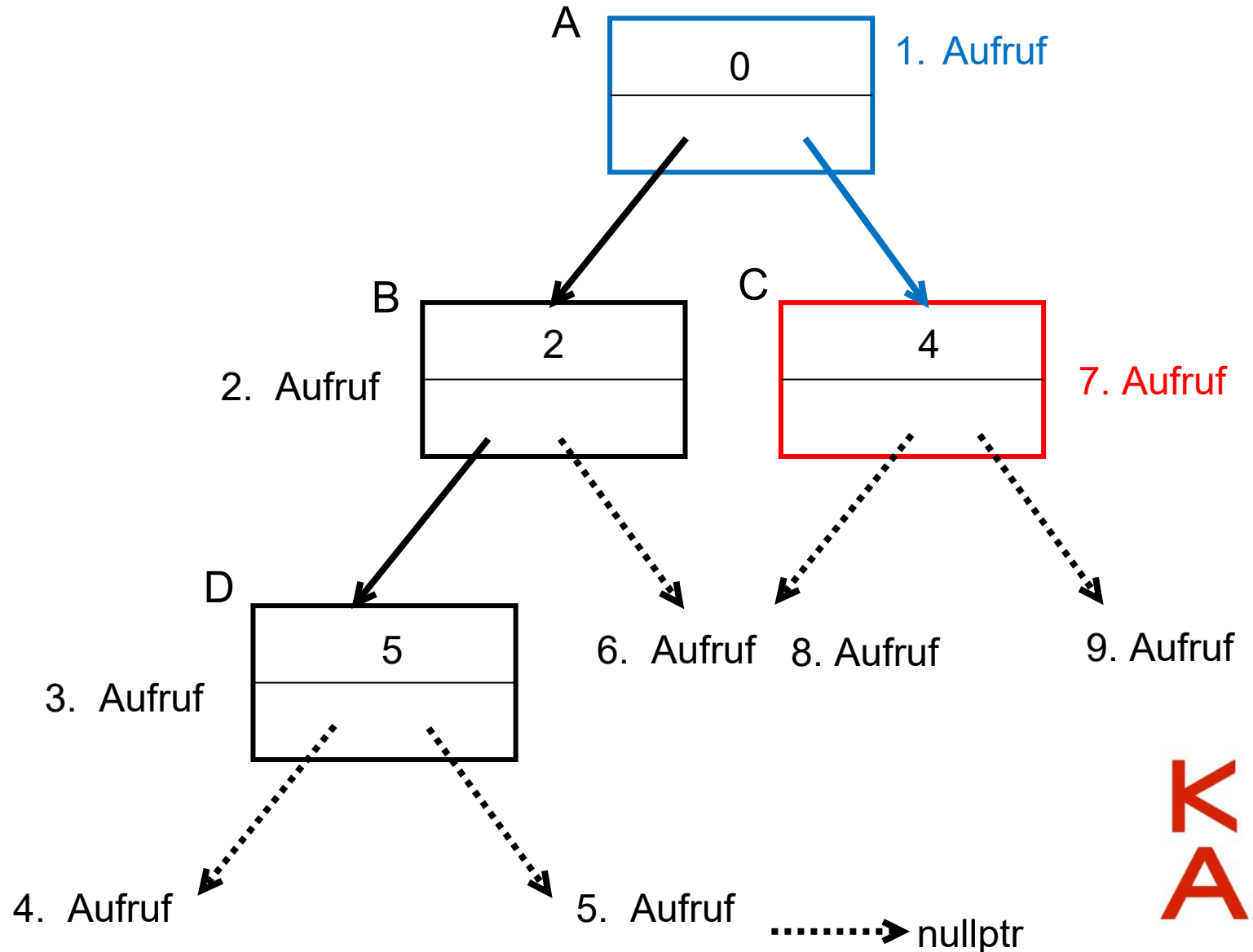
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

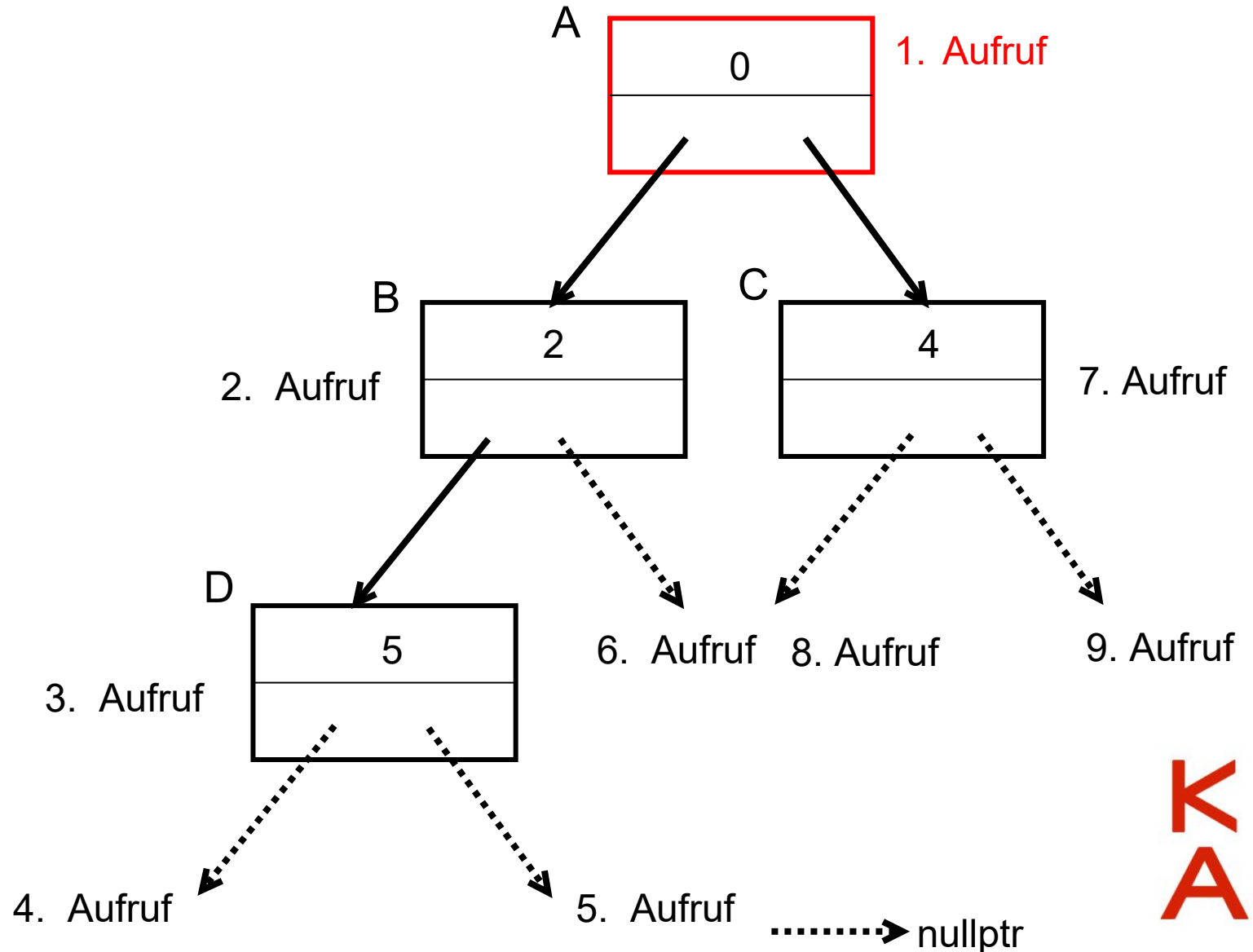
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

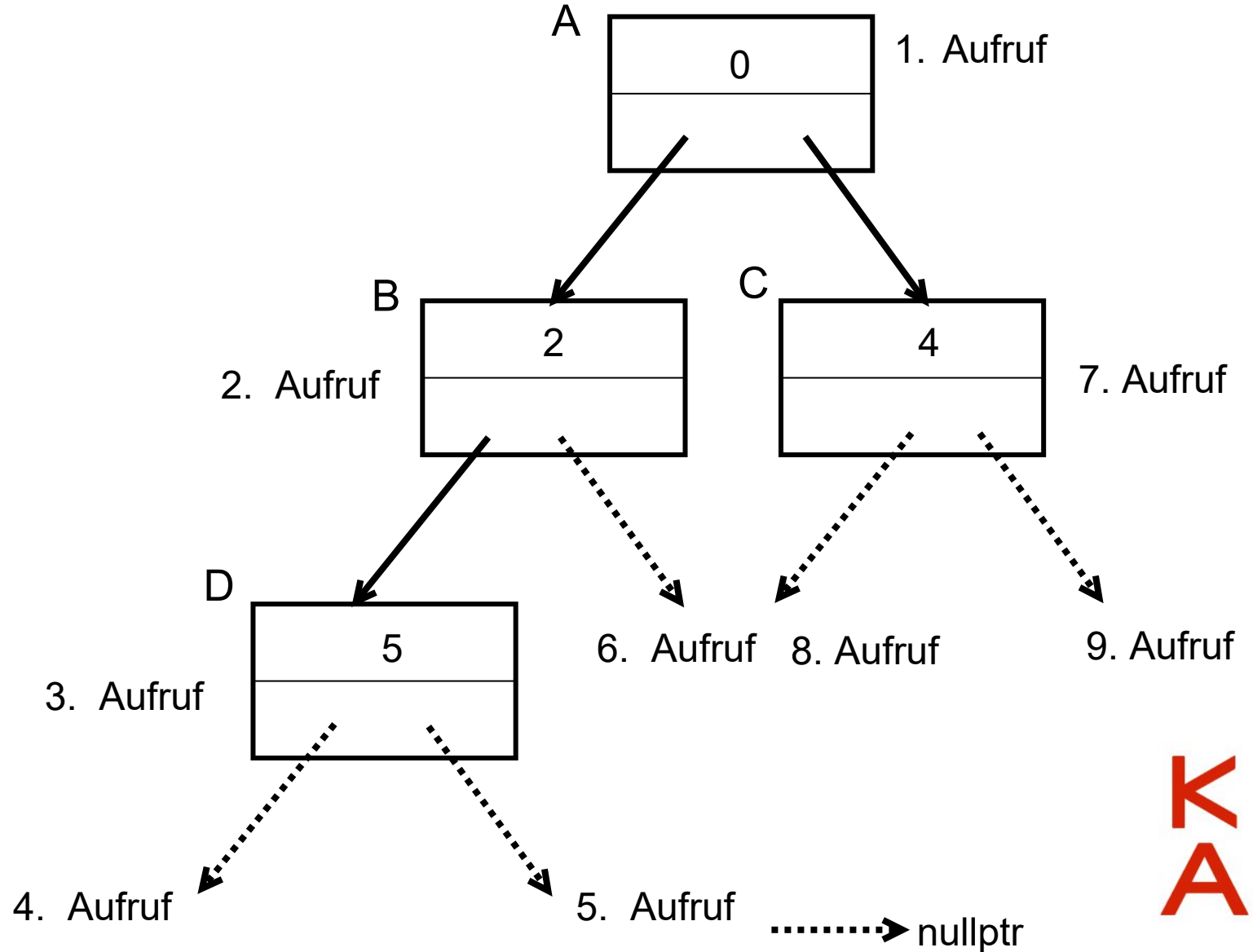
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
} ←
```





Code aus der Vorlesung



```
#include <iostream>
using namespace std;

void MeineFunktion(int x) { // rekursive Funktion
    if (x == 0) { // Abbruchbedingung
        cout << "go\n";
        return;
    }
    cout << x << endl;
    MeineFunktion(x-1); // rekursive Aufruf
}

int main() {
    // rekursive Loesung
    cout << "rekursive Loesung\n";
    MeineFunktion(10);
    // iterative Loesung
    cout << "\n\niterative Loesung\n";
    for (int jj = 10; jj > -1; jj--) {
        if (jj != 0) {
            cout << jj << endl;
        }
        else {
            cout << "Go\n";
        }
    }
    return 0;
}
```



Code aus der Vorlesung



```
int Fak(int n) {  
    if (n == 1) { // abbruchbedingung  
        return 1;  
    }  
    return n * Fak(n - 1); // rekursiver Aufruf  
}
```

```
int main() {  
    // rekursive Loesen  
    cout << Fak(5) << endl;  
  
    // iterative Lösung  
    int ergebnis = 1;  
    for (int z = 1; z < 6; z++) {  
        ergebnis *= z;  
    }  
    cout << ergebnis << endl;  
  
    return 0;  
}
```



Code aus der Vorlesung



```
/*  
Parameter:  
a ist der Exponent  
*/  
int Pot(int p, int a) {  
    if (a == 0) { // abbruchbedingung  
        return 1;  
    }  
    return p * Pot(p, a - 1); // rekursiver Aufruf  
}  
  
int main() {  
  
    cout << Pot(2, 3) << endl;  
  
    int E = 1;  
    for (int zz = 0; zz < 3; zz++) {  
        E *= 2;  
    }  
    cout << E << endl;  
    return 0;  
}
```



Code aus der Vorlesung

```
void Test(char x, int n) {  
    cout << x << endl;  
    if (n == 0) {  
        return;  
    }  
    Test(x + 1, n - 1);  
}  
  
int main() {  
    Test('A', 21);  
    return 0;  
}
```



Code aus der Vorlesung

```
struct Element {
    int data = 0;
    Element* next = nullptr;
};

void Freigabe(Element* X) {
    if (X->next == nullptr) return; // Abbruchbedingung
    Freigabe(X->next); // Selbstaufzuruf
    delete X;
}
```

```
int main() {
    int n;
    cin >> n;
    Element* Anker = nullptr;
    Element* help = nullptr;
    for (int jj = 0; jj < n; jj++) {
        Element* neuer = new Element;
        neuer->data = jj;
        if (jj == 0) {
            Anker = neuer;
            help = neuer;
        }
        else {
            help->next = neuer;
            help = neuer;
        }
    }
    Element* tmp = Anker;
    while (tmp != nullptr) {
        cout << tmp->data << ", ";
        tmp = tmp->next;
    }
    Freigabe(Anker);
    return 0;
}
```

